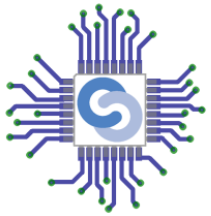


Ingineria Calculatoarelor
Fault Tolerant Computing

- Cursul 0 -

Facultatea de Automatică și Calculatoare
Universitatea Politehnica București

“If anything can go wrong, it will.”

Murphy’s Law

LET'S TEST
38
IN PRODUCTION

About me

Associate Professor @ UPB

- Research & Teaching
  - Computer architecture, hardware/software interaction
  - Embedded and Pervasive Computing
  - Wireless Sensor Networks
  - Low Power Computing Architectures
  - Fault tolerance
- Office: ED422
- Office Hours: Mondays 11:00am - 12:00pm
  - (almost) anytime on Teams

dan.tudose@upb.ro

Start-ups → Vector Watch → Fitbit → Google

Important Stuff

Laboratoare: ocw.cs.pub.ro/icalc

Notare:

- Proiect: 2p (20%)
- Laborator: 1p (10%)
- Test laborator: 2p (20%)
- Teste la curs: 2p (20%)
- Examenul final: 3p (30%)

Cerinte pentru a promova:

- minim 6 prezențe la laborator
- minim 2.5 puncte din cele 5 puncte pentru activitatea laborator
- minim 1.5 puncte din cele 3 puncte din examenul final

The Curse of Complexity

Computer engineering is the art and science of translating user requirements we do not fully understand; into hardware and software we cannot precisely analyze; to operate in environments we cannot accurately predict; all in such a way that the society at large is given no reason to suspect the extent of our ignorance.[1]

Microsoft Windows NT (1992): ≈4M lines of code
Microsoft Windows 10 (2015): ≈70M lines of code

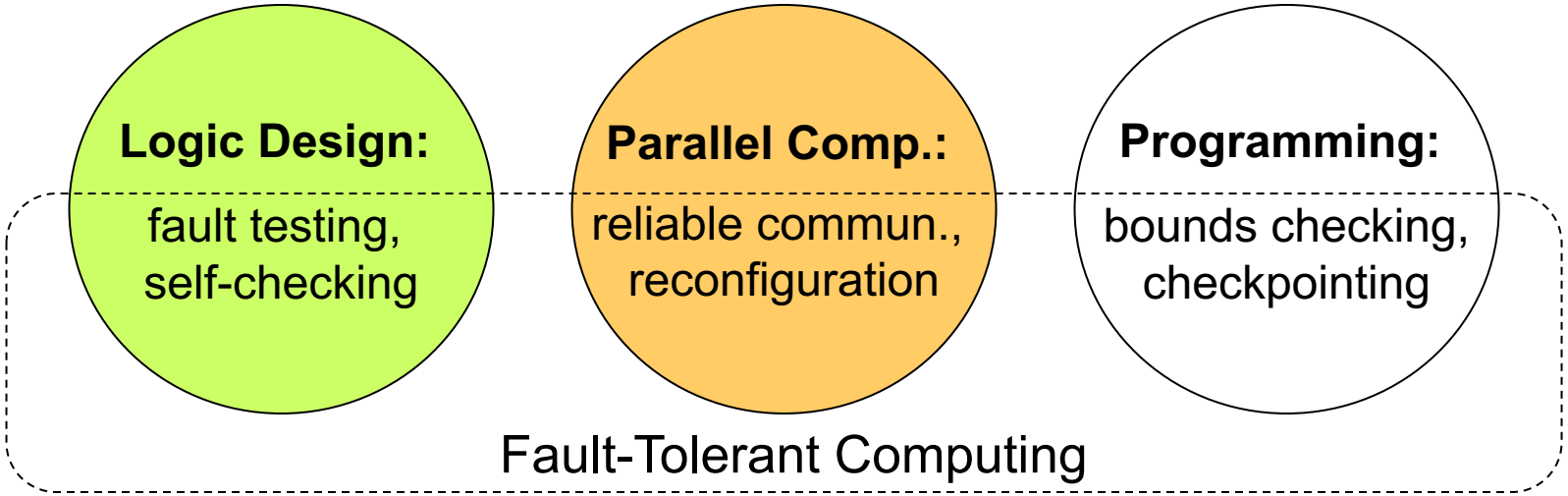
Intel Pentium processor (1993): ≈4M transistors
Intel Pentium 4 processor (2001): ≈40M transistors
Apple M2 (2022): ≈20B transistors

[1]Adapted from definition of structural engineering: Ralph Kaplan, *By Design: Why There Are No Locks on the Bathroom Doors in the Hotel Louis XIV and Other Object Lessons*, Fairchild Books, 2004, p. 229

Why This Course Shouldn't Be Needed

In an ideal world, methods for dealing with faults, errors, and other impairments in hardware and software would be covered within every computer engineering course that has a design component

Analogy: We do not teach structural engineers about building bridges in one course and about bridge safety and structural integrity during high winds or earthquakes in another (optional) course



Brief History of Dependable Computing

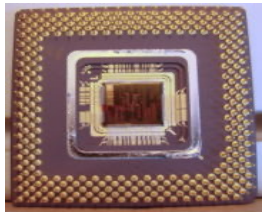
1940s: ENIAC, with 17.5K vacuum tubes and 1000s of other electrical elements, failed once every 2 days (avg. down time = minutes)

1950s: Early ideas by von Neumann (multichannel, with voting) and Moore-Shannon ("crummy" relays)

1960s: NASA and military agencies supported research for long-life space missions and battlefield computing

1970s: The field developed quickly (international conference, many research projects and groups, experimental systems)

1980s: The field matured (textbooks, theoretical developments, use of ECCs in solid-state memories, RAID concept), but also suffered some loss of focus and interest because of the extreme reliability of integrated circuits



1990s: Increased complexity at chip and system levels made verification, testing, and testability prime study topics

2000s: Resurgence of interest owing to less reliable fabrication at ultrahigh densities and "crummy" nanoelectronic components

Dependable Computing in the 2020s

There are still ambitious projects; space and elsewhere

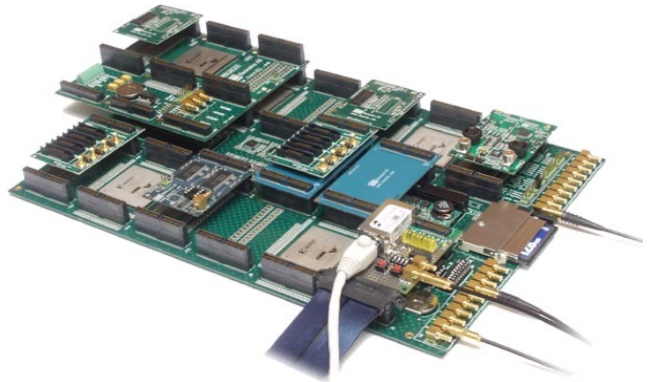
- Harsh environments (vibration, pressure, temperatures)
- External influences (radiation, micrometeoroids)
- Need for autonomy (commun. delays, unmanned probes)

The need is expanding

- More complex systems (e.g., system-on-chip)
- Critical applications (medicine, transportation, finance)
- Expanding pool of unsophisticated users
- Continued rise in maintenance costs
- Digital-only data (needs more rigorous backup)

The emphasis is shifting

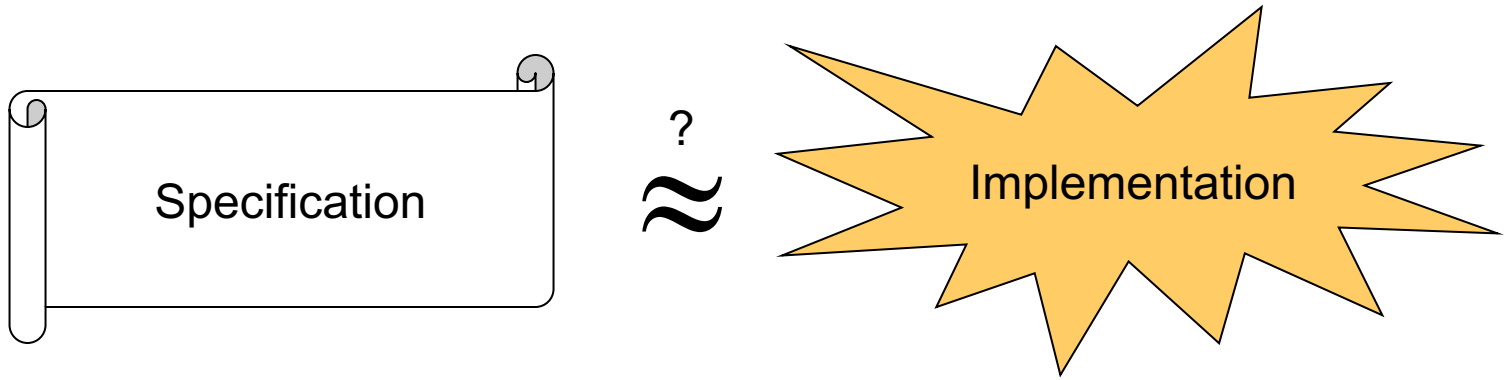
- Mostly COTS-based solutions
- Integrated hardware/software systems
- Entire units replaced (little diagnosis)



Defining Failure

Failure is an unacceptable difference between expected and observed performance.[1]

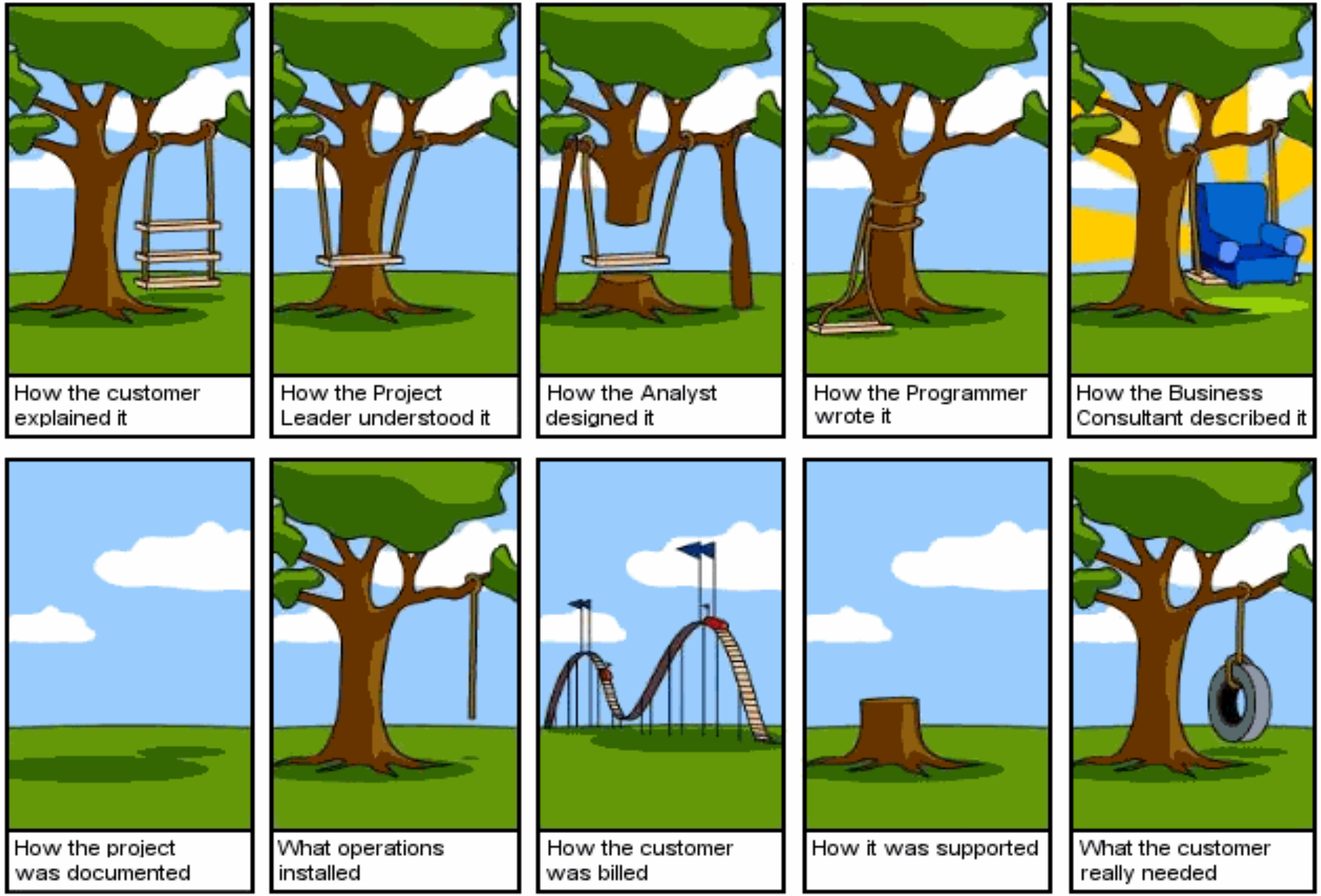
A structure (building or bridge) need not collapse catastrophically to be deemed a failure



Reasons of typical Web site failures

Hardware problems:	15%
Software problems:	34%
Operator error:	51%

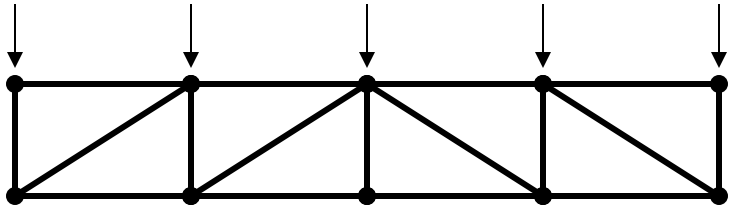
[1] Definition used by the Tech. Council on Forensic Engineering of the Amer. Society of Civil Engineers

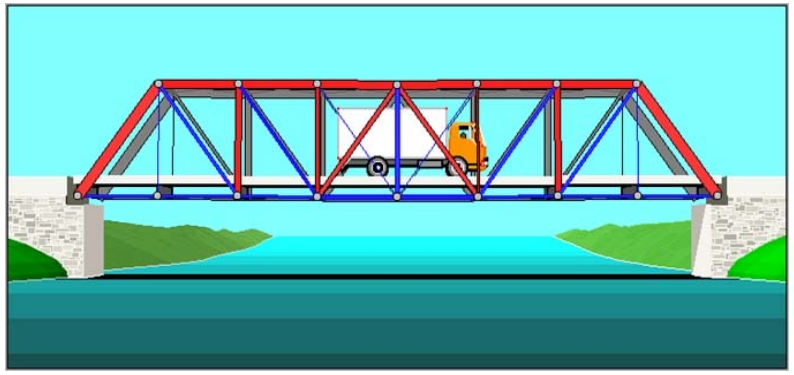

How the customer explained it
How the Project Leader understood it
How the Analyst designed it
How the Programmer wrote it
How the Business Consultant described it
How the project was documented
What operations installed
How the customer was billed
How it was supported
What the customer really needed

Design Flaws: "To Engineer is Human"[1]

Complex systems almost certainly contain multiple design flaws

Redundancy in the form of safety factor is routinely used in buildings and bridges





Example of a more subtle flaw:
Disney Concert Hall in Los Angeles reflected light into nearby building, causing discomfort for tenants due to blinding light and high temperature

[1] Title of book by Henry Petroski

Yet Another Example

The Walkie Talkie melted my Jag! Light reflected from under-construction City skyscraper buckles bodywork and mirror of businessman's car

- **Sunlight reflected from skyscraper is causing heat damage to cars beneath**
- **Several panels of a Jaguar XJ had buckled in the glare**
- **Other drivers say their vehicles have wilted in the beam of light**

By SAM WEBB

PUBLISHED: 18:56 GMT, 2 September 2013 | **UPDATED:** 22:39 GMT, 3 September 2013

Share Tweet +1 Share 800 View comments

A £200million skyscraper has had an undeniably dazzling effect on passers-by – but not, unfortunately, the one intended by its architect.

Learning Curve: "Normal Accidents"[1]

Example: Risk of piloting a plane

1903	First powered flight
1908	First fatal accident
1910	Fatalities = 32 (≈2000 pilots worldwide)
1918	US Air Mail Service founded Pilot life expectancy = 4 years 31 of the first 40 pilots died in service
1922	One forced landing for every 20 hours of flight
Today	Commercial airline pilots pay normal life insurance rates

Unfortunately, the learning curve for computers and computer-based systems is not as impressive

[1] Title of book by Charles Perrow (Ex. p. 125)

Mishaps, Accidents, and Catastrophes

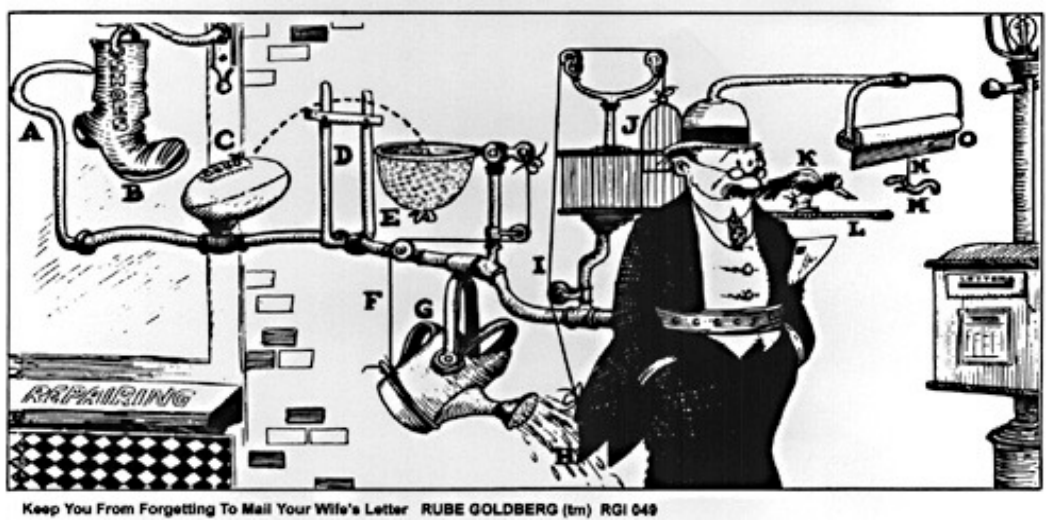
Mishap: misfortune; unfortunate accident

Accident: unexpected (no-fault) happening causing loss or injury

Catastrophe: final, momentous event of drastic action; utter failure

At one time (following the initial years of highly unreliable hardware), computer mishaps were predominantly the results of human error

Now, most mishaps are due to complexity (unanticipated interactions)



Rube Goldberg contraptions

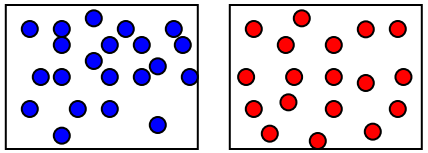
The butterfly effect

Pretest: Failures and Probabilities

This test will not be graded or even collected, so answer the test questions truthfully and to the best of your ability / knowledge

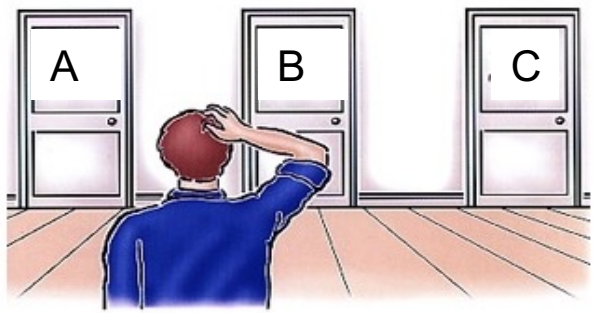
Question 1: Name a disaster that was caused by computer hardware or software failure. How do you define “disaster” and “failure”?

Question 2: Which of these patterns is more random?



Question 3: Which do you think is more likely: the event that everyone in this class was born in the first half of the year or the event that at least two people were born on the same day of the year?

Question 4: In a game show, there is a prize behind one of 3 doors with equal probabilities. You pick Door A. The host opens Door B to reveal that there is no prize behind it. The host then gives you a chance to switch to Door C.
Is it better to switch or to stick to your choice?



Pretest (Continued): Causes of Mishaps

Question 5: Does this photo depict a mishap due to design flaw, implementation bug, procedural inadequacies, or human error?

Pretest (Continued): Reliability and Risk

Question 6: Name an emergency backup system (something not normally used unless another system fails) that is quite commonplace

Question 7: Which is more reliable: plane X or plane Y that carries four times as many passengers as plane X and is twice as likely to crash?

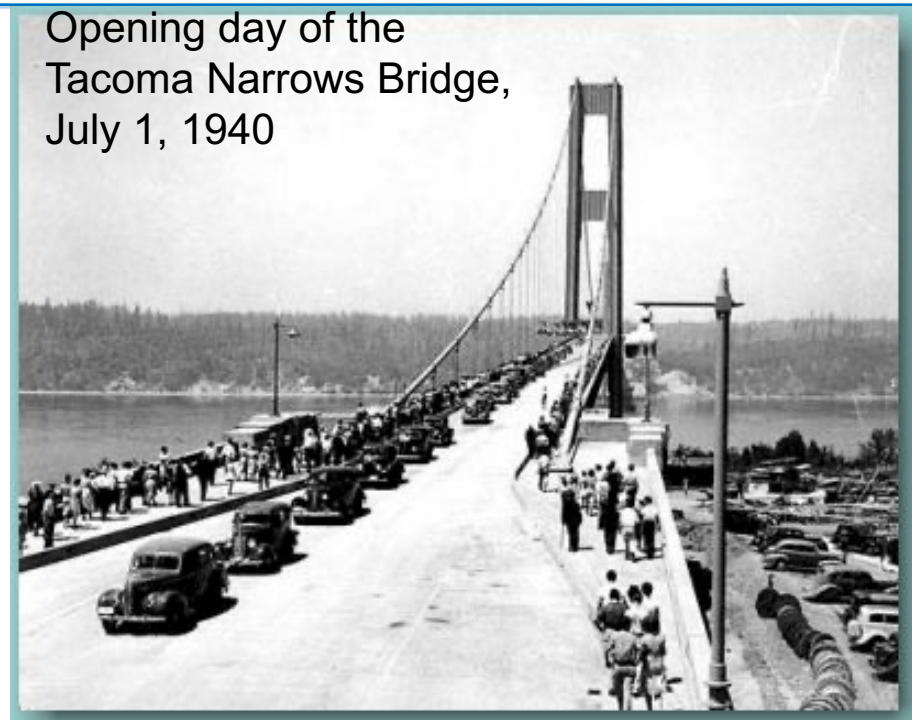
Question 8: Which is more reliable: a 4-wheel vehicle with one spare tire or an 18-wheeler with 2 spare tires?

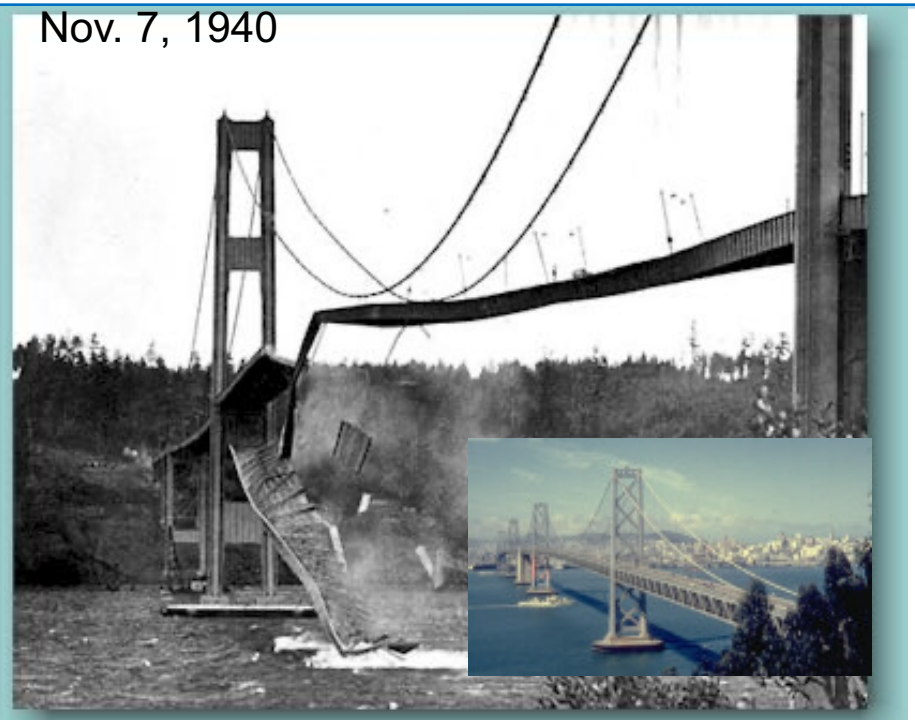
Question 9: Which surgeon would you prefer for an operation that you must undergo: Surgeon A, who has performed some 500 operations of the same type, with 5 of his patients perishing during or immediately after surgery, or surgeon B who has a perfect record in 25 operations?

Question 10: Which is more probable at your home or office: a power failure or an Internet outage? Which is likely to last longer?

If you had trouble with 3 or more questions, you really need this course!

What Do We Learn from Bridges that Collapse?





One catastrophic bridge collapse every 30 years or so

See the following amazing video clip (Tacoma Narrows Bridge):
http://www.enm.bris.ac.uk/research/nonlinear/tacoma/tacnarr.mpg

" . . . failures appear to be inevitable in the wake of prolonged success, which encourages lower margins of safety. Failures in turn lead to greater safety margins and, hence, new periods of success."

Henry Petroski, *To Engineer is Human*

. . . or from “Unsinkable” Ships that Sink?

“The major difference between a thing that might go wrong and a thing that cannot possibly go wrong is that when a thing that cannot possibly go wrong goes wrong, it usually turns out to be impossible to get at or repair.”

Douglas Adams, author of *The Hitchhiker’s Guide to the Galaxy*

… or from Poorly Designed High-Tech Trains?

Train built for demonstrating magnetic levitation technology in northwest Germany rams into maintenance vehicle left on track at 200 km/h, killing 23 of 29 aboard

Official investigation blames the accident on human error (train was allowed to depart before a clearance phone call from maintenance crew)

Not a good explanation; even low-tech trains have obstacle detection systems

Even if manual protocol is fully adequate under normal conditions, any engineering design must take unusual circumstances into account (abuse, sabotage, terrorism)

Design Flaws in Computer Systems

Hardware example: Intel Pentium processor, 1994

For certain operands, the FDIV instruction yielded a wrong quotient

Amply documented and reasons well-known (overzealous optimization)

Software example: Patriot missile guidance, 1991

Missed intercepting a scud missile in 1st Gulf War, causing 28 deaths

Clock reading multiplied by 24-bit representation of 1/10 s (unit of time) caused an error of about 0.0001%; normally, this would cancel out in relative time calculations, but owing to ad hoc updates to some (not all) calls to a routine, calculated time was off by 0.34 s (over ≈100 hours), during which time a scud missile travels more than ½ km

User interface example: Therac 25 machine, mid 1980s[1]

Serious burns and some deaths due to overdose in radiation therapy

Operator entered "x" (for x-ray), realized error, corrected by entering "e" (for low-power electron beam) before activating the machine; activation was so quick that software had not yet processed the override

[1] Accounts of the reasons vary

Causes of Human Errors in Computer Systems

1. Personal factors (35%): Lack of skill, lack of interest or motivation, fatigue, poor memory, age or disability

2. System design (20%): Insufficient time for reaction, tedium, lack of incentive for accuracy, inconsistent requirements or formats

3. Written instructions (10%): Hard to understand, incomplete or inaccurate, not up to date, poorly organized

4. Training (10%): Insufficient, not customized to needs, not up to date

5. Human-computer interface (10%): Poor display quality, fonts used, need to remember long codes, ergonomic factors

6. Accuracy requirements (10%): Too much expected of operator

7. Environment (5%): Lighting, temperature, humidity, noise

Because “the interface is the system” (according to a popular saying), items 2, 5, and 6 (40%) could be categorized under user interface

NO! - Bad User!!!
You've been warned 3 times that this file does not exist.
Now you've made us catch this worthless exception and we're upset.
Do not do this again.
OK

Big error
You really screwed up this time.
I know
What else is new
It's my first day

http://xkcd.com/292/

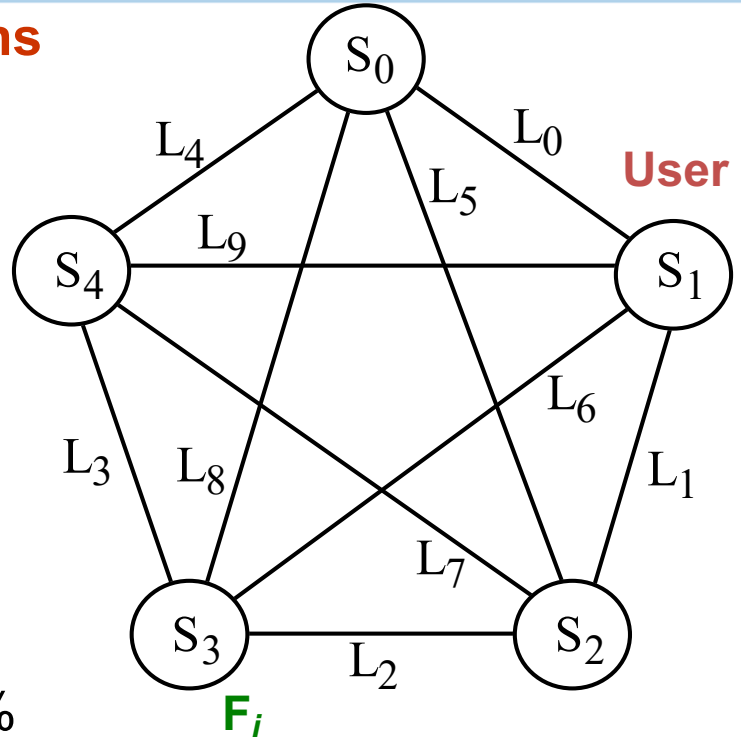
A Case Study

Data availability and integrity concerns

Distributed DB system with 5 sites
Full connectivity, dedicated links
Only direct communication allowed
Sites and links may malfunction
Redundancy improves availability

S: Probability of a site being available
L: Probability of a link being available

Single-copy availability = *SL*
Unavailability = 1 – *SL*
= 1 – 0.99 × 0.95 = 5.95%



Data replication methods, and a challenge

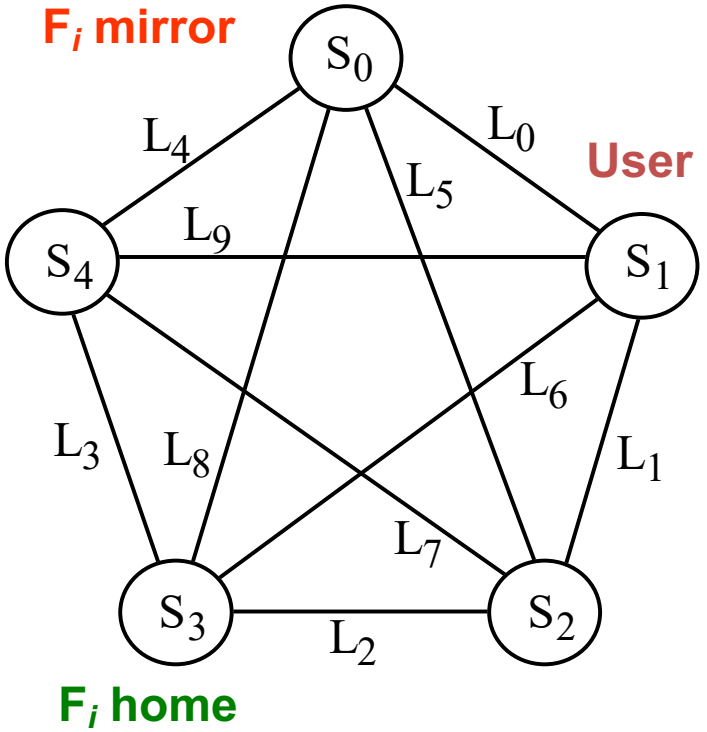
File duplication: home / mirror sites
File triplication: home / backup 1 / backup 2
Are there availability improvement methods with less redundancy?

Data Duplication: Home and Mirror Sites

S: Site availability e.g., 99%
L: Link availability e.g., 95%

$$A = SL + (1 - SL)SL$$

- SL: Primary site can be reached
- SL: Mirror site can be reached
- $(1 - SL)$: Primary site inaccessible



Duplicated availability = $2SL - (SL)^2$
Unavailability = $1 - 2SL + (SL)^2$
= $(1 - SL)^2$ = 0.35%

Data unavailability reduced from 5.95% to 0.35%

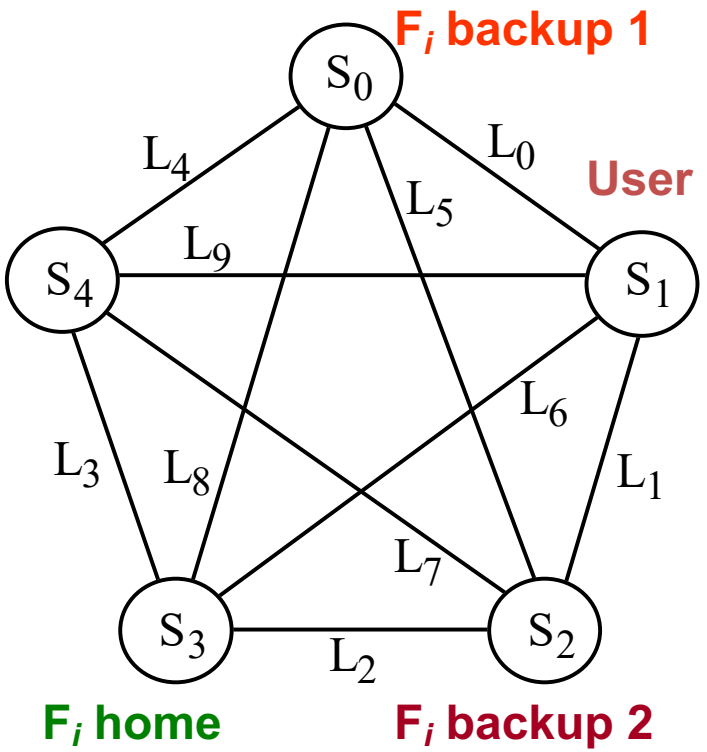
Availability improved from ≈ 94% to 99.65%

Data Triplication: Home and Two Backups

S: Site availability e.g., 99%
L: Link availability e.g., 95%

$$A = SL + (1 - SL)SL + (1 - SL)^2SL$$

- SL: Primary site can be reached
- $(1 - SL)SL$: Backup 1 can be reached; Primary site inaccessible
- $(1 - SL)^2SL$: Backup 2 can be reached; Primary and backup 1 inaccessible

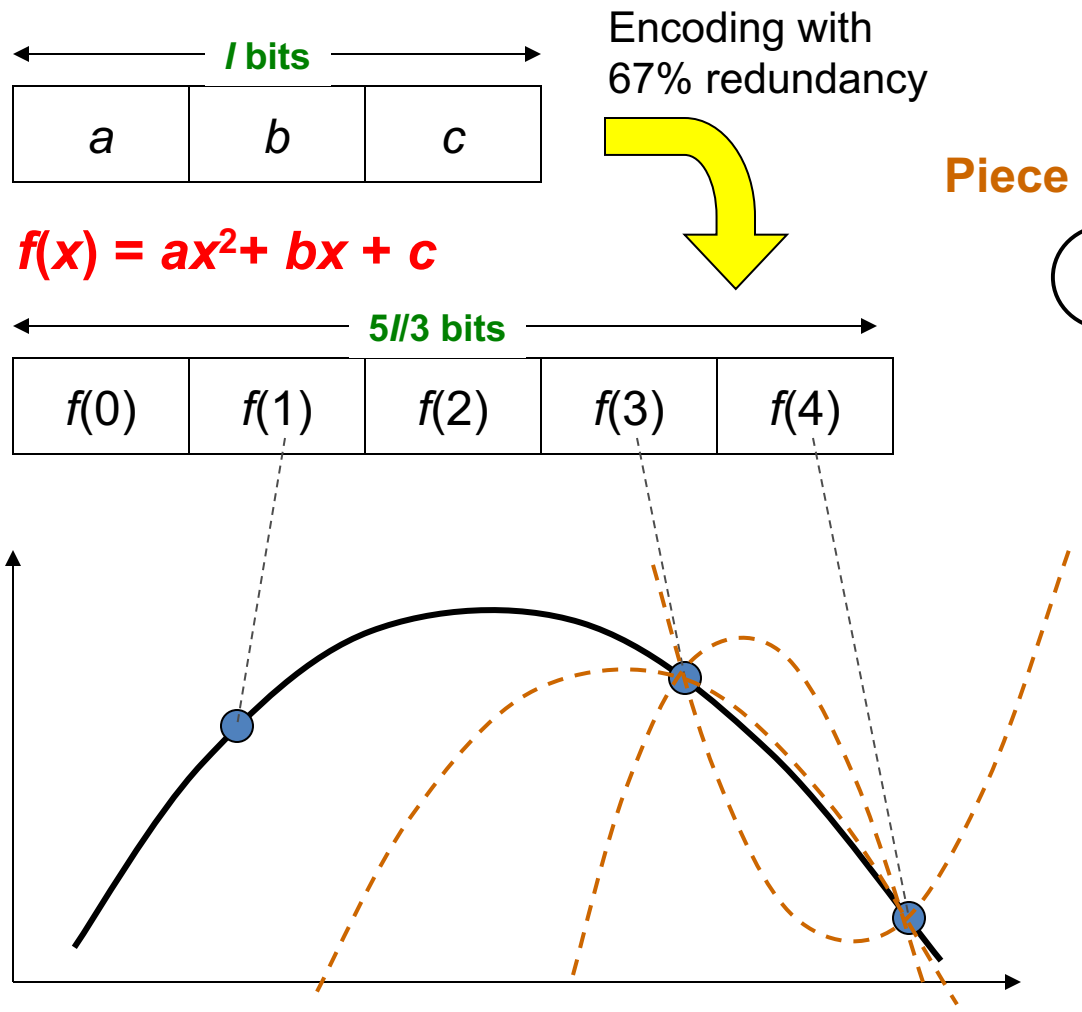


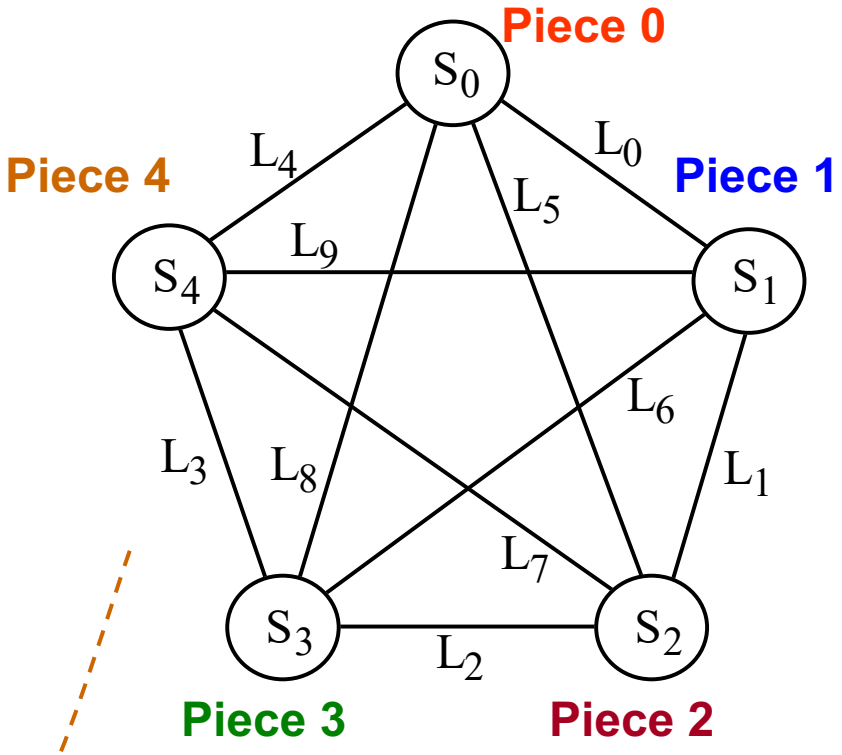
Triplicated avail. = $3SL - 3(SL)^2 - (SL)^3$
Unavailability = $1 - 3SL - 3(SL)^2 + (SL)^3$
= $(1 - SL)^3$ = 0.02%

Data unavailability reduced from 5.95% to 0.02%

Availability improved from ≈ 94% to 99.98%

Dispersion for Data Security and Integrity





Note that two pieces would be inadequate for reconstruction

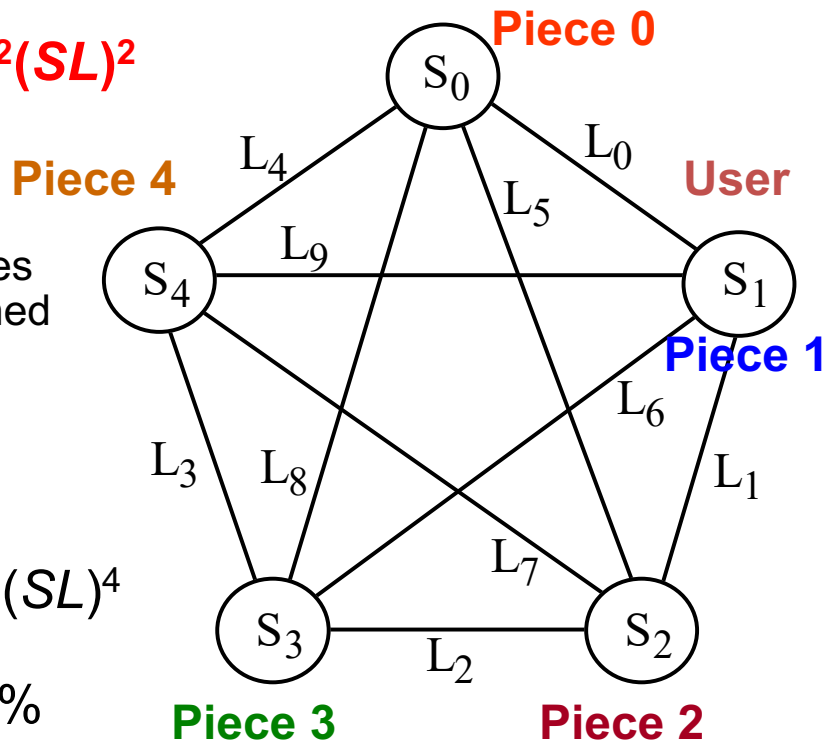
Data Dispersion: Three of Five Pieces

$$A = (SL)^4 + 4(1 - SL)(SL)^3 + 6(1 - SL)^2(SL)^2$$

All 4 pieces can be reached

Exactly 3 pieces can be reached

Only 2 pieces can be reached



S: Site availability e.g., 99%
L: Link availability e.g., 95%

Dispersed avail. $= 6(SL)^2 - 8(SL)^3 + 3(SL)^4$
Availability = 99.92%
Unavailability = 1 – Availability = 0.08%

Scheme →	Nonredund.	Duplication	Triplication	Dispersion
Unavailability	5.95%	0.35%	0.02%	0.08%
Redundancy	0%	100%	200%	67%

Questions Ignored in Our Simple Example

1. How redundant copies of data are kept consistent
When a user modifies the data, how to update the redundant copies (pieces) quickly and prevent the use of stale data in the meantime?

2. How malfunctioning sites and links are identified
Malfunction diagnosis must be quick to avoid data contamination

3. How recovery is accomplished when a malfunctioning site / link returns to service after repair
The returning site must be brought up to date with regard to changes

4. How data corrupted by the actions of an adversary is detected This is more difficult than detecting random malfunctions

The example does demonstrate, however, that:

- Many alternatives are available for improving dependability
- Proposed methods must be assessed through modeling
- The most cost-effective solution may be far from obvious

Software Development Processes

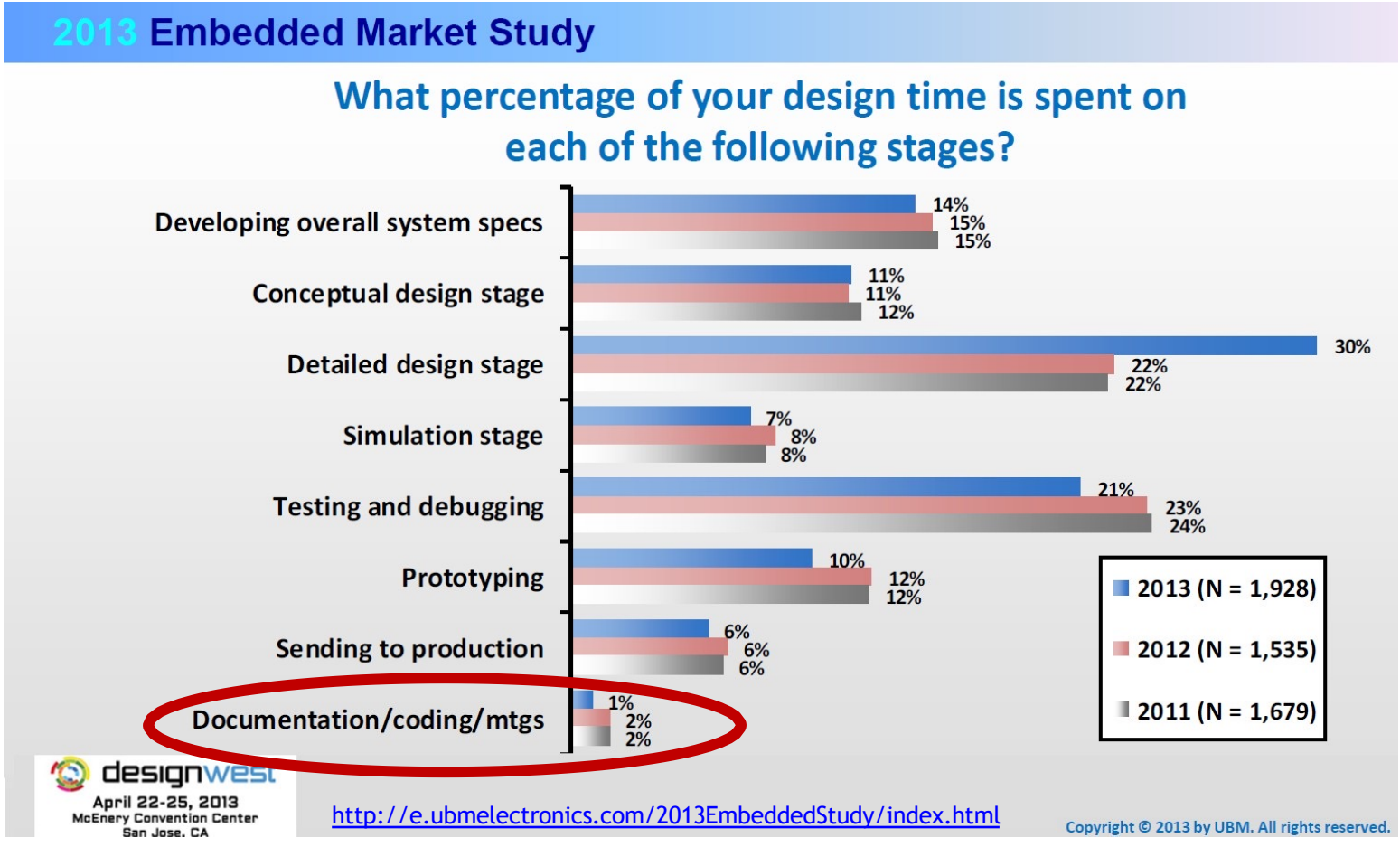
"Without requirements and design, programming is the art of adding bugs to an empty text file."

— *Louis Srygley*

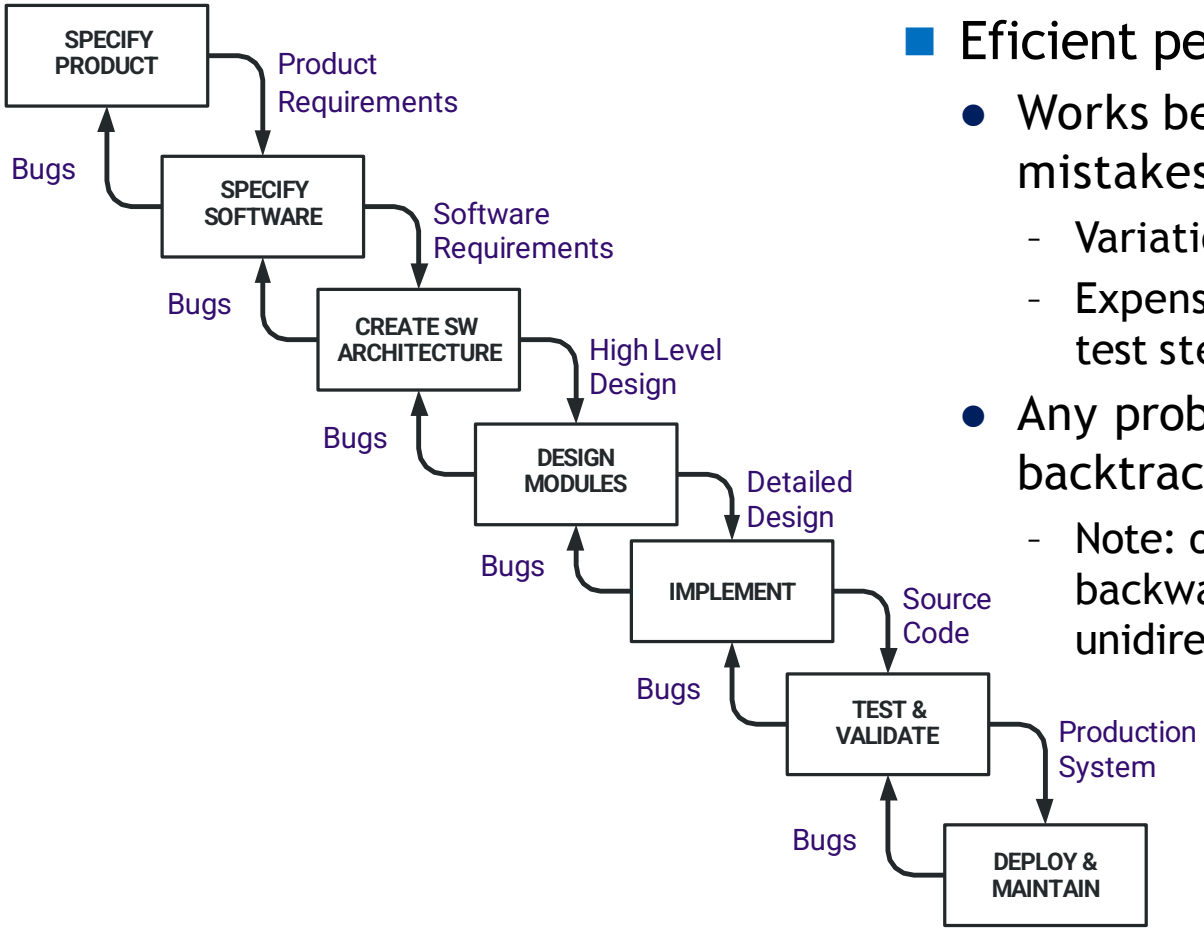
There once was a coder so bright,
Whose code was impressively tight,
But for extra reliability,
They asked for code review with civility,
Lest their code end up a spaghetti-like sight!

- *ChatGPT 2023*

Coding Is Essentially 0% of Creating Software



Old-School Waterfall Development Cycle



- Eficient pentru domenii bine înțelese
  - Works best if you don't make many big mistakes
    - Variations on existing systems
    - Expensive to fix things that escape to test steps
  - Any problem encountered requires backtracking
    - Note: original waterfall paper had these backward arrows! It was never just a unidirectional process

What We've Learned in 50+ Years of Software

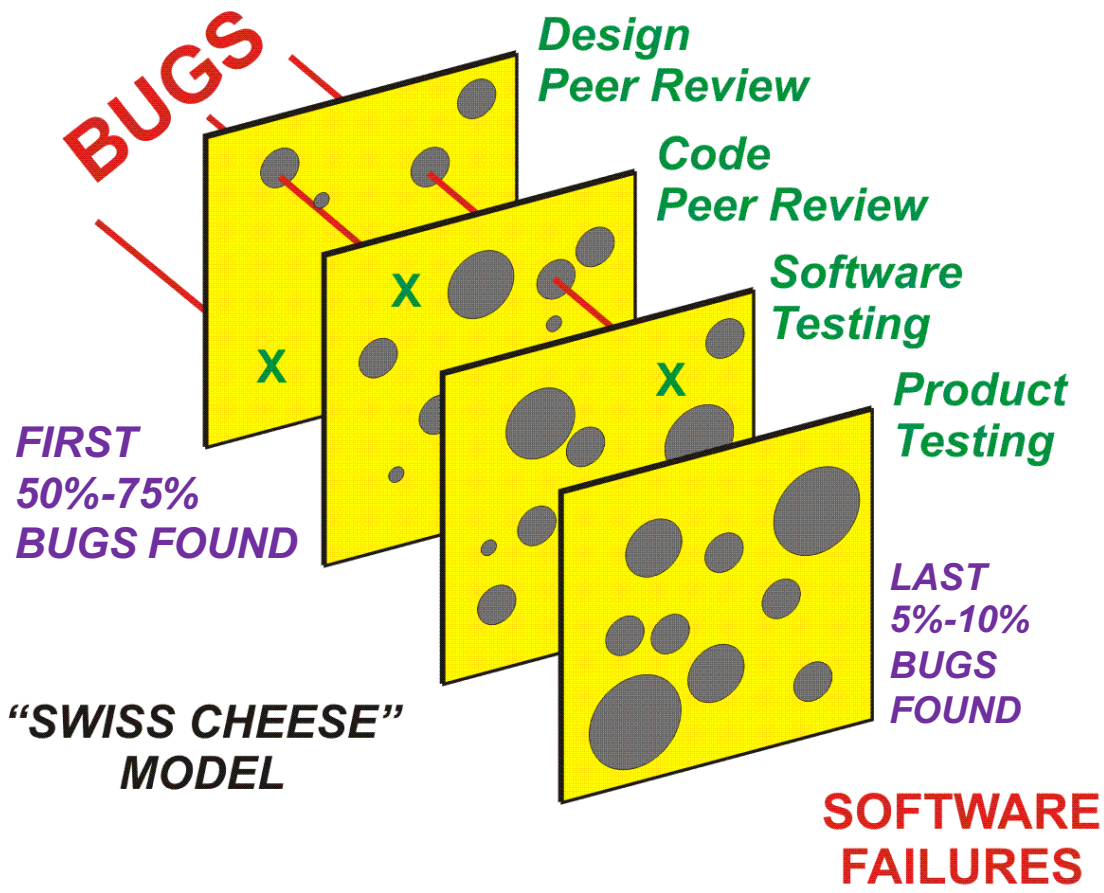
- Dividing up into subsystems is critical
  - Bad architecture will doom a project
- Process formality is a good investment
  - Traceability, formal reviews, etc.
  - Skipping steps costs more in the end
- Requirements change
  - Suggests using an iterated approach
- Finding bugs early is important
  - Traceability from high to low levels
  - Layered testing
  - Peer reviews most cost effective for this

- **If the second half of the project is "debugging" that must mean the first half is "bugging"**

— Jack Ganssle

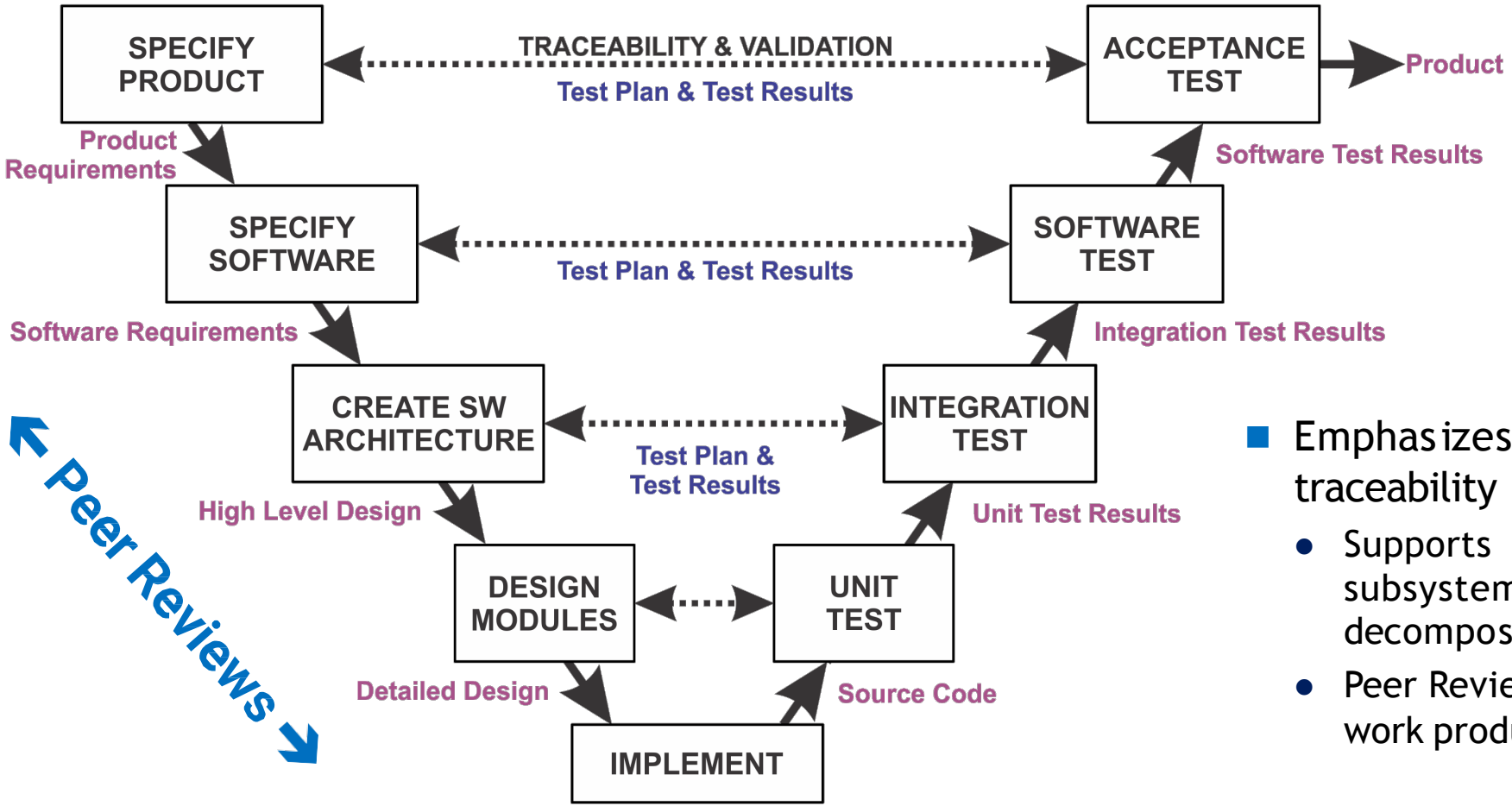
http://www.ganssle.com/rants/ontesting.htm (paraphrase)

Finding Bugs Before Product Test



- **Product Testing**
  - Late & Expensive
  - Many field escapes
- **Software Testing**
  - Unit & Integration test
- **Code Peer Review**
  - Earlier & Cheaper
- **Design Peer Review**
  - Earlier & Cheaper

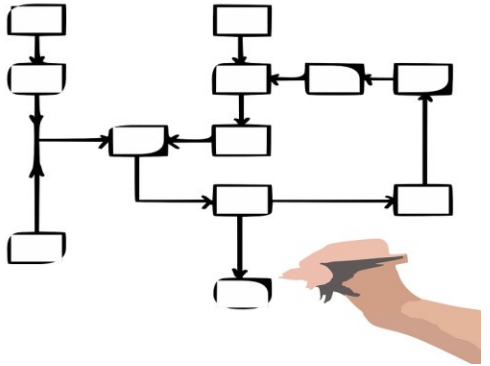
V (or "Vee") Development Cycle



- Emphasizes traceability
  - Supports subsystem decomposition
  - Peer Reviews of work products

A Design Is Not The Code

- Implementation: the code itself
  - Comments describe the implementation; they aren't the design
- Detailed Design (DD)
  - Flowcharts
  - Statecharts
  - Algorithms, control diagrams, etc.



https://pixabay.com/en/flowchart-diagram-drawing-concept-311347/

- High Level Design (HLD): architecture, component defs.
  - Pieces of the system (e.g., classes, subsystems)
  - Functional allocation to the pieces
  - Interfaces between the systems

Requirements on Top Left of Vee

- Software Requirements Specification (SRS)
  - Says “what” the software does, not “how” it does it
    - If it’s not in the SRS, the software shouldn’t do it
    - Avoids details unless mandatory due to marketing reqts.
  - Often paired with a Hardware Requirements Spec.

- Product Requirements Specification (PRS)
  - Market-facing product requirements
    - What the system does from a user point of view
  - Point of interface between software group and others
    - Might just be a feature list
    - Might be in form of customer-specified acceptance test

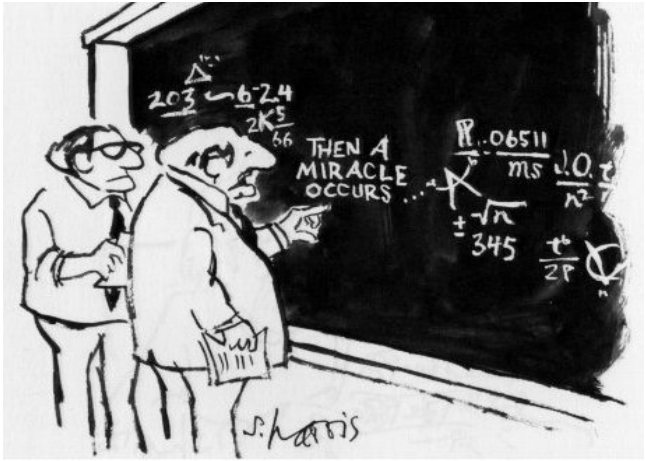
If you think good design is expensive, you should look at the cost of <u>bad</u> design!

https://youtu.be/j-zczJXSxnw

https://goo.gl/ZVRH9Y

Verification & Validation on Right Side of Vee

- Unit Test: Traces to DD
  - Test individual subroutines, procedures, “modules”
- Integration Test: Traces to HLD
  - Test module interactions (e.g., sequence diagrams)
- Software Test: Traces to SRS
  - Test functionality knowing how software is built
- Acceptance Test: Traces to PRS
  - Test customer-facing functionality
- Other activities:
  - Software Quality Assurance (SQA): did you follow the steps?
  - Peer Reviews: check quality of every step
  - Regression Test: test after bug fix to make sure bugs stay dead



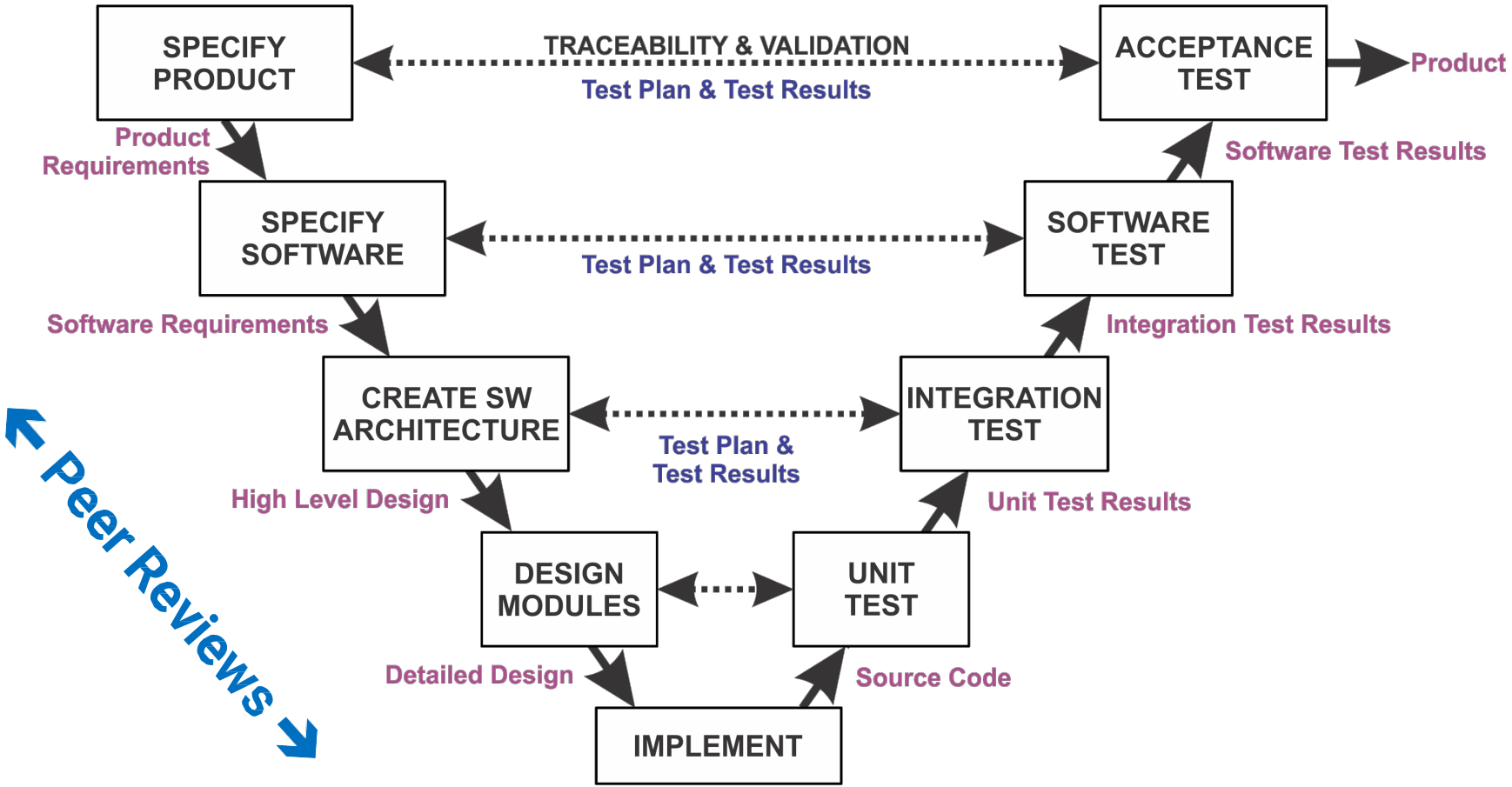
“I think you should be more explicit here in step two.”

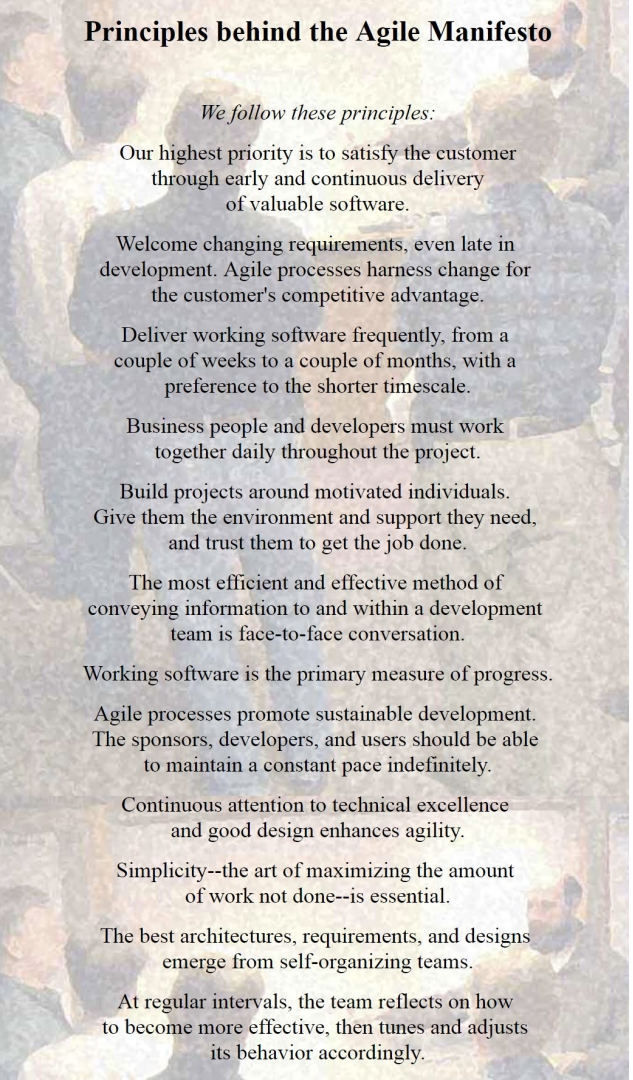
from *What's so Funny about Science?* by Sidney Harris (1977)

How Much "Paper" Is Enough?

- Old military development saying:
  - Deploy when the paper is heavier than the system. (Even aircraft carriers!)
- Does all this mean you need to be buried in paper? No.
  - Paper required to check process health
    - Be clever about minimizing paper bulk
    - But if code has *no* paperwork, throw the code out
  - Put things on paper as you go through the Vee
    - "Documentation" after writing code is really inefficient
    - If you aren't going to maintain paper, throw it out

Review: How Do the Pieces Fit Together?





Agile Methods

- Agile generally values:
  - Individuals and Interactions over processes and tools
  - Working Software over comprehensive documentation
  - Customer Collaboration over contract negotiation
  - Responding to Change over following a plan

- Example: Scrum
  - Daily "stand up" ("scrum") meetings for face-to-face collaboration
  - 2-4 week long sprints to incrementally add functionality
    - Each sprint implements items from a backlog
    - Demo at end of sprint; theoretically a shippable product
  - User stories serve as requirements
  - Scrum challenges
    - Geographically split teams with informal communication
    - External dependencies (e.g., other parts of system change)
    - No time for extensive testing, especially embedded hardware

http://agilemanifesto.org/principles.html

Introduction to Scrum

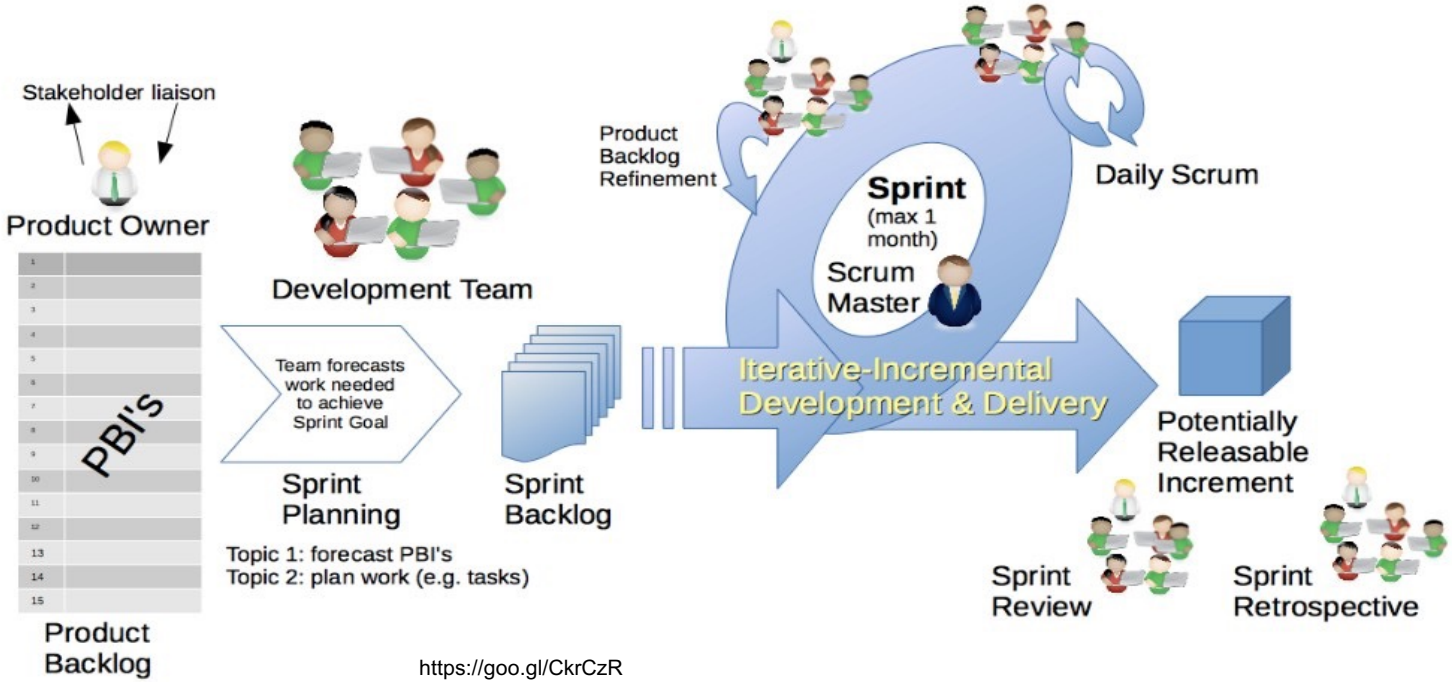
A 7 minute training

Please watch this video:
https://youtu.be/9TycLR0TqFA

by Steve Stedman

Scrum Process Example

- Heavy on implicit knowledge
  - Where are the: requirements, design, test plan, acceptance test



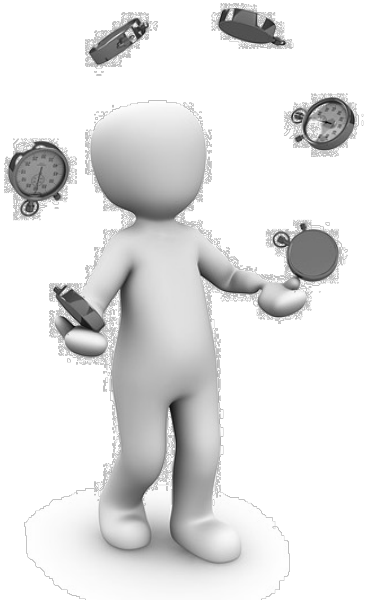
https://goo.gl/CkrCzR

When Is Agile a Good Fit?

Source: Boehm & Turner 2004, Balancing Agility and Discipline

- Agile:
  - Small teams; small products
  - "Everyday" software quality
  - Fast requirements change
  - High-skill experts throughout project
    - Including life-cycle maintenance
  - Developers can handle being empowered; usually senior
- Plan-Driven (waterfall; V)
  - Large teams; large products
  - Mission-critical products
  - Stable requirements
  - High skill primarily in design phase
    - Major versions require expert design
  - Most developers are not empowered; usually junior

Agile Methods + Embedded (?)

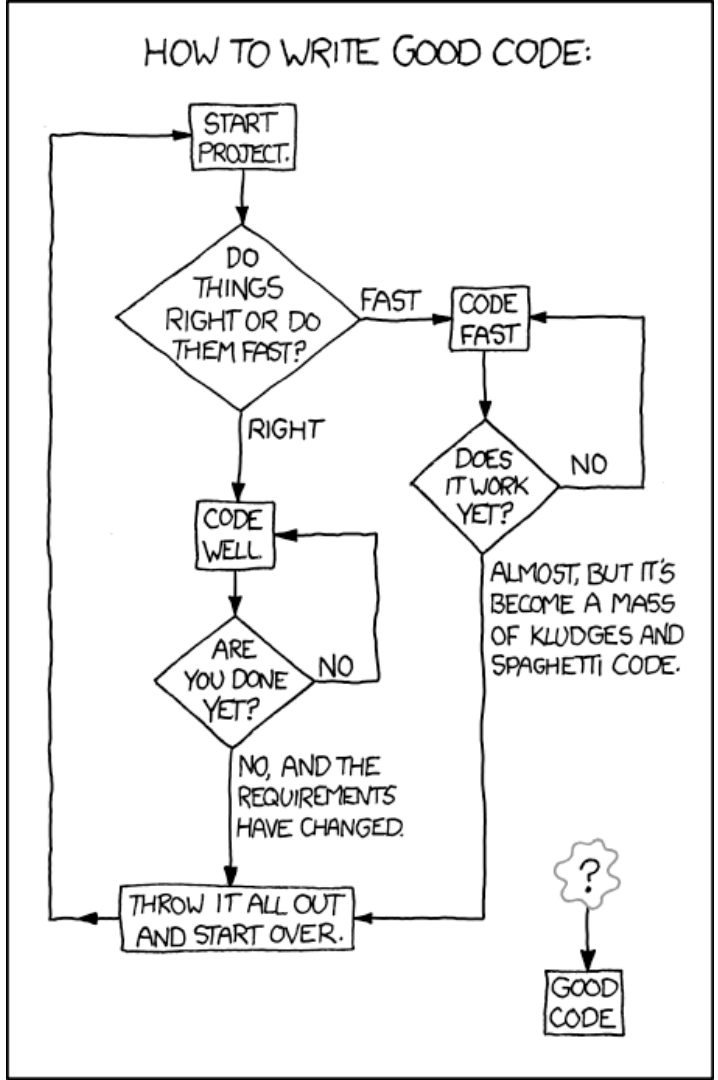


- Significant benefit is that it makes (good) developers happier
  - If done well can help with evolving requirements
  - But, but you need to manage and moderate the risks
- Issue: “Agile” is not just cowboy coding
  - Undefined, undisciplined processes are bad news
  - Yes, Agile teams should follow a rigorously defined process
- Issue: “No-paper” Agile unsuitable for long-lived systems
  - Implicit knowledge is efficient, but evaporates with the team
  - 10+ year old undocumented legacy systems are a nightmare
- Issue: Agile assumes 100% automated acceptance test
  - 100% automated system test is often impractical for physical interfaces
  - Often implicitly assumes that defect escapes are low cost because a new version is 2-4 weeks away
- Issue: Agile typically doesn’t have independent process monitoring (SQA)
  - Software Quality Assurance (SQA) tells you if your process is working
  - Agile teams may be dysfunctional and have no idea this is happening
    - Or they may be fine – but who knows if they are really healthy or not?

Best Practices For Software Process

- Follow a defined process
  - Must include all aspects shown on Vee
    - And SQA, Peer Reviews
  - It's OK to rename and reorganize steps
    - All the steps have to get done
    - Common to see "AgileFall" etc.
    - Also common to see bad process dressed up with the latest buzzwords

- Software Process Pitfalls
  - Skipping steps to get to testing faster means more bugs in test
    - Finding bugs is more expensive in testing
  - Using the wrong process for the wrong purpose
    - 3-Week product life and 30 year product life are different situations



https://xkcd.com/844/

Disclaimer

This lecture contains materials from:

- Philip Koopman - CMU

Coding Style

“Any fool can write code that a computer can understand. Good programmers write code that humans can understand.” – Martin Fowler

Coding Style: Understandability

- Anti-Patterns:
  - "Style doesn't matter; it passes all the tests"
  - Code that is clever instead of clear

> "There are two ways of constructing a software design: one way is to make it so simple that there are **obviously no deficiencies** and the other way is to make it so complicated that there are **no obvious deficiencies**."
>
> — C.A.R. (Tony) Hoare, 1980 Turing Award Talk

- Other people must understand your code
  - Peer reviews won't work if nobody can read your code
    - Write code so that others can tell it is obviously correct
  - If others can't understand it, they will inject bugs
  - If it's not obviously correct, then it's wrong.

Make Code Easy To Read

- Consistent formatting
  - Consistent indentation, braces
  - Templated headers for files and functions
  - Spaces and “()” to avoid precedence confusion
  - Use space instead of tab
- Comments
  - Explain what & why, not just code paraphrase
  - Comments are not a design
- Naming
  - Descriptive, consistent naming conventions
    - E.g., variables are nouns; functions are verbs
- Avoid magic numbers (use const)
  - Avoid macros (use inline)

Obfuscated C Winner: Flight Simulator

```
#include                                    <math.h>
#include                                    <sys/time.h>
#include                                    <X11/Xlib.h>
#include                                    <X11/keysym.h>
                                            double L ,o ,P
                                           ,_=dt,T,Z,D=1,d,
                                           s[999],E,h= 8,I,
                                           J,K,w[999],M,m,O
                                           ,n[999],j=33e-3,i=
                                           1E3,r,t, u,v ,W,S=
                                           74.5,l=221,X=7.26,
                                           a,B,A=32.2,c, F,H;
                                           int N,q, C, y,p,U;
                                          Window z; char f[52]
                                        ; GC k; main(){ Display*e=
 XOpenDisplay( 0); z=RootWindow(e,0); for (XSetForeground(e,k=XCreateGC (e,z,0,0),BlackPixel(e,0))
 ; scanf("%lf%lf%lf",y +n,w+y, y+s)+1; y ++); XSelectInput(e,z= XCreateSimpleWindow(e,z,0,0,400,400,
 0,0,WhitePixel(e,0) ),KeyPressMask); for(XMapWindow(e,z); ; T=sin(O)){ struct timeval G={ 0,dt*1e6}
 ; K= cos(j); N=1e4; M+= H*_; Z=D*K; F+=_*P; r=E*K; W=cos( O); m=K*W; H=K*T; O+=D*_*F/ K+d/K*E*_; B=
 sin(j); a=B*T*D-E*W; XClearWindow(e,z); t=T*E+ D*B*W; j+=d*_*D-_*F*E; P=W*E*B-T*D; for (o+=(I=D*W+E
 *T*B,E*d/K *B+v+B/K*F*D)*_; p<y; ){ T=p[s]+i; E=c-p[w]; D=n[p]-L; K=D*m-B*T-H*E; if(p [n]+w[ p]+p[s
 ]== 0|K <fabs(W=T*r-I*E +D*P) |fabs(D=t *D+Z *T-a *E)> K)N=1e4; else{ q=W/K *4E2+2e2; C= 2E2+4e2/ K
  *D; N-1E4&& XDrawLine(e ,z,k,N ,U,q,C); N=q; U=C; } ++p; } L+=_* (X*t +P*M+m*l); T=X*X+ l*l+M *M;
   XDrawString(e,z,k ,20,380,f,17); D=v/l*15; i+=(B *l-M*r -X*Z)*_; for(; XPending(e); u *=CS!=N){
                                          XEvent z; XNextEvent(e ,&z);
                                              ++*((N=XLookupKeysym
                                                (&z.xkey,0))-IT?
                                                N-LT? UP-N?& E:&
                                                J:& u: &h); --*(
                                                DN -N? N-DT ?N==
                                                RT?&u: & W:&h:&J
                                                 ); } m=15*F/l;
                                                 c+=(I=M/ l,l*H
                                                 +I*M+a*X)*_; H
                                                 =A*r+v*X-F*l+(
                                                 E=.1+X*4.9/l,t
                                                 =T*m/32-I*T/24
                                                  )/S; K=F*M+(
                                                  h* 1e4/l-(T+
                                                  E*5*T*E)/3e2
                                                  )/S-X*d-B*A;
                                                  a=2.63 /l*d;
                                                  X+=( d*l-T/S
                                                   *(.19*E +a
                                                   *.64+J/1e3
                                                   )-M* v +A*
                                                   Z)*_; l +=
                                                   K *_; W=d;
                                                   sprintf(f,
                                                   "%5d  %3d"
                                                   "%7d",p =l
                                                   /1.7,(C=9E3+
 O*57.3)%0550,(int)i); d+=T*(.45-14/l*
X-a*130-J* .14)*_/125e2+F*_*v; P=(T*(47
*I-m* 52+E*94 *D-t*.38+u*.21*E) /1e2+W*
179*v)/2312; select(p=0,0,0,0,&G); v-=(
 W*F-T*(.63*m-I*.086+m*E*19-D*25-.11*u
  )/107e2)*_; D=cos(o); E=sin(o); } }
```

http://blog.aerojockey.com/post/iocccsim

Good Code Hygiene

Modularity

- Many smaller .c/.cpp files (one per class)
- Externally visible declarations into .h file

Conditional Statements

- Boolean conditional expression results; no assignments
- All switch statements have a default (usually error trap)
- Limited nesting (see also cyclomatic complexity)

Variables

- Descriptive names that differ significantly
- Smallest practicable scope for variables; initialize at point of definition
- Use typedefs to define narrow types (also use uint32_t, use enum, etc.)
- Range checks & bounds checks (e.g., buffer overflow)

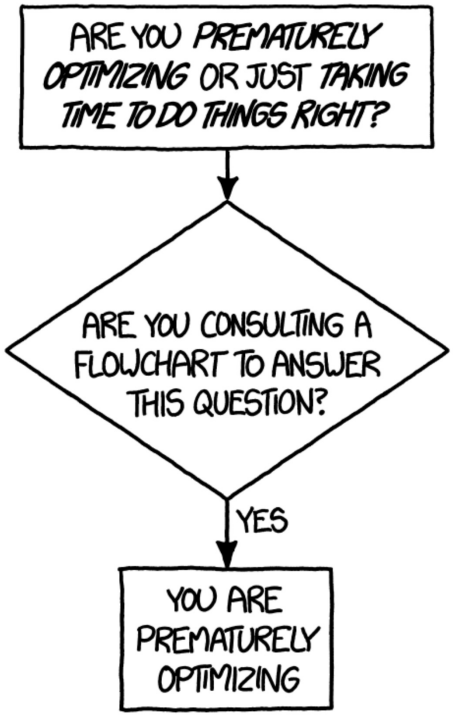
Handle errors returned by called functions

Optimization

"We should forget about small efficiencies, say about 97% of the time: premature optimization is the root of all evil. Yet we should not pass up our opportunities in that critical 3%"

- Donald Knuth (December 1974). "Structured Programming with go to Statements". ACM Journal Computing Surveys 6 (4): 268.

- Don't optimize unless you have performance data
  - Most code doesn't matter for speed
  - Use little or no assembly language. Get a better compiler.
- Optimization makes it hard to know your code is right
  - Do you want correct code or tricky code?
    - (Pick one. Which one is safer?)
  - Buy a bigger CPU if you have to

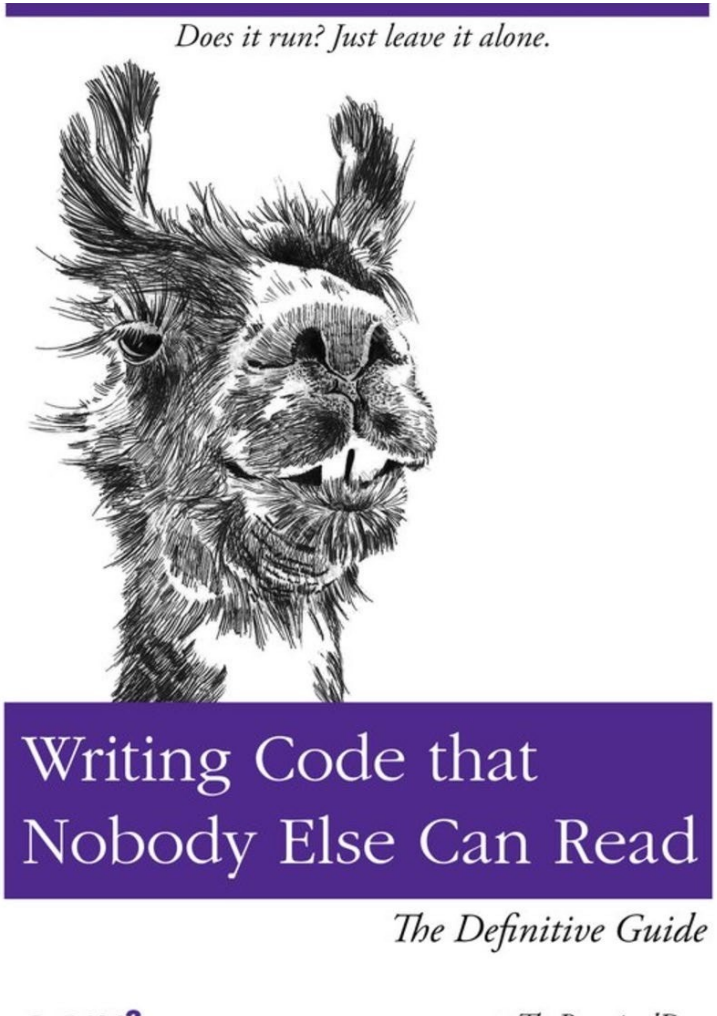


https://xkcd.com/1691/

Coding Understandability Best Practices

- Pick a coding style and follow it
  - Use tool support for language formatting
  - Evaluate naming as part of peer review
  - Comments are there to explain implementation
- The point of good style is to avoid bugs
  - Make it hard for a reviewer to miss a problem
    - Even better, make it easy for a tool to find problem
  - Also need to have a good technical style
- Coding style pitfalls:
  - Optimizing for the author instead of the reviewer
  - Making it too easy to deviate from style rules

Great style depends upon point of view.



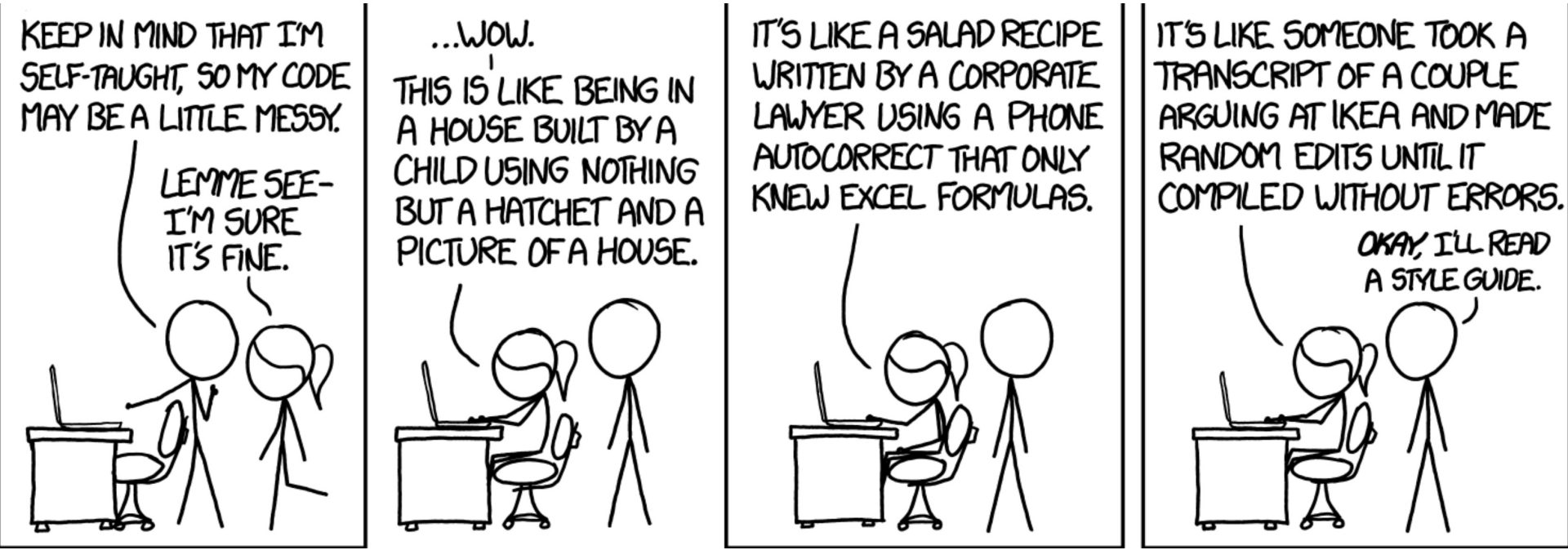
https://goo.gl/pvDMHX CC BY-NC 2.0

“Always code as if the guy who ends up maintaining your code will be a violent psychopath who knows where you live.

Code for readability.”

(Author unclear)

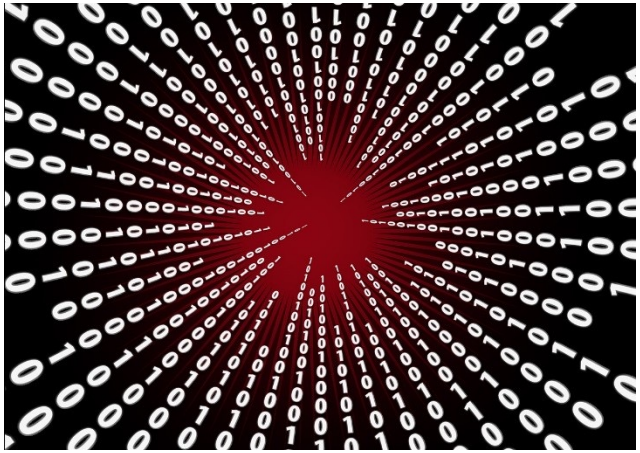
© 2020 Philip Koopman



https://xkcd.com/1513/

Coding Style: Language Use

- Anti-Patterns:
  - Code compiles with warnings
  - Warnings are turned off or over-ridden
  - Insufficient warning level set
  - Language safety features over-ridden



- Make sure the compiler understands what you meant
  - A warning means the compiler might not do what you think
    - Your particular language use might be "undefined"
  - A warning might mean you're doing something that's likely a bug
    - It might be valid C code, but should be avoided
  - Don't over-ride features designed for safe language use

The C Language Doesn't Always Play Nice

- Defined, but potentially dangerous
  - `if (a = b) { … }  // a is modified`
  - `while (x > 0);  {x = x-1;}  // infinite loop`

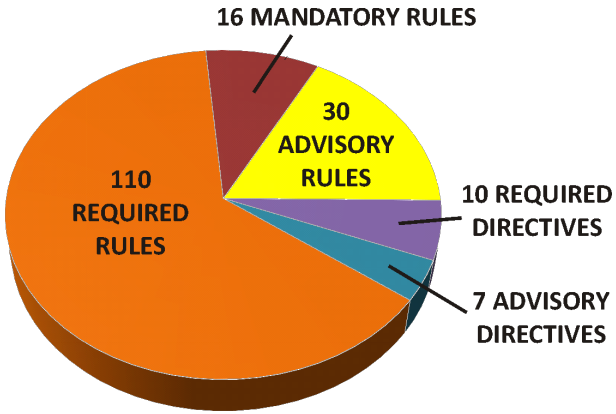
- Undefined or unspecified ➔ dangerous
  - You might think you know what these do …

…but it varies from system to system

  - `int *p = NULL;  x = *p;  // null pointer dereference`
  - `int b;  c = b;  // uninitialized variable`
  - `int x[10]; …  b = x[10];  // access past end of array`
  - `x = (i++) + a[i];  // when is i incremented?`

Language Use Guidelines & Tools

- MISRA C, C++
  - Guidelines for critical systems in C (e.g., no malloc)
  - Portability, avoiding high risk features, best practices
- CERT Secure C, C++, Java
  - Rules to reduce security risks (e.g., buffer overflows)
  - Includes list of which tools check which rules



MISRA C:2012 with Security

- Static analysis tools
  - More than compiler warnings (e.g., strong type warnings)
  - Many tools, both commercial and free. Start by going far past "–Wall" on gcc
- Dynamic Analysis tools
  - Executes the program with checks (e.g., memory array bounds)
  - Again, many tools. Start by looking at Valgrind tool suite

MISRA C 2012 Example

Rule 13.4 The result of an assignment operator should not be *used*

C90 [Unspecified 7, 8; Undefined 18], C99 [Unspecified 15, 18; Undefined 32] [Koenig 6]

Category Advisory

Analysis Decidable, Single Translation Unit

Amplification

This rule applies even if the expression containing the assignment operator is not evaluated.

Rationale

The use of assignment operators, simple or compound, in combination with other arithmetic operators is not recommended because:

- It can significantly impair the readability of the code;
- It introduces additional *side effects* into a statement making it more difficult to avoid the undefined behaviour covered by Rule 13.2.

Example

```
x = y;                          /* Compliant                                */
a[ x ] = a[ x = y ];            /* Non-compliant - the value of x = y
                                 * is used                                 */

/*
 * Non-compliant - value of bool_var = false is used but
 * bool_var == false was probably intended
 */
if ( bool_var = false )
{
}
```

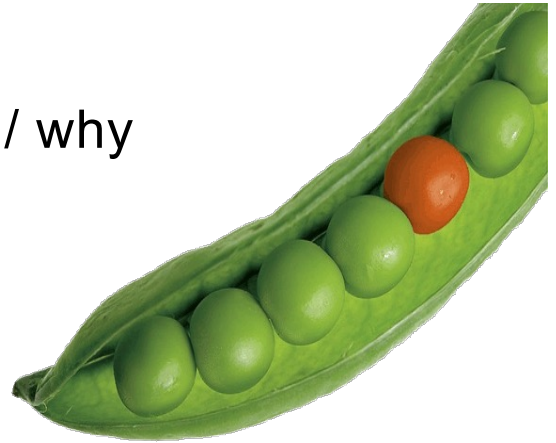
[MISRA C-2012 Guidelines; FairUse]

Let the Language Help!

- Use enum instead of int
  - `enum color {black, white, red}; // avoids bad values`
- Use const instead of #define
  - `const uint64_t x = 1; // helps with type checking`
    `uint64_t y = x << 40; // avoids 32-bit overflow bug`
- Use inline instead of #define
  - If it's too big to inline, the call overhead doesn't matter
  - Many compilers inline automatically even without keyword
- Use typedef with static analysis
  - `typedef uint32_t feet; typedef uint32_t meters;`
    `feet   x = 15;`
    `meters y = x;  // feet to meters assignment error`
- Use stdint.h for portable types
  - int32_t is 32-bit integer, uint16_t is 16-bit unsigned, etc.

Deviations & Legacy Code

- Use deviations from rules with care
  - Use “pragma” deviations sparingly; comment what / why



- What about legacy code that generates lots of warnings?
  - Strategy 1: fix one module at a time
    - Useful if you are refactoring/re-engineering the code
    - Sometimes might need to keep warnings off for 3rd party headers
  - Strategy 2: turn on one warning at a time
    - Useful if you have to keep a large codebase more or less in synch
  - Strategy 3: start over from scratch
    - If the code is bad enough this is more efficient … if business conditions permit

Or – You Can Use A Better Language!

- Desirable language capabilities:
  - Type safety and strong typing (e.g., pointers aren't ints)
  - Memory safety (e.g., bounds on arrays)
  - Robust static analysis (language & tool support)
  - In general, no surprises

```
procedure Increment (X : in out Counter_Type)
  with Global  => null,
       Depends => (X => X),
       Pre     => X < Counter_Type'Last,
       Post    => X = X'Old + 1;
```

- Spark Ada as a safety critical language
  - Formally defined language; verifiable programs
    - The language doesn't have ambiguities or undefined behaviors
  - You can prove that a program is correct
    - E.g., can prove absence of: array index out of range, division by zero
    - (In practice, this makes you clean up your code until proof succeeds)
  - Key idea: design by contract
    - Preconditions, post-conditions, side effects are defined

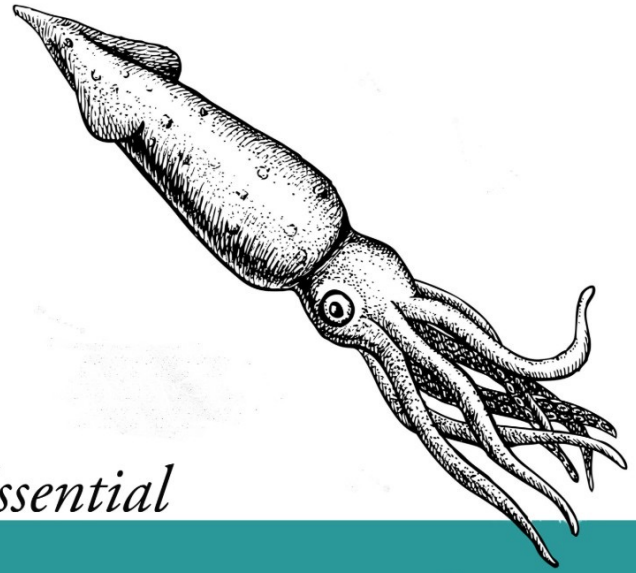
Wikipedia
https://goo.gl/3w6RF6

Spark Ada is a subset of the Ada programming language.

Language Style Best Practices

- Adopt a safe coding style (or a safe language)
  - MISRA C & CERT C are good starting points
  - Specify a static analysis tool and config settings
    - To degree practical, let machines find the style problems
  - When static analysis is set up, add dynamic analysis
- The point of good style is to avoid bugs
  - Let the compiler find many bugs automatically
  - Reduce chance of compiler mistaking your intention
- Coding style pitfalls:
  - “The code passes tests, so warnings don’t matter”
  - Real bugs lost in a huge mass of warnings
  - Making it too easy to deviate from style rules

It's only a clever hack if you're the one who wrote it



Essential

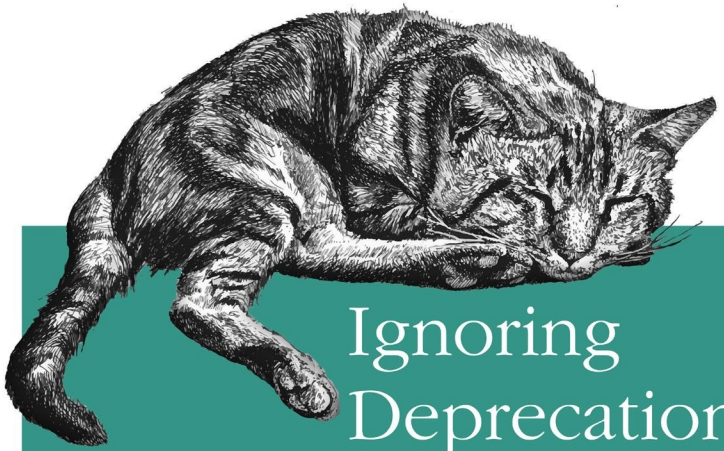
Hating Other People's Code

O RLY?

@ThePracticalDev

https://goo.gl/pvDMHX CC BY-NC2.0

Maybe they'll just go away on their own.



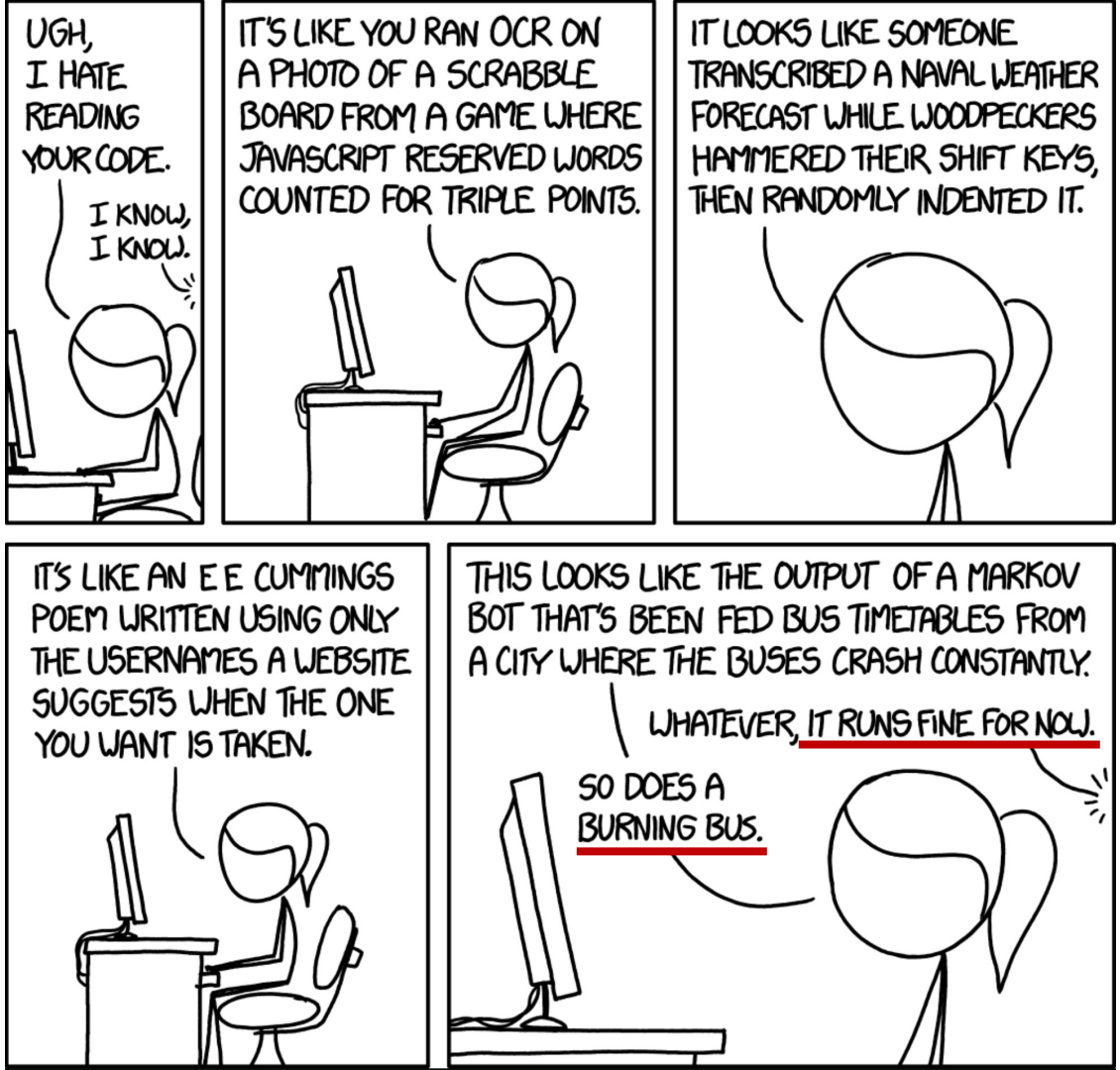
Ignoring Deprecation Warnings

A Practical Guide

O RLY?

@ThePracticalDev

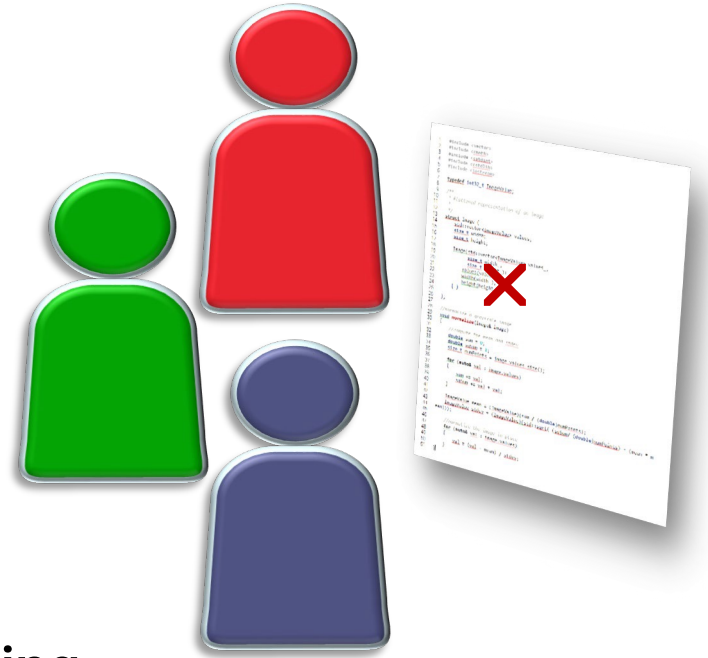
https://goo.gl/pvDMHX CC BY-NC2.0



https://xkcd.com/1695/

Peer Reviews

- Anti-Patterns:
  - No peer reviews
  - Reviews too informal/too fast
  - Reviews find <50% of all bugs



- Fresh eyes find defects
  - Code and other document benefit from a second (and third) set of eyes
  - Peer reviews find more bugs/$ than testing
    - And, they find them earlier when bugs are cheaper to fix
  - Everything written down can benefit from a review

Most Effective Quality Practices

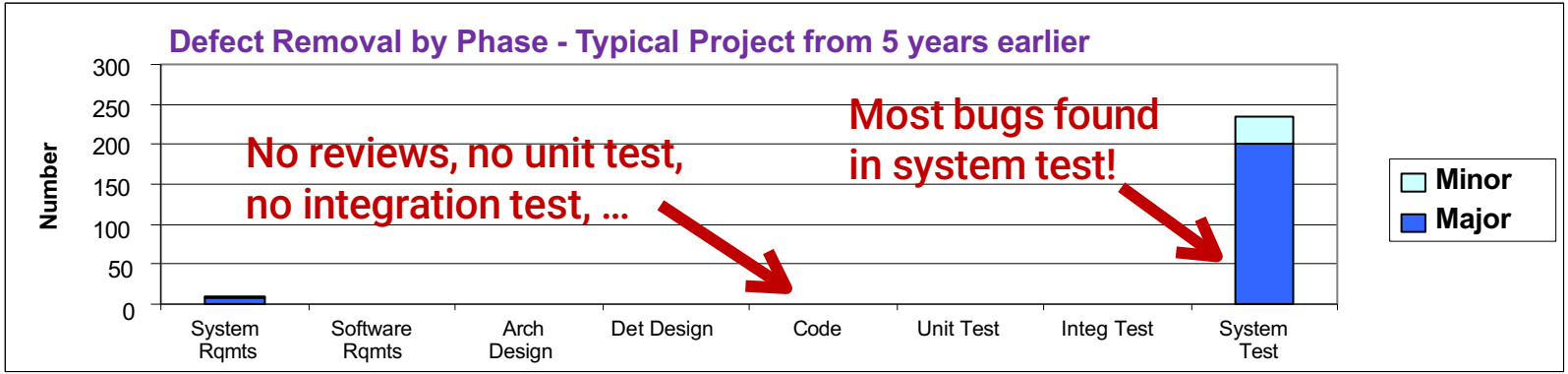
Ebert & Jones, "Embedded Software: Facts, Figures, and Future," IEEE Computer, April 2009, pp. 42-52

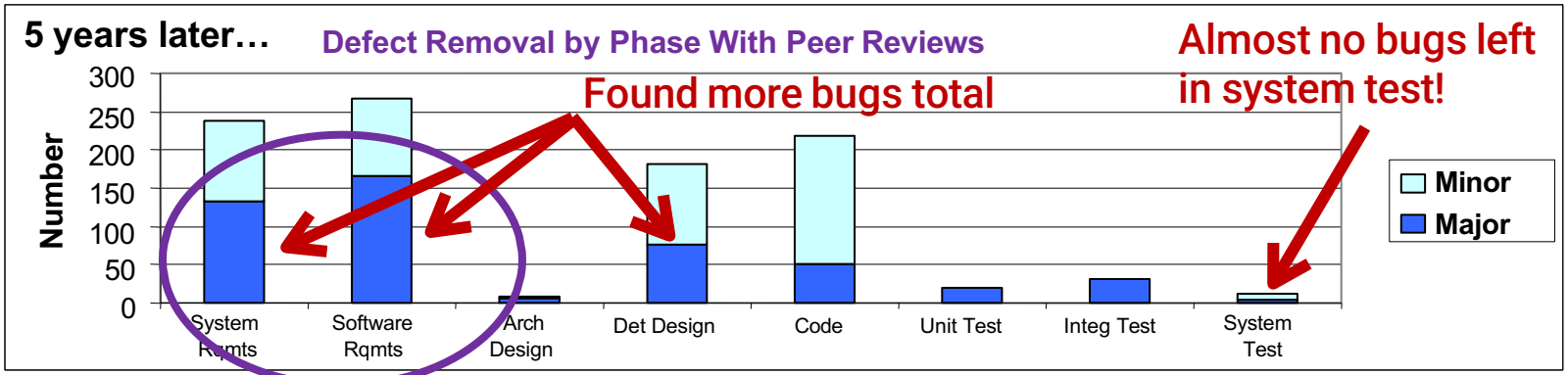
Ranked by defect removal effectiveness in percent defects detectable at that stage that are removed.

"*" means exceptionally productive technique (more than 750+ function points/month)

- * 87% static code analysis ("lint" tools, compiler warnings)
- 85% design inspection
- 85% code inspection
- 82% Quality Function Deployment (requirements analysis)
- 80% test plan inspection
- 78% test script inspection
- * 77% document review (other documents)
- 75% pair programming (informal on-the-fly review)
- 70% bug repair inspection
- * 65% usability testing
- 50% subroutine testing (unit test)
- * 45% SQA (Software Quality Assurance) review
- * 40% acceptance testing

Peer Reviews Are Effective + Efficient





Found many bugs up front, where fixes are cheaper

[Source: Roger G., Aug. 2005]

Gold Standard: Fagan Style Inspections

- Methodical, in-person review meetings
  - Pre-meeting familiarity with project
  - Producer explains item then leaves
  - Moderator keeps things moving
  - Reader (not author) summarizes as you go
  - Reviewers go over every line, using checklists (perspective-based)
  - Recorder takes written notes
  - Result: written list of defects. The Producer fixes code off-line
  - Re-inspection if the defect rate was too high
- Methodical reviews are the most cost effective
  - Important to measure bug discovery rate to ensure review quality

Rules for Successful Peer Reviews

- Inspect the item, not the author
  - Don't attack the author.
- Don't get defensive
  - Nobody writes perfect code. Get over it.
- Find but don't fix problems
  - Don't try to fix them; just identify them.
- Limit meetings to two hours
  - People are less productive after that point.
- Keep a reasonable pace
  - About 150 lines of code (or equivalent) per hour. Too fast and too slow are both bad.
- Avoid "religious" debates on style
  - Enforce conformance to your style guide. No debates on whether style guide is correct.
- Inspect, early, often, and as formally as you can
  - Keep records to document value (might take a while to mature).

Example Light-Weight Review Report

	Peer Review Template for Project X	
Date:	4/17/2011	
Artifact:	Xyzzy.cpp Functions: Foo(), Bar(), Baz()	
Reviewers:	Stella K., Joe B., Sam Q., Trish R.	
Size:	357	SLOC
Time Spent:	112	Minutes
# Issues:	3	
Outcome:	Re-Review of Bug Fixes Required	

 # issues found is the most important item!

Issue#	Issue Description	Status
1	Issue 1…..	Fixed
2	Issue 2….	Bugzilla
3	Issue 3….	Bugzilla
4	Issue 4….	Not a Bug
5		
6		
7		
8		

Free form text issue description

Status Key:	Fixed (trivial fix by author; no need to enter in defect list)
	Bugzilla (entered into project defect system)
	Not a Bug (false alarm)

Just enter "fixed" if fixed within 24 hours

Perspective-Based Peer Reviews

- Perspective-based Peer Reviews are 35% more effective

[https://www.cs.umd.edu/projects/SoftEng/ESEG/papers/82.78.pdf]

- Mechanics of a Perspective-based review
  - Divide a peer review checklist into three sections
  - Assign each participant a different section of the checklist
    - OK to notice other things, but primary responsibility is that section
    - Multiple sets of eyes + perspective breadth

- Example perspectives for a review:
  - Control flow issues
  - Data handling issues
  - Style issues

Peer Review Checklist Template

Peer Review Checklist: Embedded C Code

Before Review:

0	_____	Code compiles clean with extensive warning checks (e.g. MISRA C rules)

Reviewer #1:

1	_____	
2	_____	
3	_____	
4	_____	
5	_____	
6	_____	
7	_____	

Reviewer #2:

8	_____	Single point of exit
9	_____	Loop entry and exi
10	_____	Conditionals shoul
11	_____	All functions can b
12	_____	Use const and inlir
13	_____	Avoid use of magic
14	_____	Strong typing (incl
15	_____	All variables have

Reviewer #3:

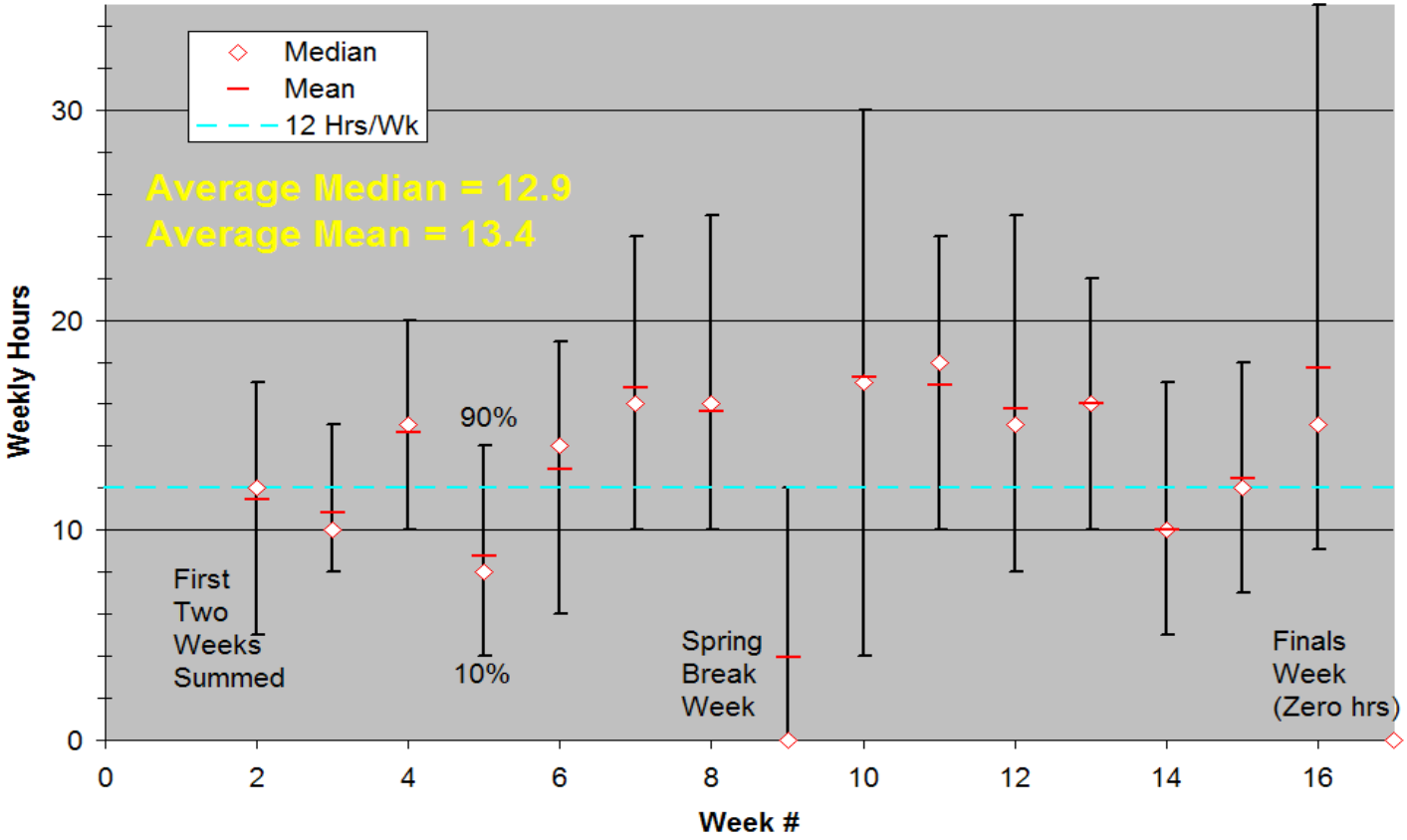
16	_____	Minimum scope for all functions and variables; essentially no globals
17	_____	Concurrency (locking, volatile keyword, minimize blocking time)
18	_____	Input parameter checking (style, completeness)
19	_____	Error handling for function returns
20	_____	Handle null pointers, division by zero, null strings, boundary conditions
21	_____	Floating point issues (equality, NaN, INF, roundoff); use of fixed point
22	_____	Buffer overflow safety (bound checking, avoid unsafe string operations)

All Reviewers

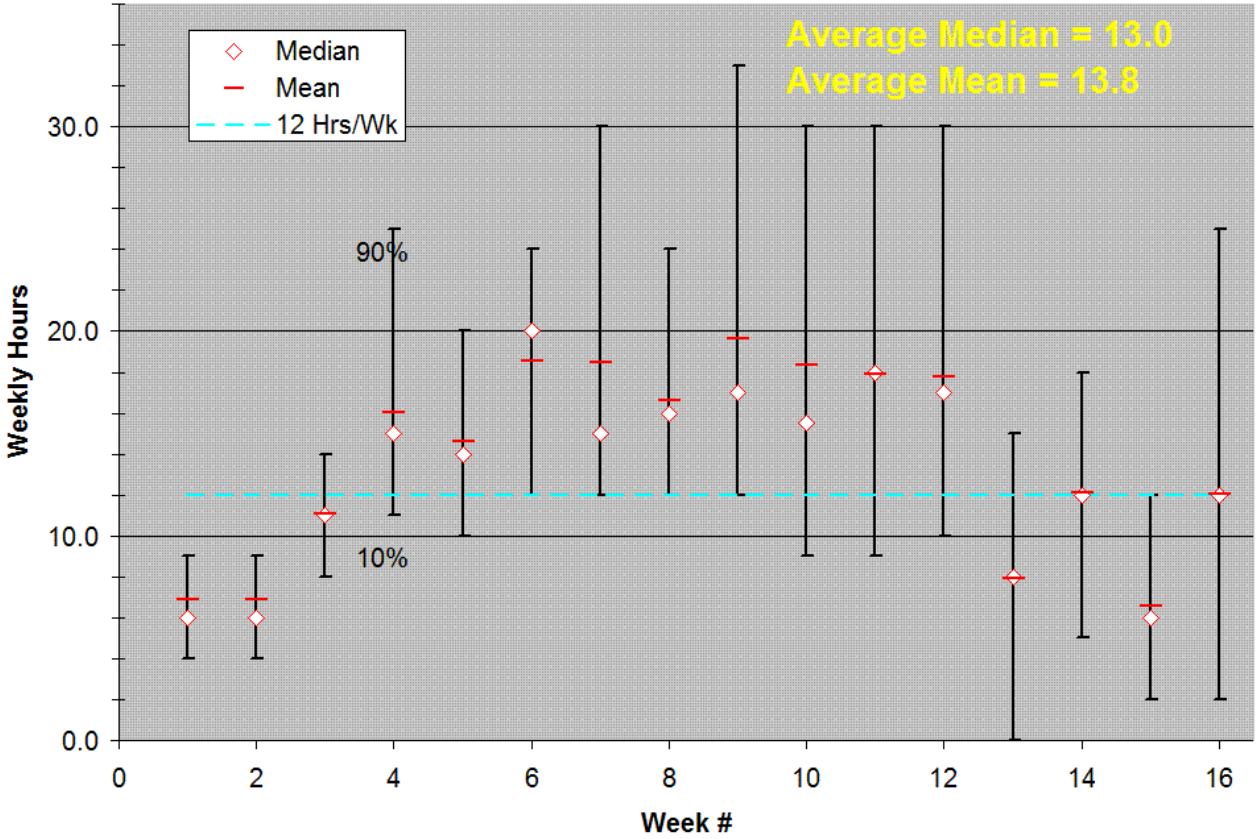
23	_____	Does the code match the detailed design (correct functionality)?
24	_____	Is the code as simple, obvious, and easy to review as possible?
		For TWO Reviewers assign items: Reviewer#1: 1-11; 23-24 Reviewer#2: 12-24

- **Customize as needed**

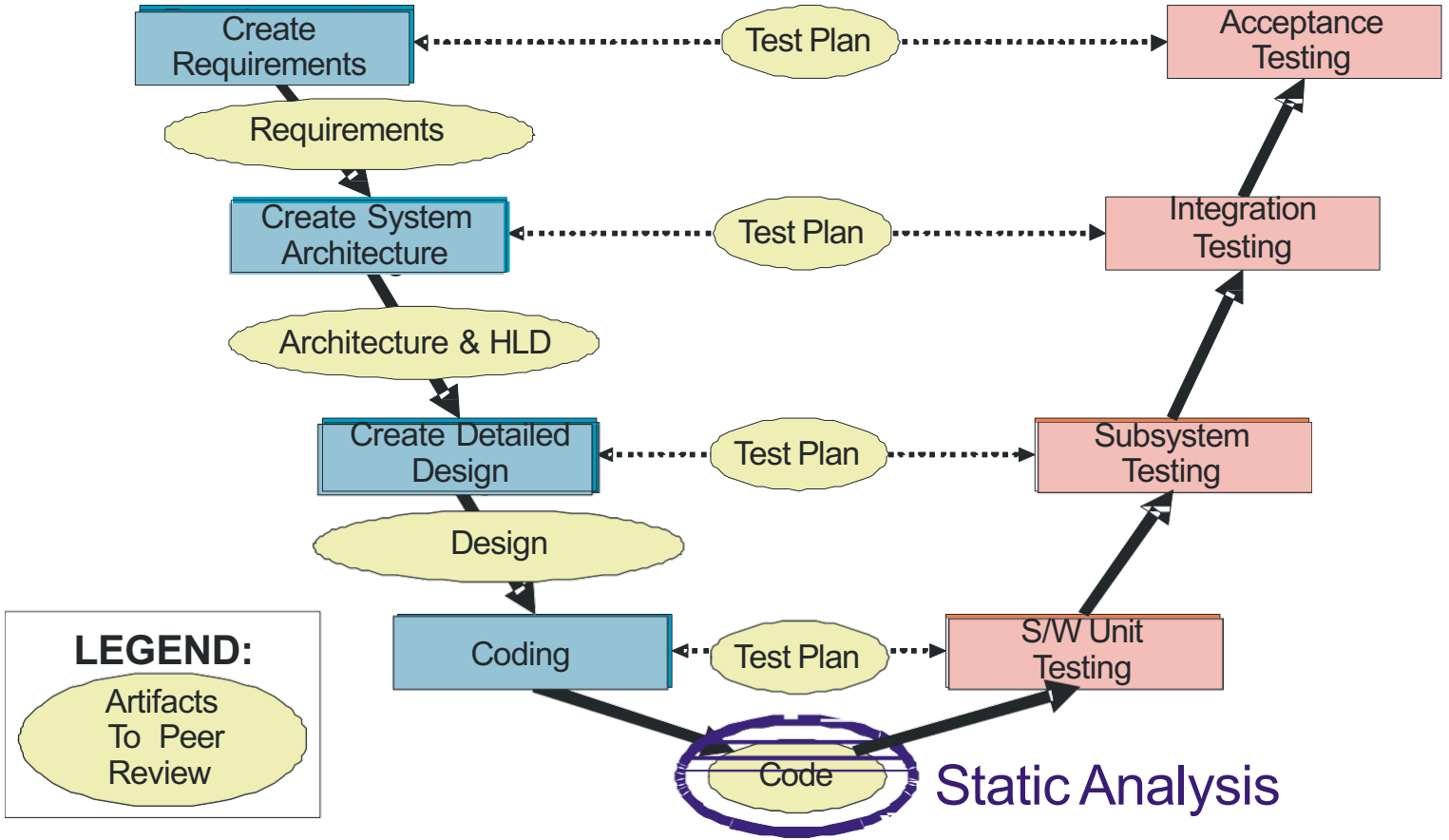
Before (Ineffective Reviews)



With Weekly Defect Reporting



Review More Than Just The Code



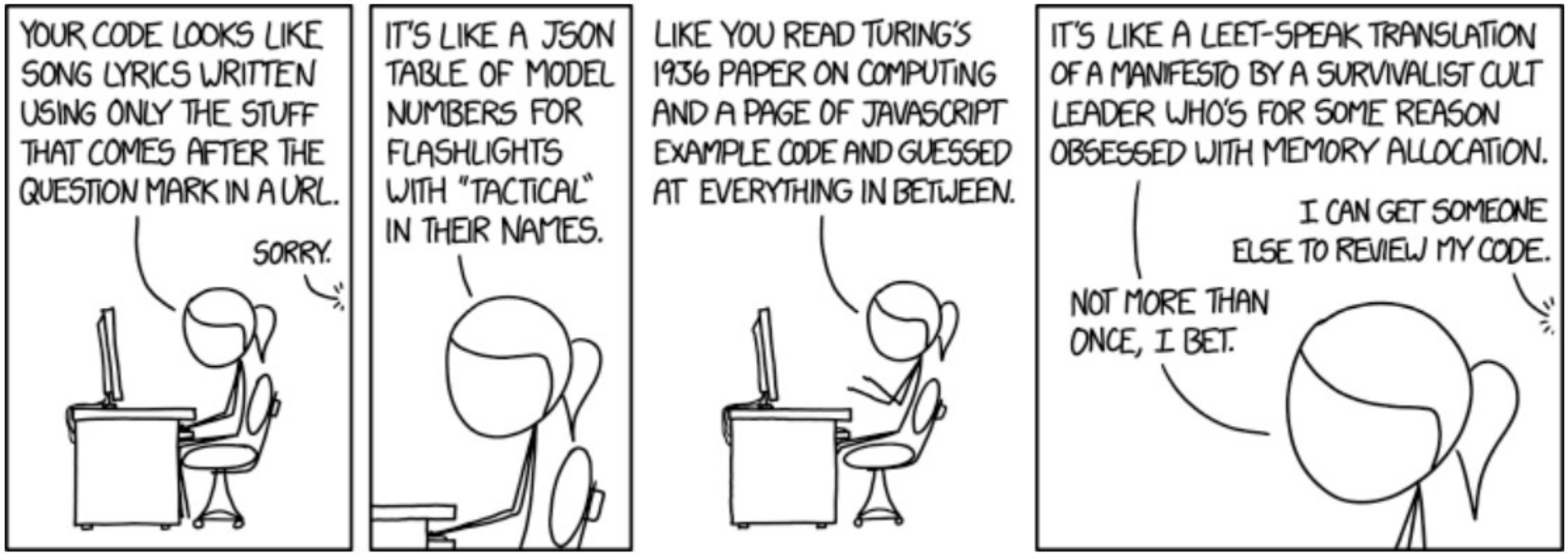
Economics Of Peer Review

- Peer reviews provide more eyeballs to find bugs in an affordable way
  - Good embedded coding rate is 1-2 lines of code/person-hr
    - (Across entire project, including reqts, test, etc.)
  - A person can review 50-100 times faster than they can write code
    - If you have 4 people reviewing, that is still >10x faster than writing!
  - How much does peer review cost?
    - 4 people * 100-200 lines of code reviewed per hour
    - E.g., 300 lines; 4 people; 2 hrs review+1 hr prep = 25 LOC/person-hr
  - Reviews are only about 5%-10% of your project cost
- Good peer reviews find at least half the bugs!
  - And they find them early, so total project cost can be reduced
- Why is it folks say they don't have time to do peer reviews?

Peer Review Best Practices

- Formal reviews (inspections) optimize bugs/$
  - Target 10% of project effort to find 50% of bugs
    - You can review 100x faster than write code; it's cheap
  - Review everything written down, not just code
  - Use a perspective-based checklist to find more bugs

- Review pitfalls
  - If your reviews find <50% of defects, they are BROKEN
    - The 80/20 rule does NOT apply to review formality! Formal reviews are best.
    - You can't review at end; need to review throughout project
- Review tools
  - On-line review tools are OK, but not a substitute for in-person meeting
  - Static analysis tools are great – but not a review!



https://www.xkcd.com/1833/

Disclaimer

This lecture contains materials from:

- Philip Koopman - CMU

Global Variables Are Evil!

"Global variables are responsible for much undebuggable code, reentrancy problems, global warming, and male pattern baldness. Avoid them!"

— *Jack Ganssle*

Global Variables Are Evil!

- Anti-Patterns:
  - More than a few read/write globals
  - Globals shared between tasks/threads
  - Variables have larger scope than needed

- Global variables are visible everywhere:
  - Use of globals indicates poor modularity
    - Globals are prone to tricky bugs and race conditions
  - Local static variables are best if you need persistence
    - File static variables can be OK if used properly
    - Don't make procedures globally visible if not needed

Global vs. Static Variables

- Globals:

```
uint32_t gVar = 0;
void gProc(…)   { … }
```

- Global risks
  - Written from anywhere
    - Debugging: who wrote it?
  - Read from anywhere
    - Changes break everything
  - Multithreaded race conditions
  - Increased complexity
    - Data flow “spaghetti”

- **File Static:**

```
static uint32_t fsVar = 0;
static void fsProc(…)   { … }
```

  - Only inside .c file
  - Use with small .c files
  - Like C++ “private”

- **Local Static:**

```
void gProc(…)
{ static uint32_t sVar = 0;
… }
```

  - Persistent variable value
  - Can’t be seen outside procedure

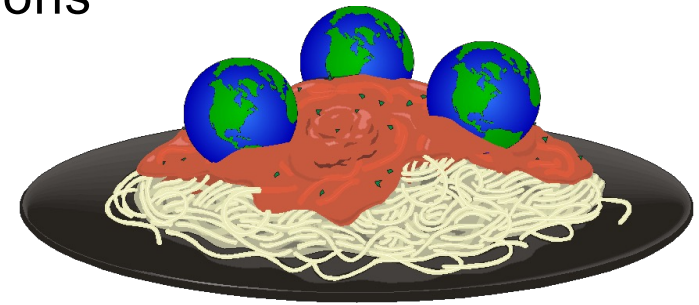
Avoiding And Removing Globals

- Define smallest scope possible (variables and procedures)
  - Change global to file static; file static to local static
- Arrange .c files based on access to data
  - Example: time of day updated by ISR
    - File static time of day variable in TimeOfDay.c
    - Put timer tick ISR in TimeOfDay.c
    - Put procedure to disable interrupts & read time of day in TimeOfDay.c
- Configuration values & constants
  - Use const keyword – prevents multiple writers
  - Read-only access to global configuration data structure
  - Limit visibility to need-to-know within relevant .h file

Best Practices For Avoiding Globals

- Use smallest practical scope for variables & procedures
  - Ideally, zero global variables
  - Use file static if you must; local static if you can
  - A good compiler will generate efficient code

- Reorganize code to reduce scope
  - Write anything except locking variables only in one place
  - File static variables for small groups of functions
    - More or less the idea of C++ private keyword
    - Take care of data locking when reading



- Global Variable Pitfalls
  - Lots of global variables is a sign of bad code

Example

https://betterembsw.blogspot.com/2013/09/getting-rid-of-global-variables.html

You have a "globals.c" file that defines a mess of globals, including:

```
int g_ErrCount;
```

which might be used to tally the number of run-time errors seen by the system. I've used a "g_" naming convention to emphasize that is a global, which means that every .c file in the program can read and write this variable with wild abandon.

Let's say you also have the following places this variable is referenced, including globals.c just mentioned:

- **globals.c:** `int g_ErrCount;  // define the variable`
- **globals.h:** `extern int g_ErrCount; // other files include this`
- **init.c:** `g_ErrCount = 0; // init when program starts`
- **moduleX.c:** `g_ErrCount++;  // tally another error`
- **moduleY.c:** `XVar = g_ErrCount; // get current number of errors`
- **moduleZ.c:** `g_ErrCount = 0;  // clear number of reported errors`

Create an Error Counting Module

Create separate "object" for error counting: ErrCount.c

- **globals.c:** // not needed any more for this variable
- **ErrCount.c:** int g_ErrCount; // define the variable
- **ErrCount.h:** extern int g_ErrCount; // other files include this
- **init.c:** g_ErrCount = 0; // init when program starts
- **moduleX.c:** g_ErrCount++; // tally another error
- **moduleY.c:** XVar = g_ErrCount; // get current number of errors
- **moduleZ.c:** g_ErrCount = 0; // clear number of reported errors

Initialize Where Defined

- **ErrCount.c:** int g_ErrCount = 0; // define and init variable
- **ErrCount.h:** extern int g_ErrCount; // other files include this
- **init.c:** // no longer needed
- **moduleX.c:** g_ErrCount++; // tally another error
- **moduleY.c:** XVar = g_ErrCount; // get current number of errors
- **moduleZ.c:** g_ErrCount = 0; // clear number of reported errors

Convert to File Static

- **ErrCount.c:** static int ErrCount = 0; // only visible in this file
- **ErrCount.h:** // static variables are invisible outside .c file
- **moduleX.c:** g_ErrCount++; // tally another error
- **moduleY.c:** XVar = g_ErrCount; // get current number of errors
- **moduleZ.c:** g_ErrCount = 0; // clear number of reported errors

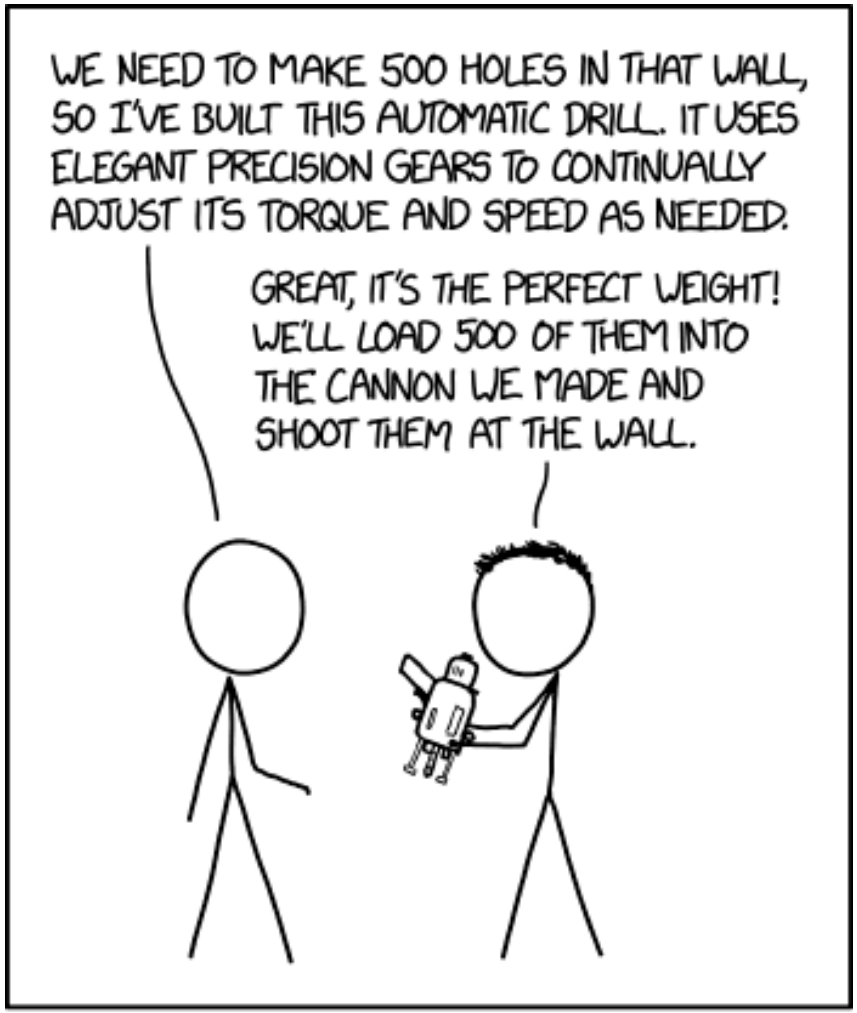
Add Accessor Function

- **ErrCount.c:** static int ErrCount = 0; // only visible in this file
  - inline void ErrCount_Incr() { ErrCount++; }
  - inline int ErrCount_Get() { return(ErrCount); }
  - inline void ErrCount_Reset() { ErrCount = 0; }
- **ErrCount.h:**
  - inline int ErrCount_Get(); // get current count value
  - inline void ErrCount_Reset(); // reset count
  - inline void ErrCount_Incr(); // increment the count

  // Note that there is NO access to ErrCount directly
- **moduleX.c:** ErrCount_Incr(); // tally another error
- **moduleY.c:** XVar = ErrCount_Get(); // get current number of errors
- **moduleZ.c:** ErrCount_Reset(); // clear number of reported errors

Advantages of this Approach

- Software authors can only perform intended functions specific to an error counter: increment, read, and reset. Setting to an arbitrary value isn't allowed. If you don't want the value changed other than via incrementing, you can just delete the reset function. This prevents some types of bugs from ever happening.
- If you need to change the data type or representation of the counter used that all happens inside ErrCount.c with no effect on the rest of the code. For example, if you find a bug with error counts overflowing, it is a lot easier to fix that in one place than every place that increments the counter!
- If you are debugging with a breakpoint debugger it is easier to know when the variable has been modified, because you can get rid of the "inline" keywords and put a breakpoint in the access functions. Otherwise, you need watchpoints, which aren't always available.
- If different tasks in a multitasking system need to access the variable, then it is a lot easier to get the concurrency management right inside a few access functions than to remember to get it right everywhere the variable is read or written (get it right once, use those functions over and over). Don't forget to make the variable volatile and disable interrupts when accessing it if concurrency is an issue.



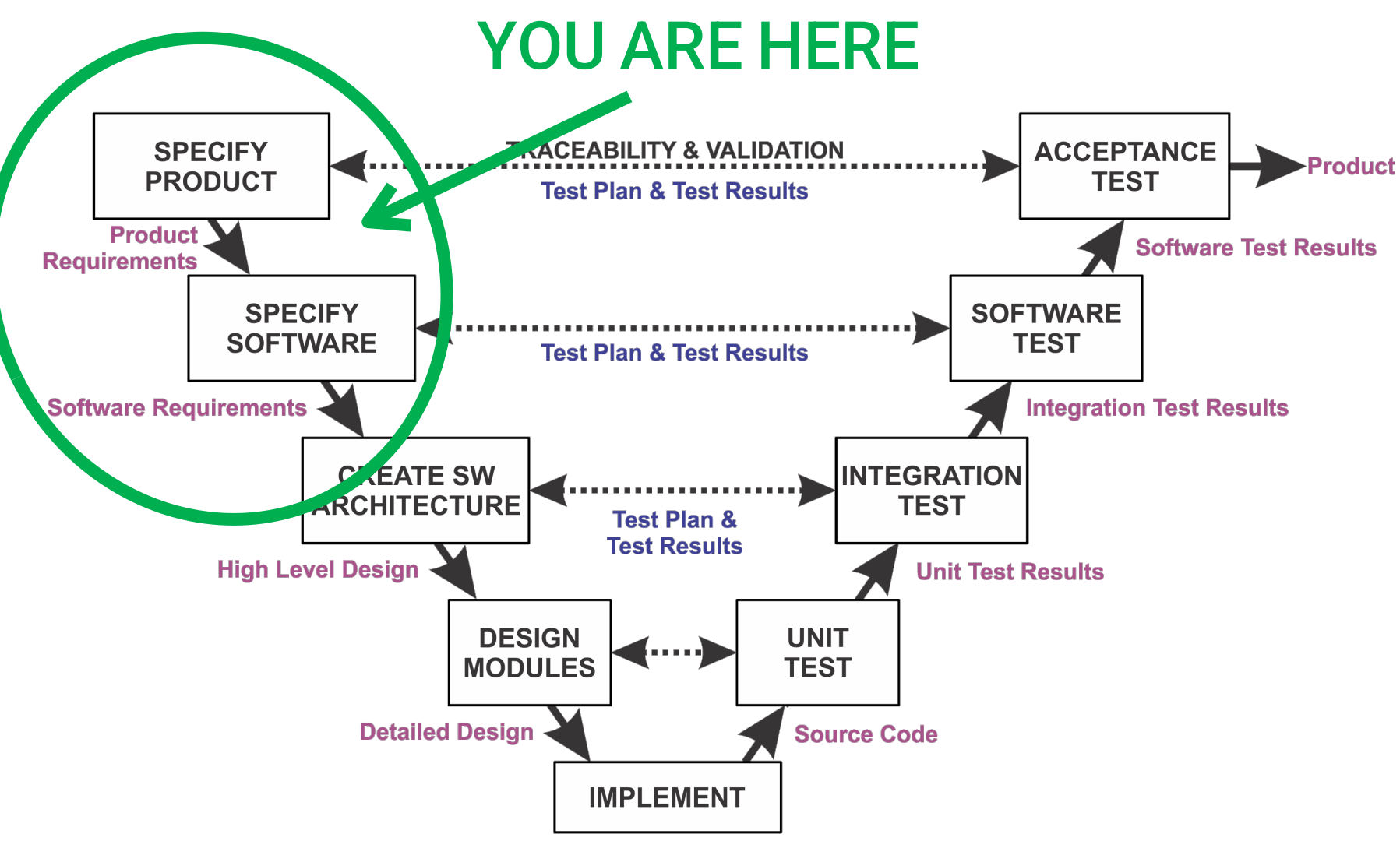
HOW SOFTWARE DEVELOPMENT WORKS

https://xkcd.com/2309/

Embedded Software Requirements

"In spite of appearances, people seldom know what they want until you give them what they ask for. "

– Donald Gause and Gerald Weinberg,
Are Your Lights On?


YOU ARE HERE
SPECIFY PRODUCT
TRACEABILITY & VALIDATION
Test Plan & Test Results
ACCEPTANCE TEST
Product
Product Requirements
Software Test Results
SPECIFY SOFTWARE
Test Plan & Test Results
SOFTWARE TEST
Software Requirements
Integration Test Results
CREATE SW ARCHITECTURE
Test Plan & Test Results
INTEGRATION TEST
High Level Design
Unit Test Results
DESIGN MODULES
UNIT TEST
Detailed Design
Source Code
IMPLEMENT

Requirements Overview

- Anti-Patterns:
  - Requirements aren't written down
  - Requirements incomplete, imprecise
  - "Be like last version, except…"

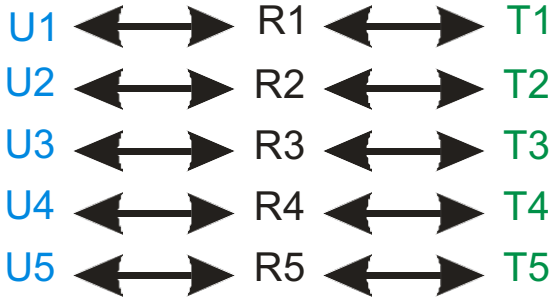
- Requirements
  - Requirements faults can defeat a design before it is even built
  - Describe what system does
    - Also what it's not supposed to do
  - Precise, testable language
    - Each requirement traces to system test

- 2005:
  $170M
  FBI Virtual Case File project terminated

- Requirements issues:
  - Requirements not defined when development contract signed
  - "We will know it when we see it"
  - Repeated requirements changes
  - Scope creep (new requirements added) of 80%

Characteristics of Good Requirements

- Precise and minimally constrained
  - Describes what system should do, not how it does it
  - Uses "shall" to require an action; "should" to state a goal
  - If possible has a numeric target instead of qualitative term
    - Has tolerance (e.g., 500 msec +/- 10%, "less than X")
- Traceable & testable
  - Each requirement has a unique label (e.g., "R-7.3")
  - Each requirement cleanly traces to an acceptance test
  - Requirement satisfaction has a feasible yes/no test
- Supported within context of system
  - Supported by rationale or commentary
  - Uses consistent terminology
  - Any conflicting requirements resolved or prioritized

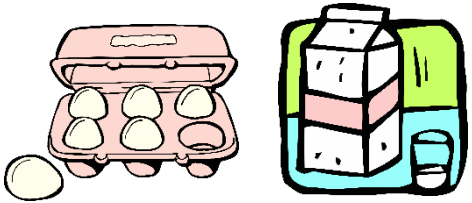


Problematic Requirements

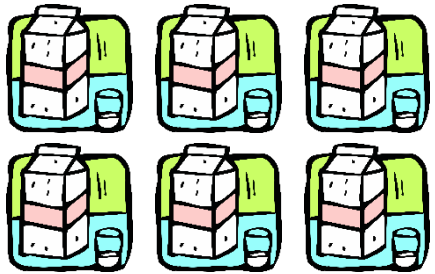
- Untraceable (no label)
  - System shall shut down when E-STOP is activated.
- Untestable
  - R-1.1: System shall never crash
- Imprecise
  - R-1.7: The system provides quick feedback to the user.
- No measurement tolerance
  - R-2.3: LED shall flash with a period of 500 msec
- Overly complex
  - R-7.3: Pressing the red button shall activate Widget X, while pressing the blue button should cause LED Z to blink instead of LED Y illuminating steadily, which would be accomplished via the yellow button.
- Describes implementation
  - R-8.3: Pressing button W shall cause two 16-bit integer values to be added, then …

Requirements Ambiguity

- A requirements engineer gets a text message:
  "On the way home, please pick up one carton of milk.
  And if they have eggs, get six."



- The requirements engineer comes home with:
  6 cartons of milk and no eggs.

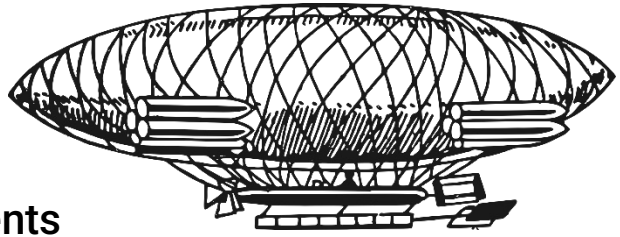


- Spouse: "Why did you buy six cartons of milk?!"
- Requirements Engineer: "They had eggs."

Adapted from: www.ganssle.com/jokes.htm.

Extra-Functional Requirements

- Emergent properties (things hard to attribute to one component)
  - Performance, real-time deadlines
  - Security, Safety, Dependability in general
  - Size, Weight and Power consumption ("SWaP")
    - Often handled with an allocation budget across components
  - Forbidden behaviors ("shall not do X")
    - Often in context of safety requirements
    - "Safety function" is a way to ensure a negative behavior, but some behaviors are emergent



- Design constraints
  - Must meet a particular set of standards
  - Must use a particular technology
  - System cost, project deadline, project staffing

https://goo.gl/hT3nDU

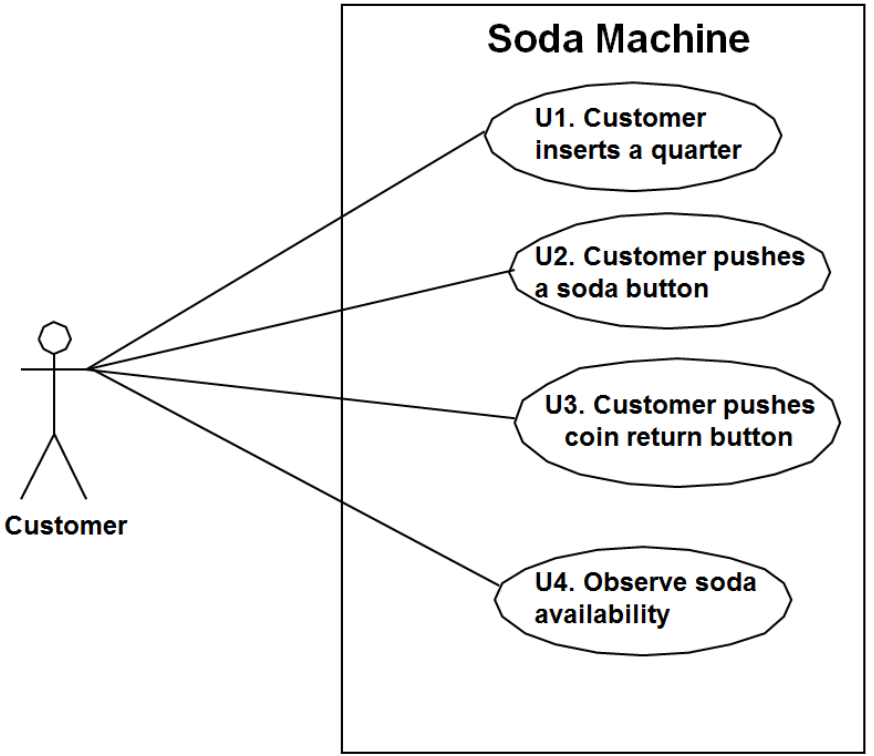
Product vs. Engineering Requirements

- Product level requirements:
  what the product does
  - Example:
    "PR6. The clock shall support a user-settable audible alarm."
  - Gives a feature list of what the product actually does
  - Can be the interface between marketing and engineering

- Detailed functional/engineering requirements:
  how the product actually works
  - Example: "R5. Time set buttons shall change the alarm set time."
  - Embedded systems often have detailed requirements tied to operational modes
    - "R5. In Alarm Set Mode the time set buttons shall change the alarm set time."
    - "R6. Pressing the "+" time set button shall increase time value by one minute per button press according to the current set mode."

Requirements Approaches

- Text document with list of requirements
  - Works best if domain experts already know reqts.
  - Over time, this can converge to OK reqts.
- UML Use Cases
  - Different activities performed by actors
  - Requirements are scenarios attached to each use case
- Agile User Stories
  - Each story describes a system interaction
- Functional decomposition
  - Start with primary system functions
  - Make more and more detailed lists of sub-functions (creates a “functional architecture”)
- Prototyping to elicit requirements
  - Customers know it when they see it
  - Sometimes a paper mock-up is enough



UML Use Case Diagram

Requirements Templates (EARS)

- Easy Approach to Requirements Syntax (Mavin et al.) e.g.: https://bit.ly/2CQSF37
- [While/Where <precondition>] [when/if <trigger> then]
  <system> shall <response>
  - Ubiquitous: The touch screen shall have a response time of less than 250 msec.
  - State-driven: WHILE an external speaker is connected, the internal speaker shall mute.
  - Event-driven: WHEN a card is inserted, the card reader shall verify credentials.
  - Optional feature: WHERE a convertible roof is installed, a park/roof motion interlock function shall be provided.
  - Unwanted: IF an invalid value is entered THEN an error message shall be displayed.
  - Complex: combinations of the above
- Requirements issues to avoid:
  - Ambiguous, vague, complex, omitted, duplicated, wordy, implementation, untestable

Example Software Requirements

National Aeronautics and Space Administration

GRC-CONN-REQ-0084
EFFECTIVE DATE: 02/23/2010

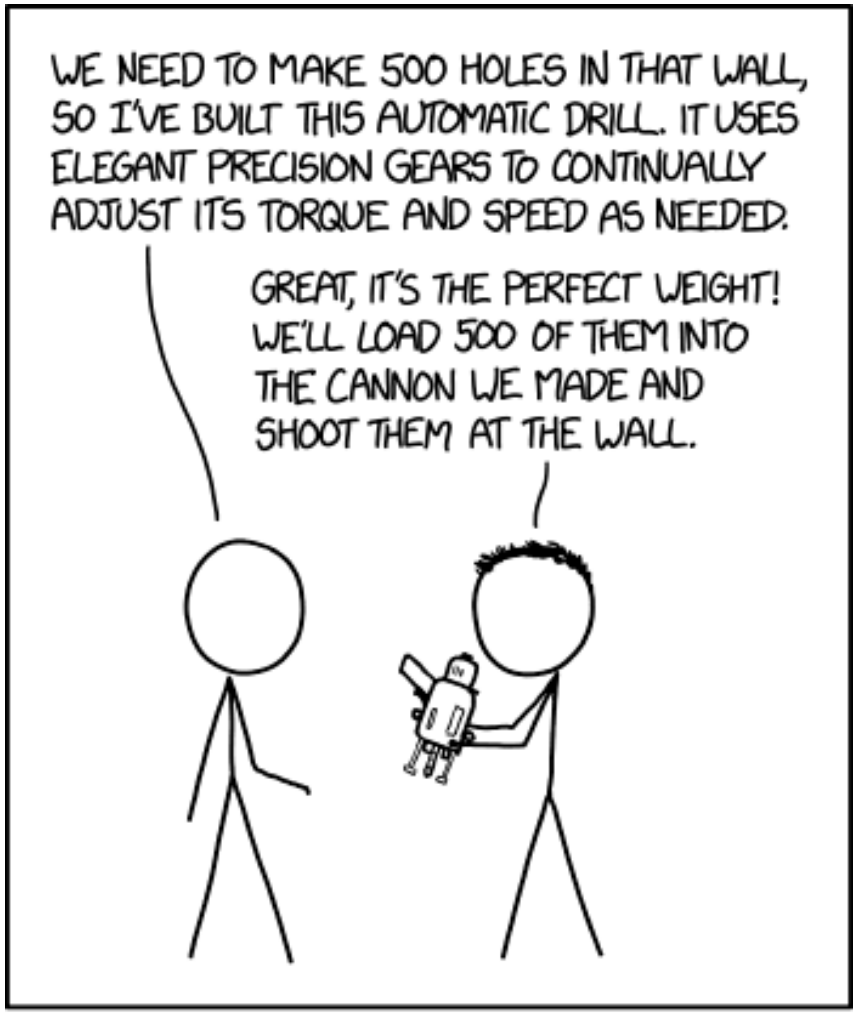
Parent Req	ReqID	Requirement Text and Rationale	Prior-ity	Allocated To
FSRD-3714	SRS-3.2.6.12	The Software **shall** send data at a user data rate from zero up to and including 100 Mbps. *Rationale: The maximum data rate the Payload Avionics Software must send is 100 Mbps. Lower rates must also be handled.*	P2	PAS
FSRD-3133	SRS-3.2.6.13	The software **shall** send and receive data on two SpaceWire channels simultaneously at up to the maximum SDR interface data rate (full duplex) that can be sustained by both SDRs. *Rationale: When communicating with multiple radios, the Software will need to sustain an achievable data rate. In this requirement, it is defined as the minimum data rate of the two (or three, if possible) SDRs involved in the experiment. For instance, this data could be provided in the routing table. If two other radios are involved, then the data rate may change, based on the capability of those two radios (i.e. a new minimum interface data rate). This value should not be hard coded, but should have the capability for change, once on-orbit.*	P2	PAS

https://goo.gl/qct5tL

Best Practices for Requirements

- Six C-terms for Good Requirements
  - Clear, Concise, Correct, Coherent, Complete and Confirmable
- Also:
  - Deal with extra-functional issues
  - Relate requirements to design flow
    - Associate with user stories or use cases
    - Trace to corresponding test
- Requirements pitfalls
  - Avoid unnecessary details and implementation
  - If it's missing from requirements, it won't get done
  - If it's not testable, you won't know if it got done

https://goo.gl/6H3dxi



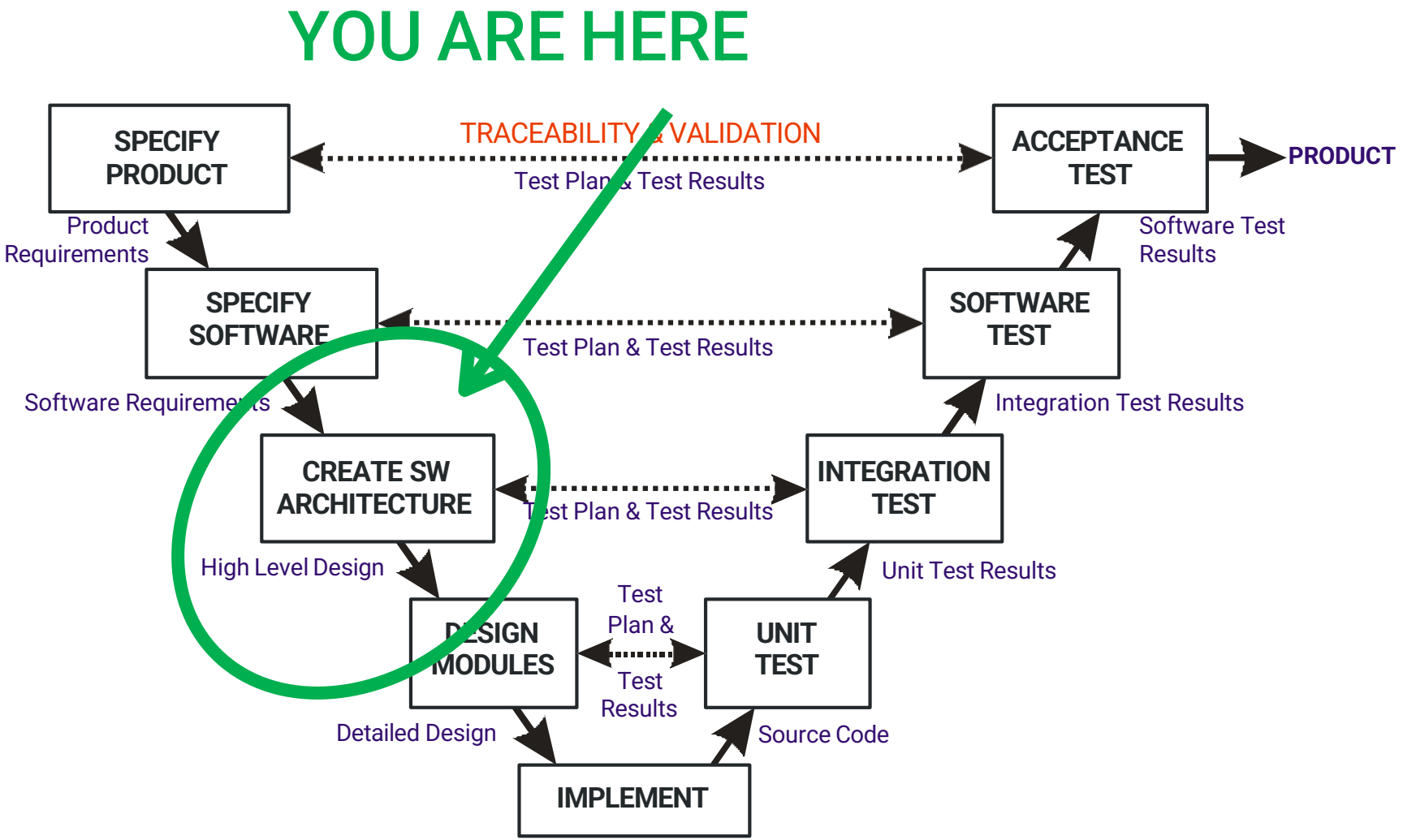
HOW SOFTWARE DEVELOPMENT WORKS

https://xkcd.com/2021/

Software Architecture & High Level Design

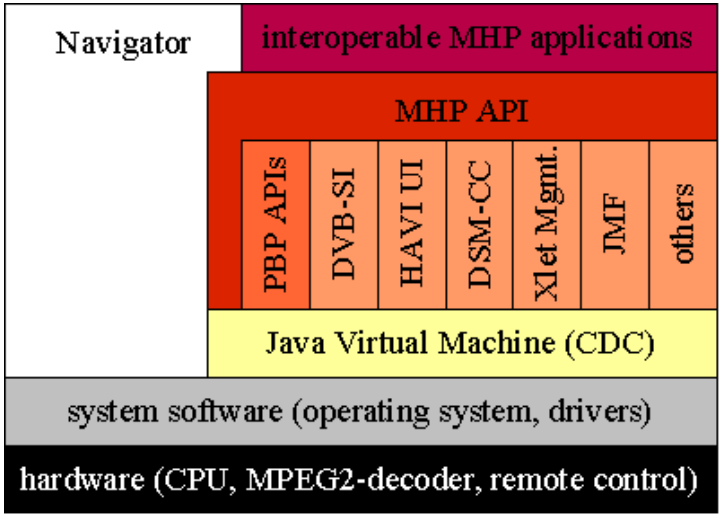
All the really important mistakes are made the first day.

– Eberhardt Rechtin,
System Architecting


YOU ARE HERE
SPECIFY PRODUCT
TRACEABILITY & VALIDATION
Test Plan & Test Results
ACCEPTANCE TEST
PRODUCT
Product Requirements
Software Test Results
SPECIFY SOFTWARE
Test Plan & Test Results
SOFTWARE TEST
Software Requirements
Integration Test Results
CREATE SW ARCHITECTURE
Test Plan & Test Results
INTEGRATION TEST
High Level Design
Unit Test Results
Test Plan &
DESIGN MODULES
Test Results
UNIT TEST
Detailed Design
Source Code
IMPLEMENT

Architecture & High Level Design (HLD)

- Anti-Patterns:
  - Skipping from requirements to code
  - No picture that shows how all the components fit together
  - “Wedding cake” layer diagram that omits interface information

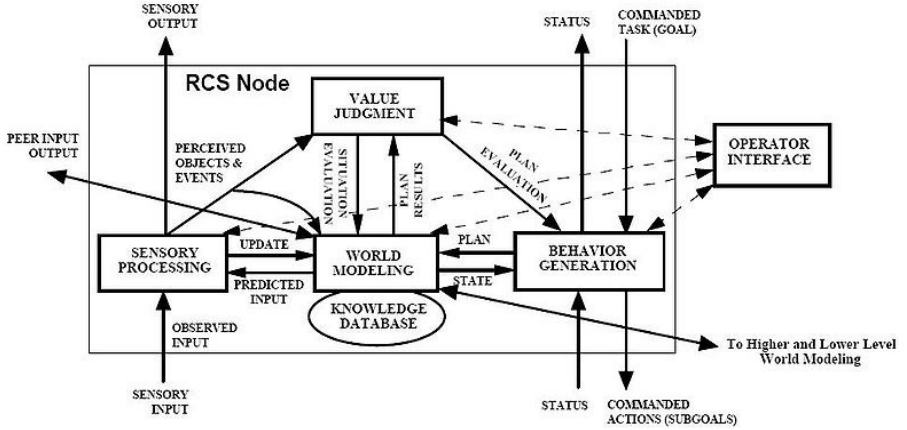


https://goo.gl/J8MAuK

- Elements of High Level Design
  - Architecture: boxes, arrows, interfaces
    - Arrows/interfaces show communication paths between components
    - Recursive: one designer’s system is another designer’s component
  - High Level Design (HLD) = architecture (nouns) + requirements (verbs)
    - Sequence Diagrams (SDs) show interactions

Architecture: Boxes and Arrows

- Software architecture
  shows the big picture
  - Boxes: software modules/objects
  - Arrows: interfaces
  - Box and arrow semantics well-defined
    - Meaning of box/arrow depends on goal
  - Components all on a single page
    - Nesting of diagrams is OK

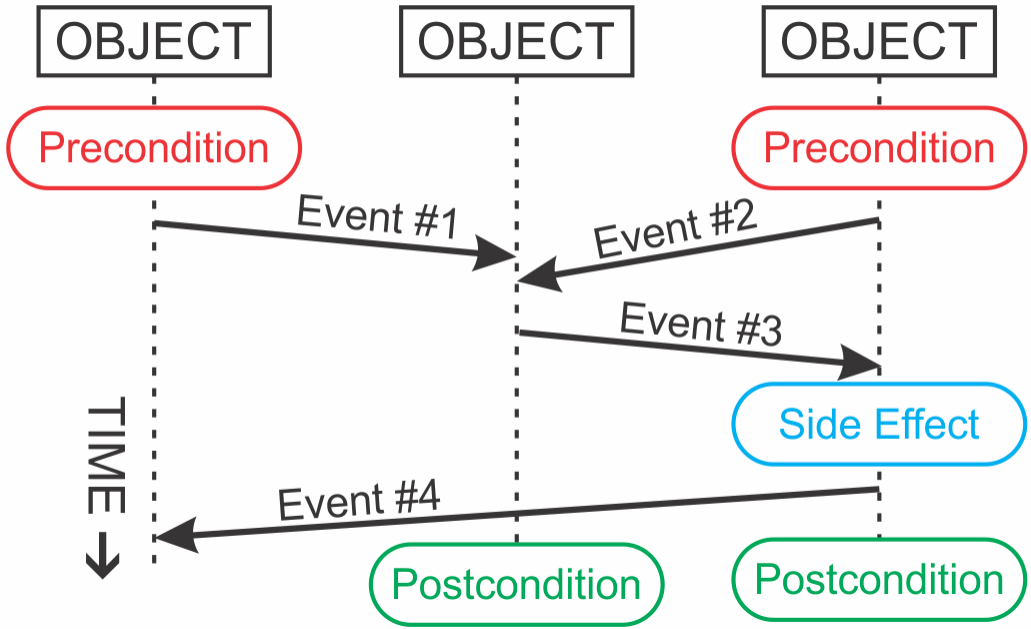


https://goo.gl/WnciF3

- Many different architecture diagrams are possible, such as:
  - Software architecture (components and data flow types)
  - Hardware architecture with software allocation
  - Controls architecture showing hierarchical control
  - Call graph showing run-time hierarchy

Sequence Diagram as HLD Notation

- SD construction:
  - Each object has a time column extending downward
  - Arcs are interactions between objects
- Each SD shows a scenario
  - Top ovals are preconditions
  - Middle ovals are side effects
  - Bottom ovals are postconditions

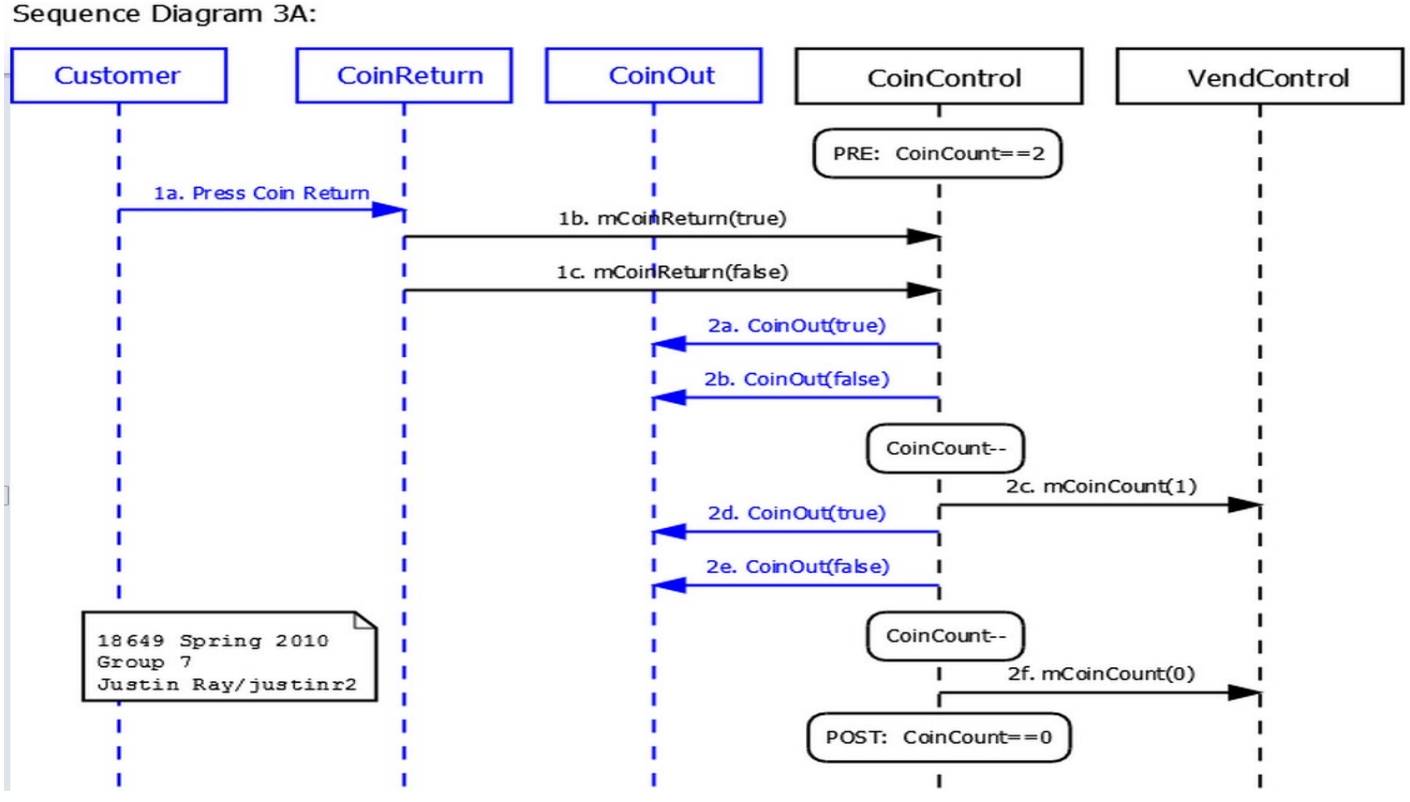


- SD is a partial behavioral description for objects
  - Generally, each object participates in multiple SDs; each SD only has some objects
  - The set of all SDs forms the HLD for all objects in the system

Example Sequence Diagram

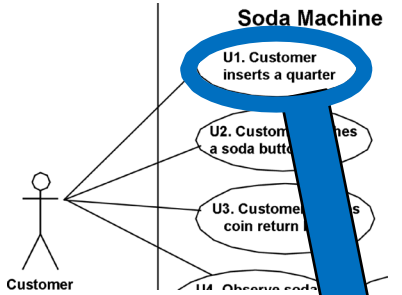
Legend: Blue = physical objects / Black = microcontrollers with software

PRE = precondition / POST = postcondition / other ovals are side effects



Use Cases to Sequence Diagrams

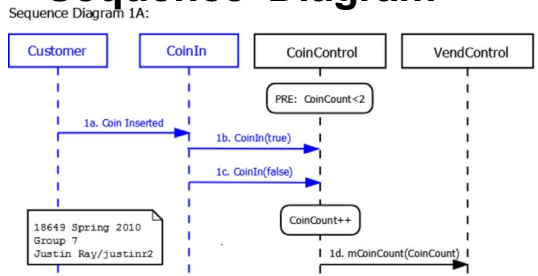
- Use Case diagram – types of interactions
  - System has multiple use cases
  - Example: Use Case #1: Insert a coin
- Scenario – a specific variant of a use case
  - Each use case has one or more scenarios
    - Scenario 1.1: insert coin to add money
    - Scenario 1.2: insert excess coin (too many inserted)
    - Scenario 1.3: … some other situation…
  - Interactions between objects are different for each scenario
- Sequence Diagram – a specific scenario design
  - For our purposes each scenario has one sequence diagram
    - Sequence diagrams 1.1, 1.2, 1.3 show specific interactions
- Statechart – design that incorporates all scenarios
  - One StateChart per object, addressing all scenarios



Pre-Conditions:
The soda machine is not vending.
No button is pressed.
The system has received fewer coins than the cost of a soda since the last vend cycle.
Scenario:
The customer inserts a coin.
Post-Conditions:
The soda machine has one additional coin for this vend cycle.

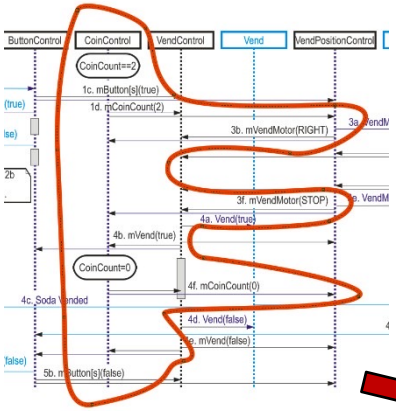
Scenario

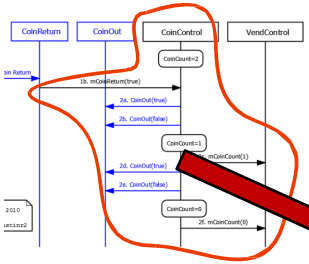
Sequence Diagram



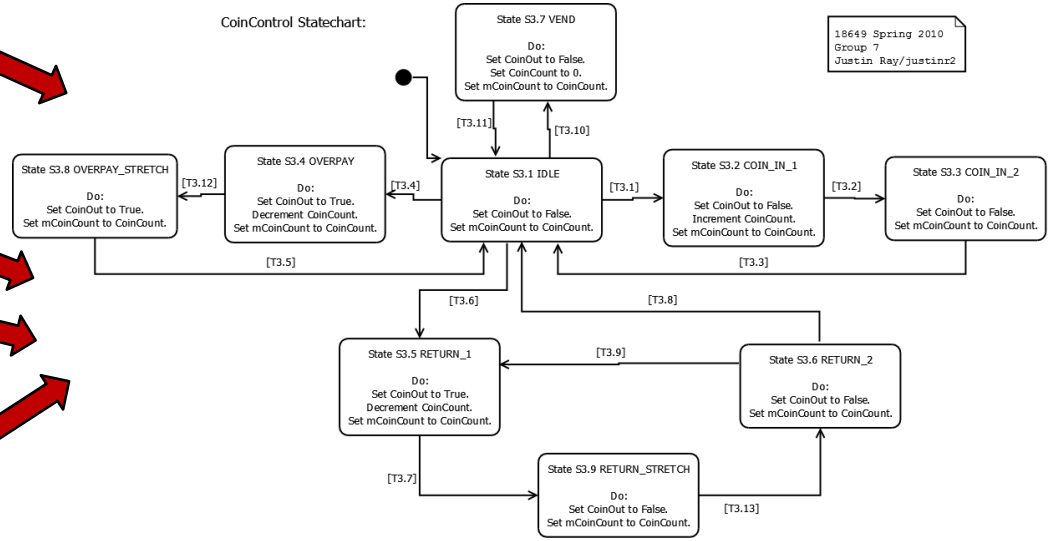
Combining SDs To Make Statecharts

- For each object in each SD: identify input & output arcs
  - Detailed Design: design statechart that accounts for all SD behaviors

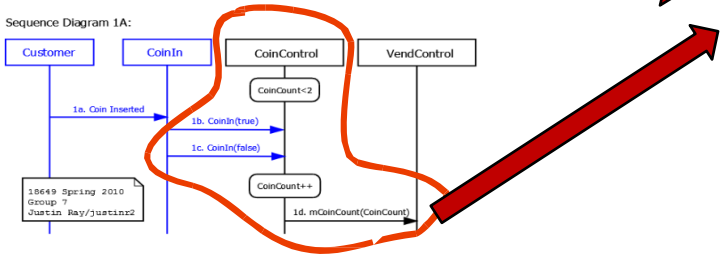




Statechart Must Exhibit All Those Behaviors

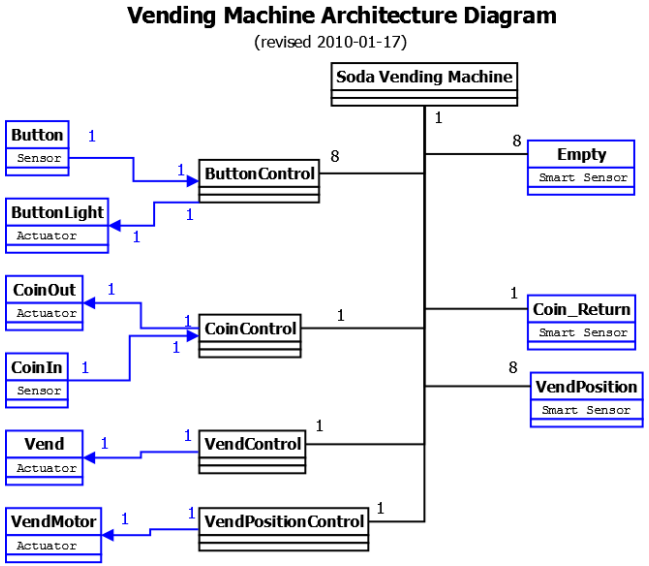


SD set specifies behaviors



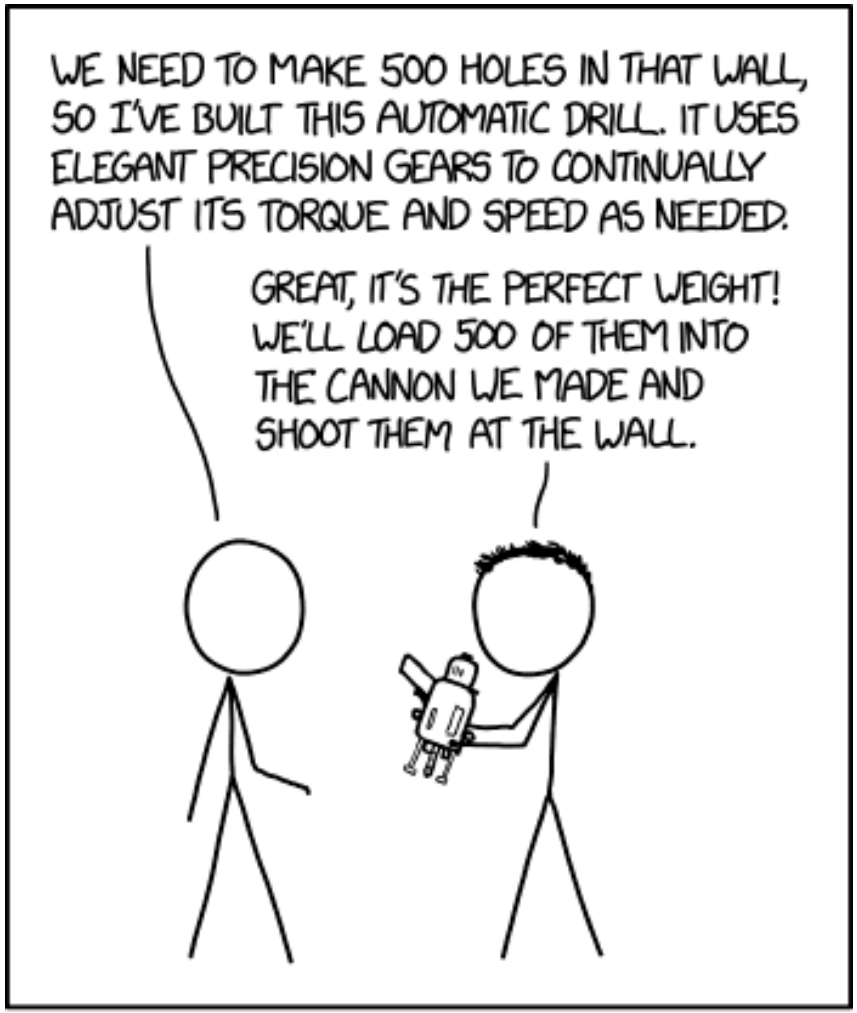
High Level Design Best Practices

- HLD should include:
  - One or more architecture diagrams
    - Defines all components & interfaces
    - HW arch., SW arch., Network arch., …
  - Sequence Diagrams
    - Both nominal and off-nominal interactions
  - HLD must co-evolve with requirements
    - Need both nouns + verbs to define a system!



- High Level Design pitfalls:
  - Diagrams that leave out interactions
  - Boxes and arrows don't have well defined meanings
  - HLD that bleeds into detailed design information
    - Should have separate Detailed Design per component

https://users.ece.cmu.edu/~koopman/ece649/project/sodamachine/index.html



HOW SOFTWARE DEVELOPMENT WORKS

https://xkcd.com/974/

Disclaimer

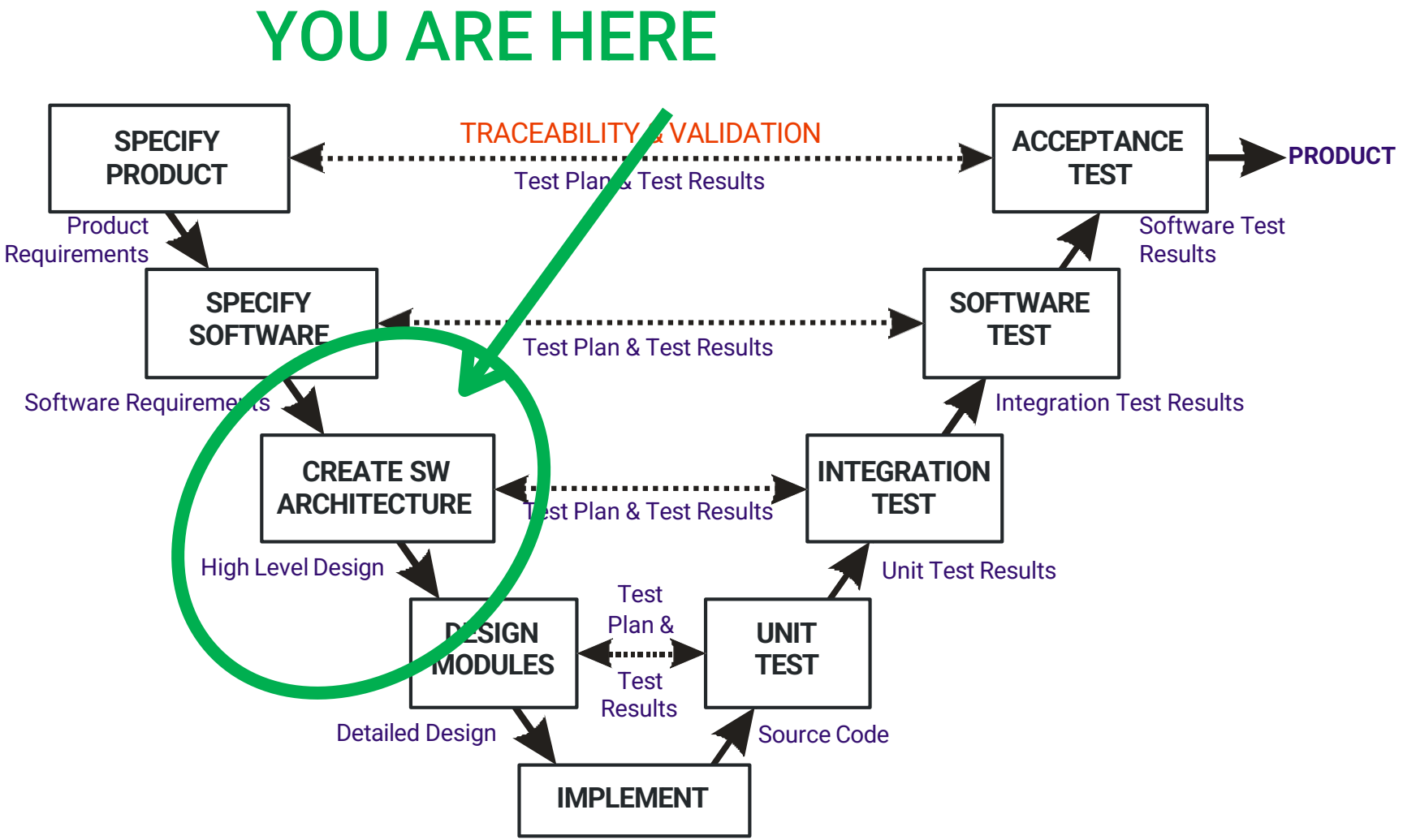
This lecture contains materials from:

- Philip Koopman - CMU

Software Architecture & High Level Design

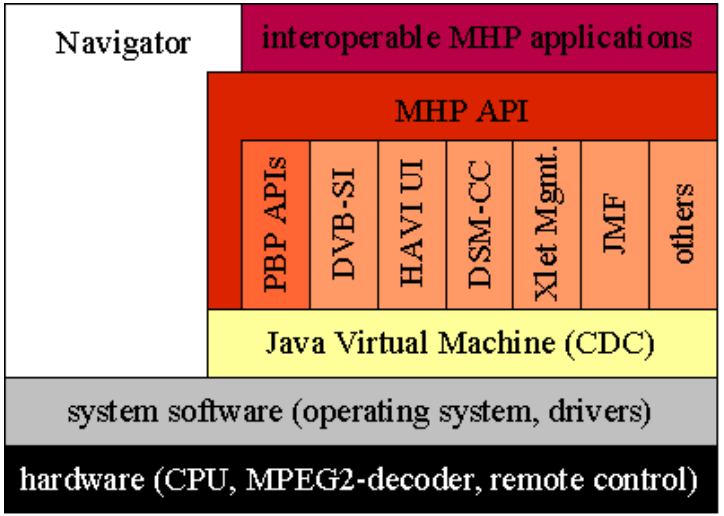
All the really important mistakes are made the first day.

– Eberhardt Rechtin,
System Architecting


YOU ARE HERE
TRACEABILITY & VALIDATION
SPECIFY PRODUCT
Test Plan & Test Results
ACCEPTANCE TEST
PRODUCT
Product Requirements
Software Test Results
SPECIFY SOFTWARE
Test Plan & Test Results
SOFTWARE TEST
Software Requirements
Integration Test Results
CREATE SW ARCHITECTURE
Test Plan & Test Results
INTEGRATION TEST
High Level Design
Unit Test Results
Test Plan &
DESIGN MODULES
Test Results
UNIT TEST
Detailed Design
Source Code
IMPLEMENT

Architecture & High Level Design (HLD)

- Anti-Patterns:
  - Skipping from requirements to code
  - No picture that shows how all the components fit together
  - “Wedding cake” layer diagram that omits interface information

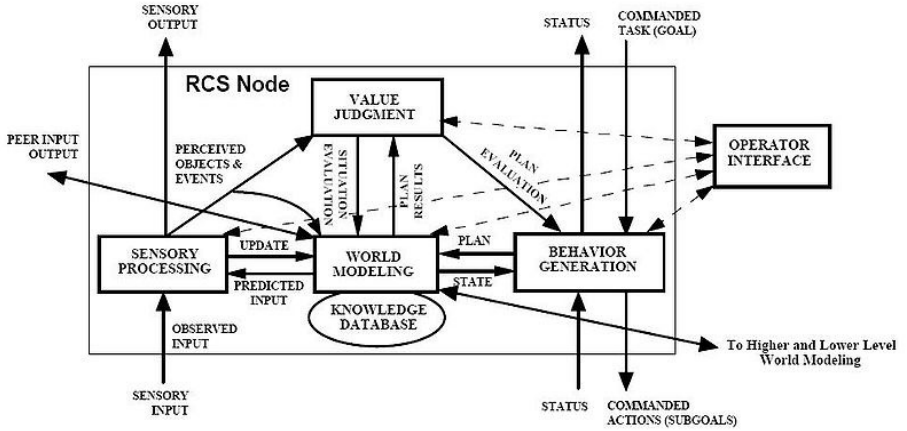


https://goo.gl/J8MAuK

- Elements of High Level Design
  - Architecture: boxes, arrows, interfaces
    - Arrows/interfaces show communication paths between components
    - Recursive: one designer’s system is another designer’s component
  - High Level Design (HLD) = architecture (nouns) + requirements (verbs)
    - Sequence Diagrams (SDs) show interactions

Architecture: Boxes and Arrows

- Software architecture
  shows the big picture
  - Boxes: software modules/objects
  - Arrows: interfaces
  - Box and arrow semantics well-defined
    - Meaning of box/arrow depends on goal
  - Components all on a single page
    - Nesting of diagrams is OK

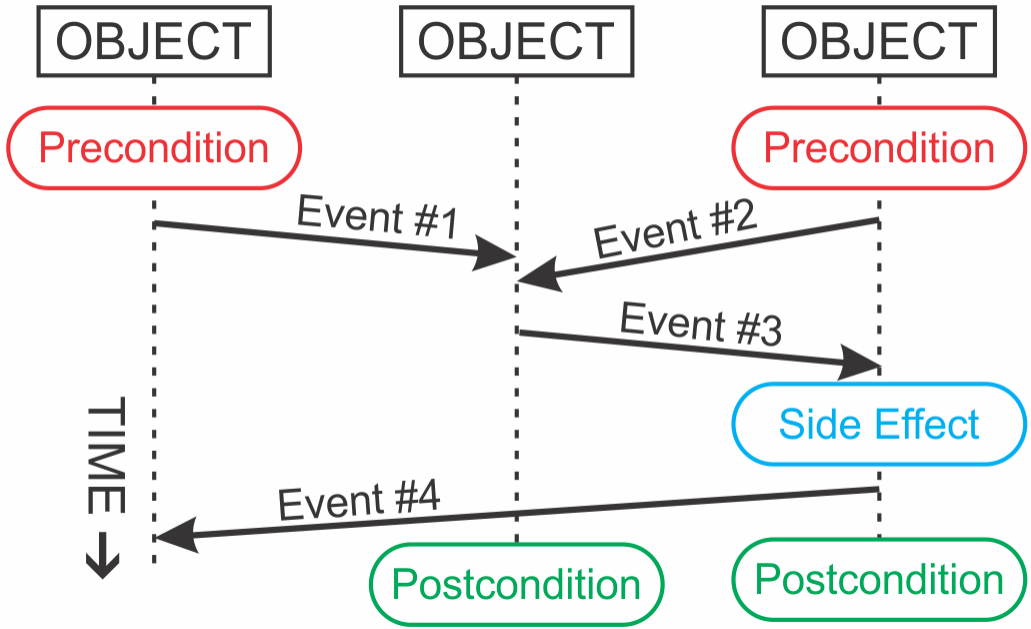


https://goo.gl/WnciF3

- Many different architecture diagrams are possible, such as:
  - Software architecture (components and data flow types)
  - Hardware architecture with software allocation
  - Controls architecture showing hierarchical control
  - Call graph showing run-time hierarchy

Sequence Diagram as HLD Notation

- SD construction:
  - Each object has a time column extending downward
  - Arcs are interactions between objects
- Each SD shows a scenario
  - Top ovals are preconditions
  - Middle ovals are side effects
  - Bottom ovals are postconditions

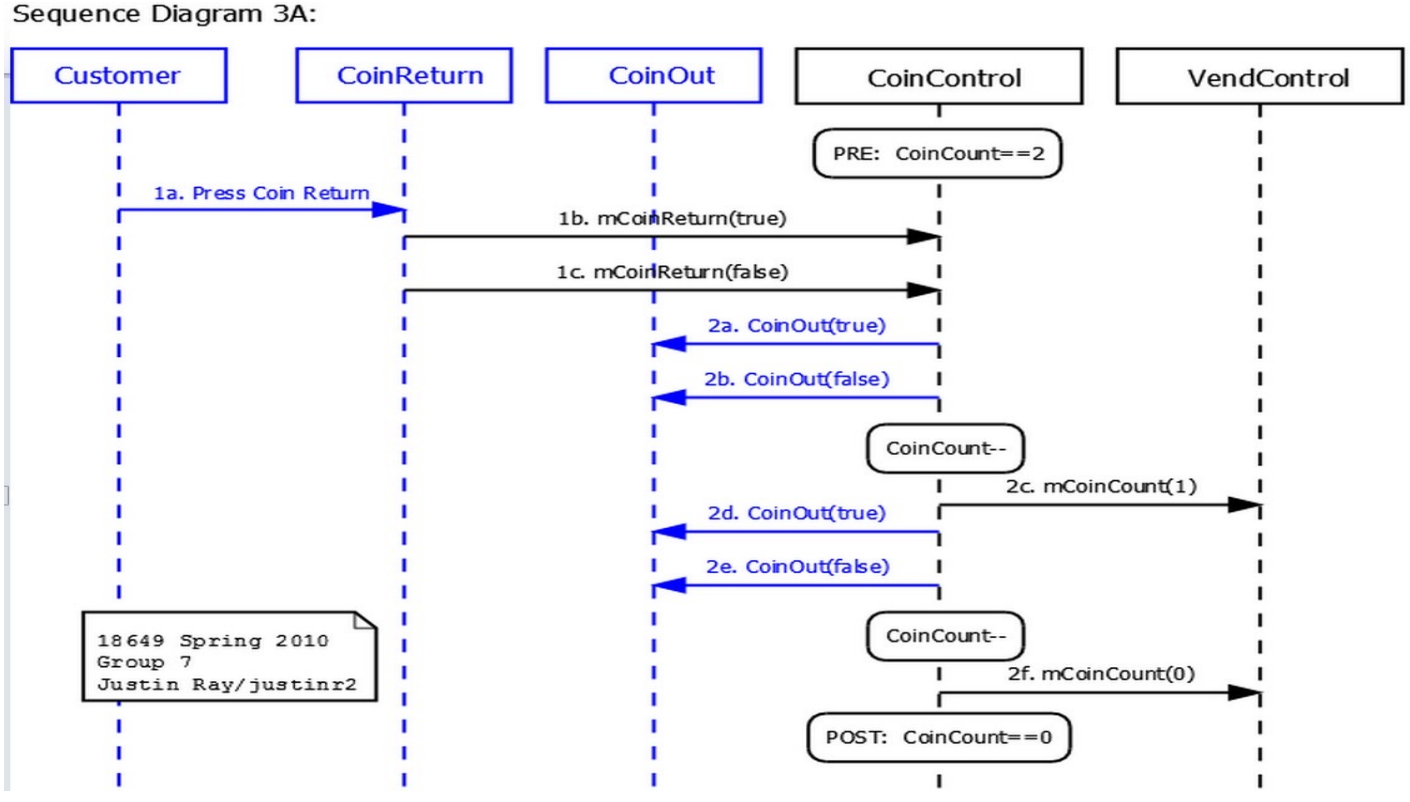


- SD is a partial behavioral description for objects
  - Generally, each object participates in multiple SDs; each SD only has some objects
  - The set of all SDs forms the HLD for all objects in the system

Example Sequence Diagram

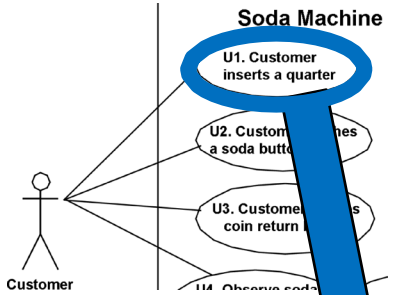
Legend: Blue = physical objects / Black = microcontrollers with software

PRE = precondition / POST = postcondition / other ovals are side effects



Use Cases to Sequence Diagrams

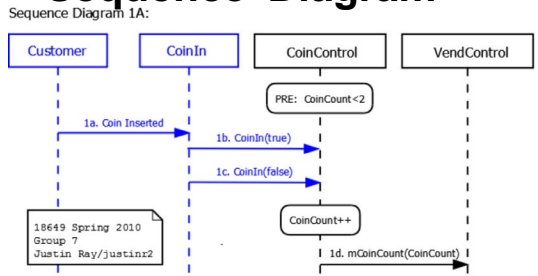
- Use Case diagram – types of interactions
  - System has multiple use cases
  - Example: Use Case #1: Insert a coin
- Scenario – a specific variant of a use case
  - Each use case has one or more scenarios
    - Scenario 1.1: insert coin to add money
    - Scenario 1.2: insert excess coin (too many inserted)
    - Scenario 1.3: … some other situation…
  - Interactions between objects are different for each scenario
- Sequence Diagram – a specific scenario design
  - For our purposes each scenario has one sequence diagram
    - Sequence diagrams 1.1, 1.2, 1.3 show specific interactions
- Statechart – design that incorporates all scenarios
  - One StateChart per object, addressing all scenarios



Pre-Conditions:
The soda machine is not vending.
No button is pressed.
The system has received fewer coins than the cost of a soda since the last vend cycle.
Scenario:
The customer inserts a coin.
Post-Conditions:
The soda machine has one additional coin for this vend cycle.

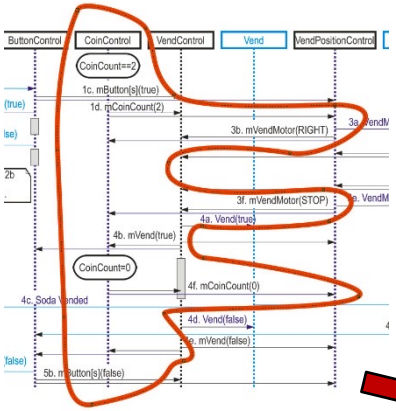
Scenario

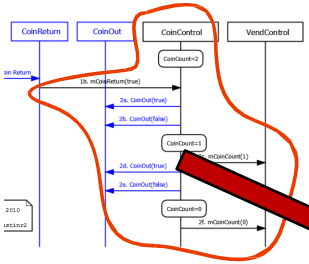
Sequence Diagram



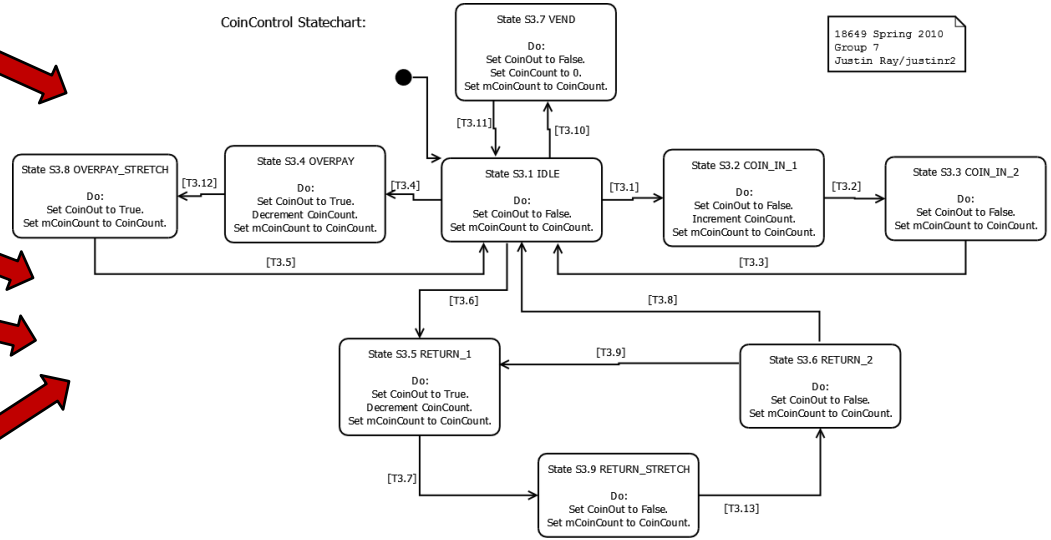
Combining SDs To Make Statecharts

- For each object in each SD: identify input & output arcs
  - Detailed Design: design statechart that accounts for all SD behaviors

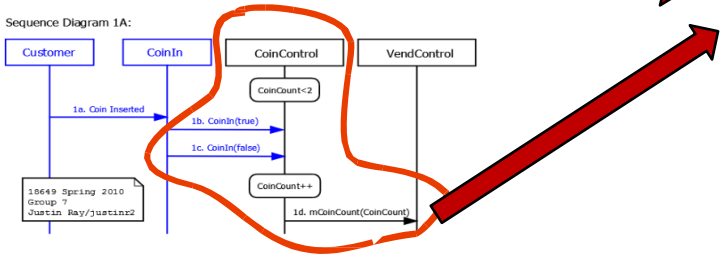




Statechart Must Exhibit All Those Behaviors

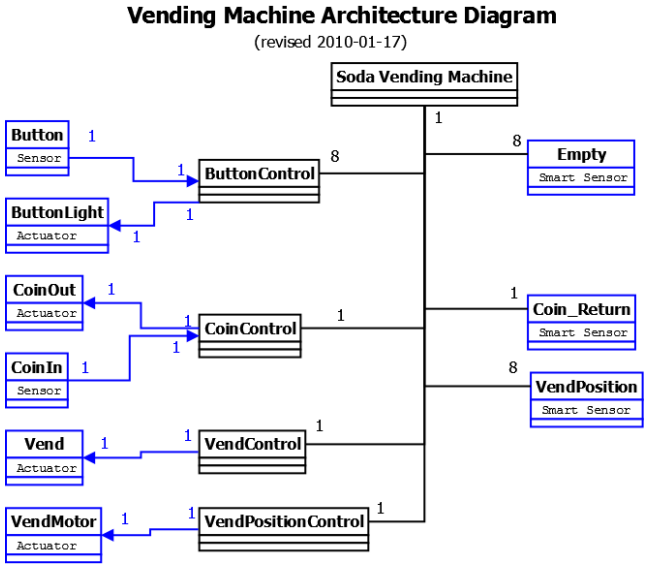


SD set specifies behaviors

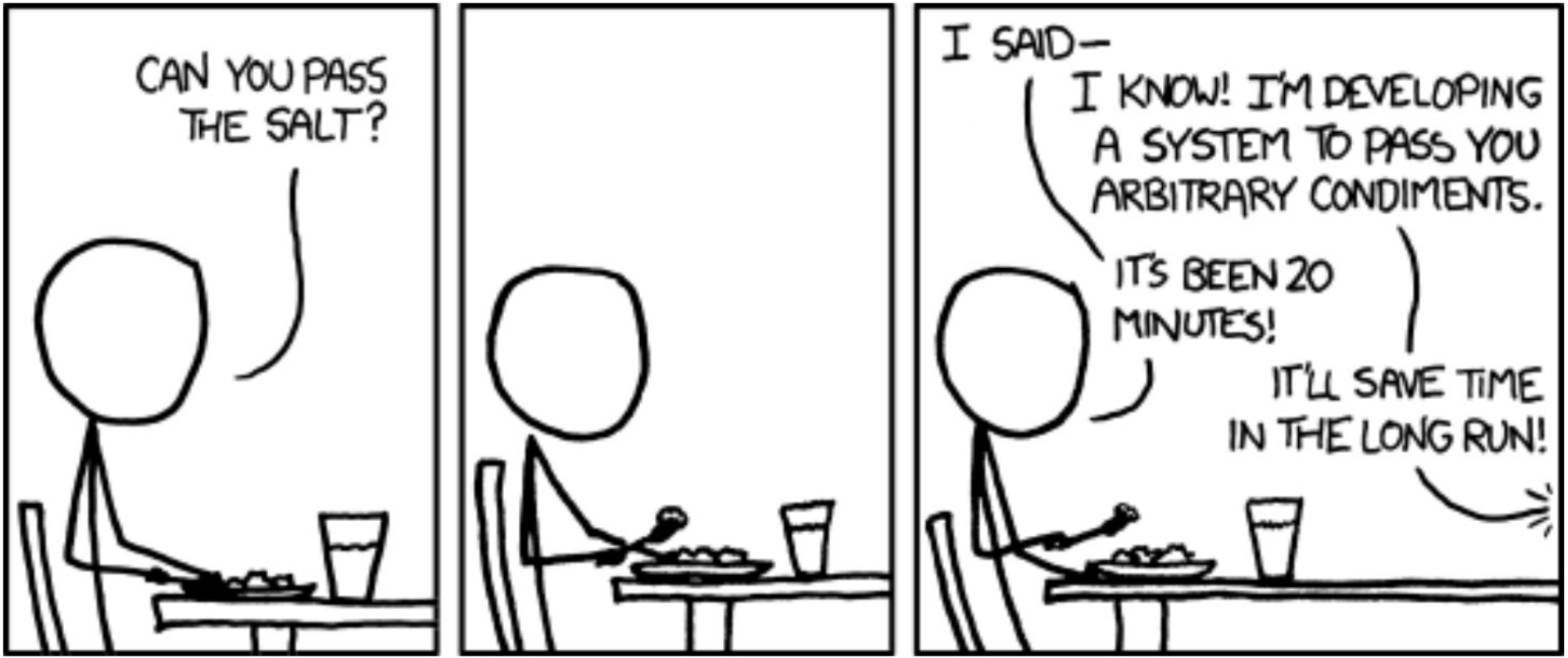


High Level Design Best Practices

- HLD should include:
  - One or more architecture diagrams
    - Defines all components & interfaces
    - HW arch., SW arch., Network arch., …
  - Sequence Diagrams
    - Both nominal and off-nominal interactions
    - See 18-649 soda machine for a fully worked example
  - HLD must co-evolve with requirements
    - Need both nouns + verbs to define a system!
- High Level Design pitfalls:
  - Diagrams that leave out interactions
  - Boxes and arrows don't have well defined meanings
  - HLD that bleeds into detailed design information
    - Should have separate Detailed Design per component



https://users.ece.cmu.edu/~koopman/ece649/project/sodamachine/index.html



https://xkcd.com/974/

Disclaimer

This lecture contains materials from:

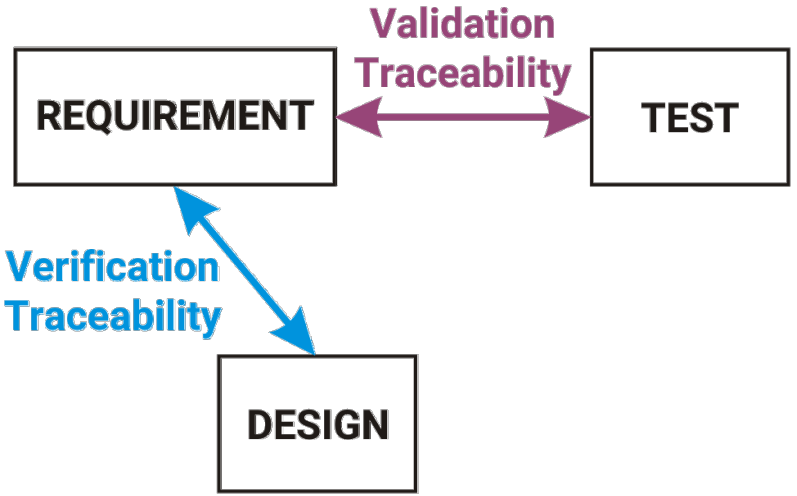
- Philip Koopman - CMU

Traceability

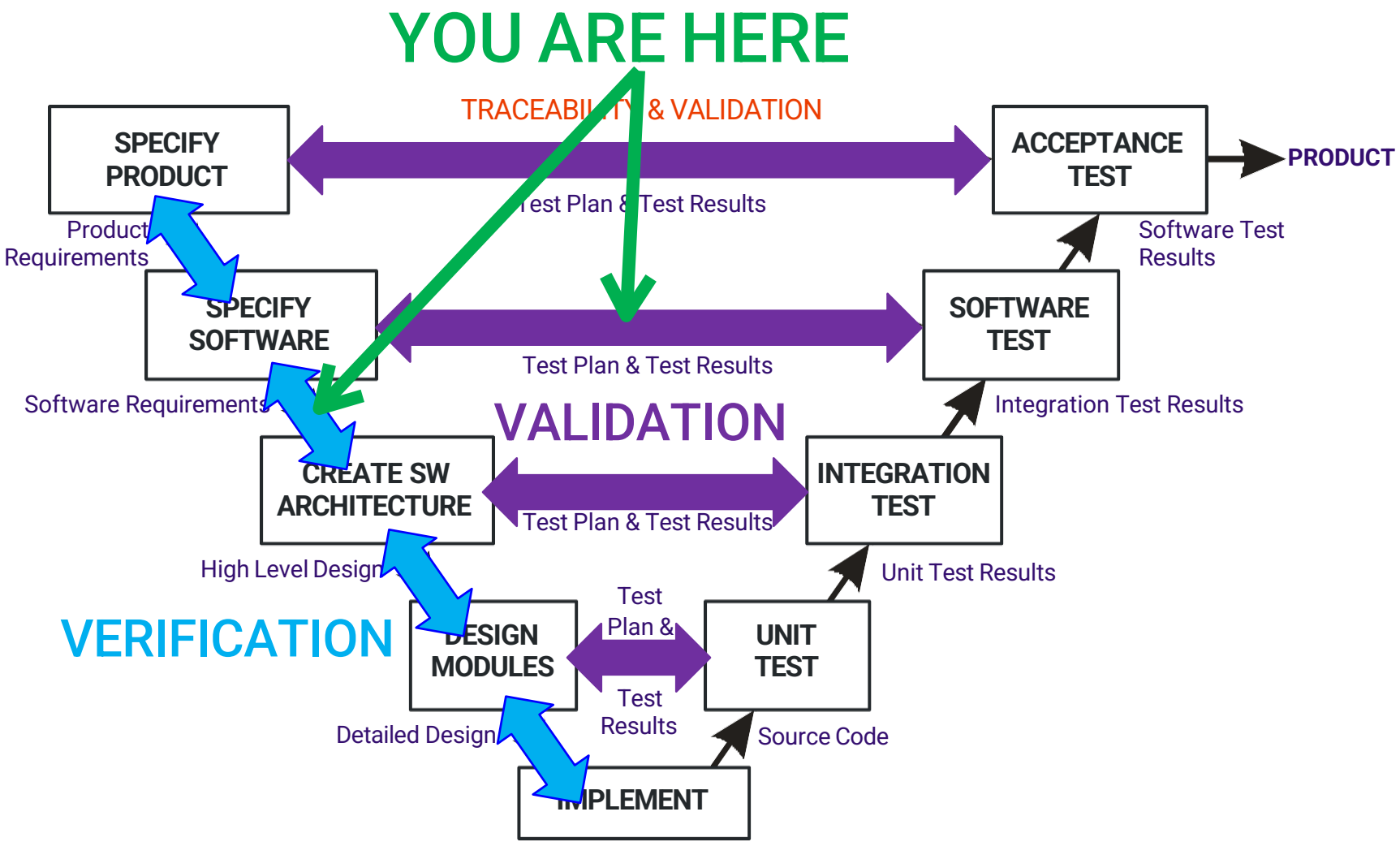
There is never enough time to do it right the first time
… but there is always enough time to do it over.

Traceability

- Anti-Patterns:
  - Tests don't map to requirements
  - Requirements aren't tested
  - Reqts/design elements missing
  - Gold plating (extra functionality)



- Traceability
  - Creating something traces to a quality check on the result
    - Verification: you did something the way you said you'd do it
    - Validation: the thing you created behaves the way it should
  - Ensure nothing left out; nothing added that shouldn't be there


YOU ARE HERE
TRACEABILITY & VALIDATION
SPECIFY PRODUCT
ACCEPTANCE TEST
PRODUCT
Test Plan & Test Results
Product Requirements
Software Test Results
SPECIFY SOFTWARE
SOFTWARE TEST
Test Plan & Test Results
Software Requirements
VALIDATION
Integration Test Results
CREATE SW ARCHITECTURE
INTEGRATION TEST
Test Plan & Test Results
High Level Design
Unit Test Results
VERIFICATION
DESIGN MODULES
Test Plan &
UNIT TEST
Test Results
Detailed Design
Source Code
IMPLEMENT

Traceability Examples

- Design traceability
  - Requirement ➔ design ➔ implementation
  - Requirement ➔ test
- SQA traceability
  - Confirm process is being carried out
  - Process step ➔ document/artifact ➔ quality metric
- Safety analysis traceability
  - Confirm all hazards successfully mitigated
  - Hazard ➔ requirement ➔ mitigation ➔ validation
- Defect traceability
  - Ensure that all bugs are fixed
  - Bug report ➔ defect identified ➔ fix task ➔ code check-in ➔ regression test

Requirements-to-Statecharts Traceability

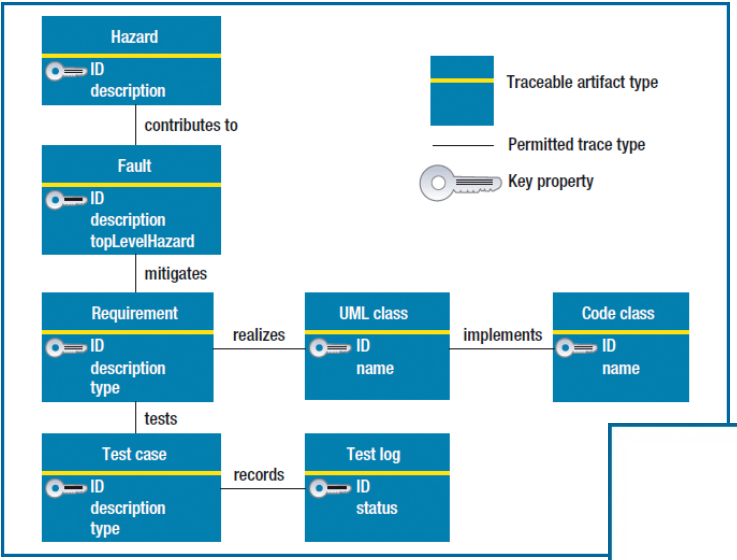
	Requirements					
States	R2.1	R2.2	R2.3	R2.4a	R2.4b	R2.5
S2.1 IDLE	x		x			x
S2.2 EMPTY	x	x				x
S2.3 VEND	x			x	x	
S2.4 FLASH_OFF	x			x	x	
S2.5 FLASH_ON	x			x	x	
Transitions						
T2.1				x	x	x
T2.2			x			
T2.3		x				
T2.4					x	
T2.5					x	
T2.6		x				
T2.7			x			

REQUIREMENTS TRACEABILITY MATRIX

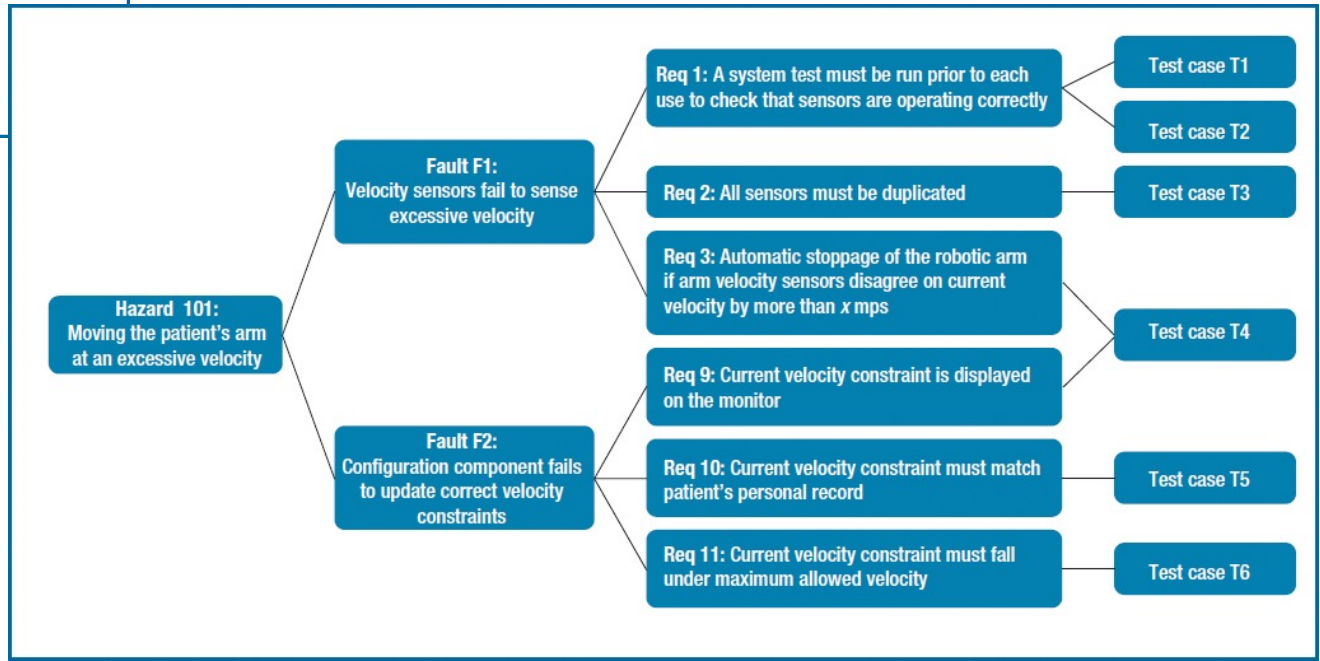
Project Name: Online Flight Booking Application

Business Requirements Document BRD		Functional Requirements Document FSD			Test Case Document
Business Requirement ID#	Business Requirement / Business Use case	Functional Requirement ID#	Functional Requirement / Use Case	Priority	Test Case ID#
BR_1	Reservation Module	FR_1	One Way Ticket booking	High	TC#001 TC#002
		FR_2	Round Way Ticket		TC#003 TC#004
		FR_3	Multicity Ticket booking	High	TC#005 TC#006
BR_2	Payment Module	FR_4	By Credit Card	High	TC#007 TC#008
		FR_5	By Debit Card	High	TC#009
		FR_6	By Reward Points	Medium	TC#010 TC#011

Traceability for Hazard Mitigation

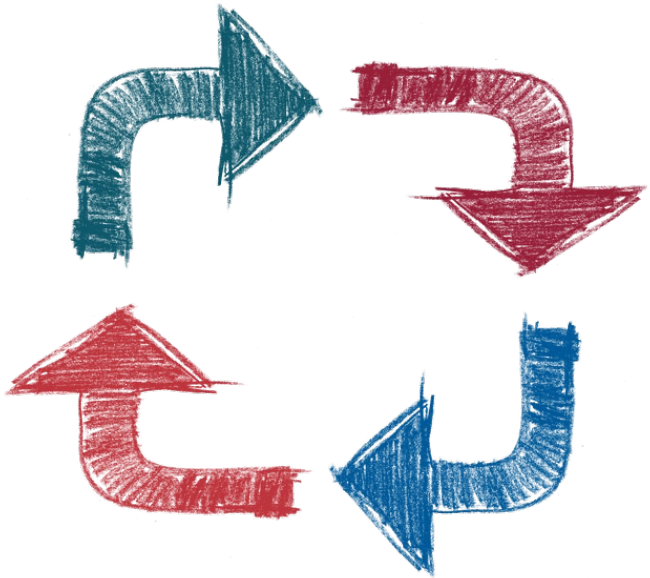


Mader et al. 2013:
http://doi.ieeecomputersociety.org/10.1109/MS.2013.60

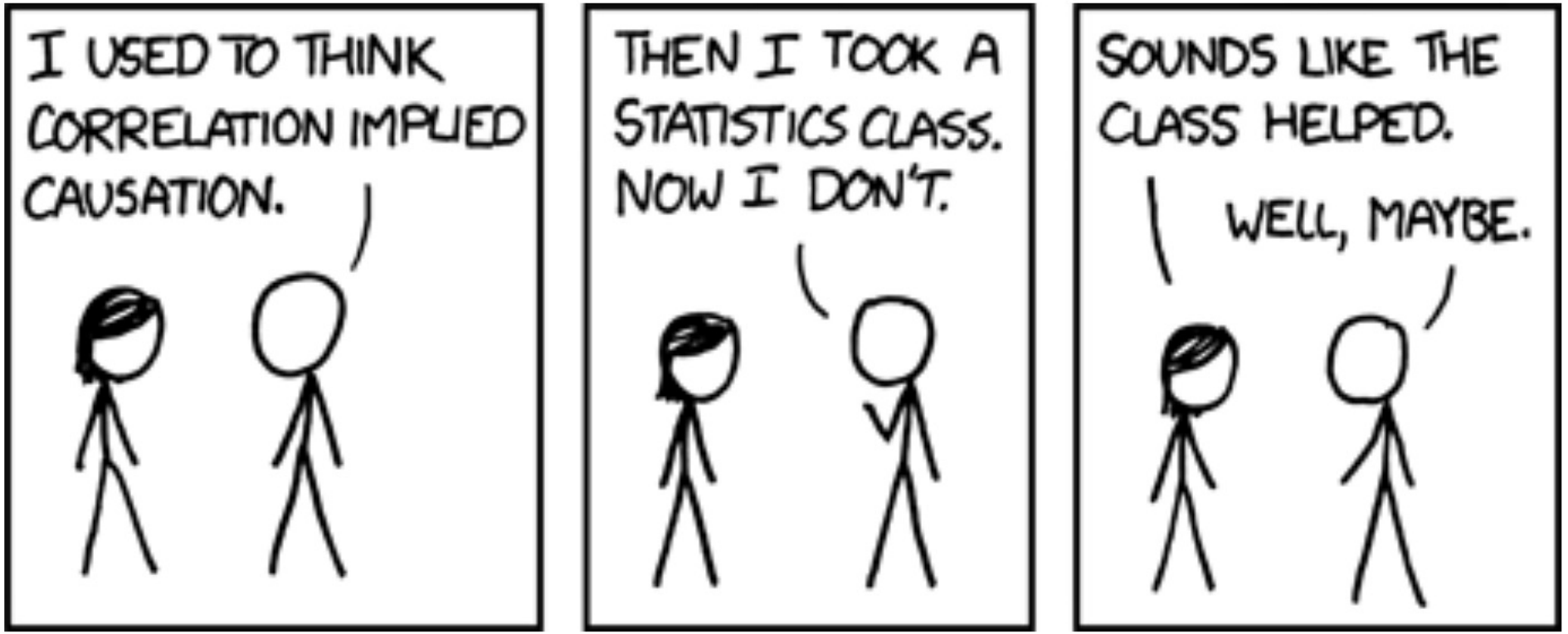


Traceability Best Practices

- Trace everything in design package
  - Even simple traceability checks can find problems
    - Gold plating: design item not traced to a requirement
  - Everything in design has an ID tag for traceability
    - Map left and right sides of V to each other
    - Map each layer of V upward and downward
  - Trace changes to see what else they affect



- Traceability pitfalls
  - Making granularity of trace IDs too big causes problems
  - Re-numbering breaks auto-generated document sections used as trace IDs
  - Don't use the wrong tool
    - Spreadsheets don't scale to big projects
    - Big project tools might be overkill where a spreadsheet approach will do



https://xkcd.com/552/

Disclaimer

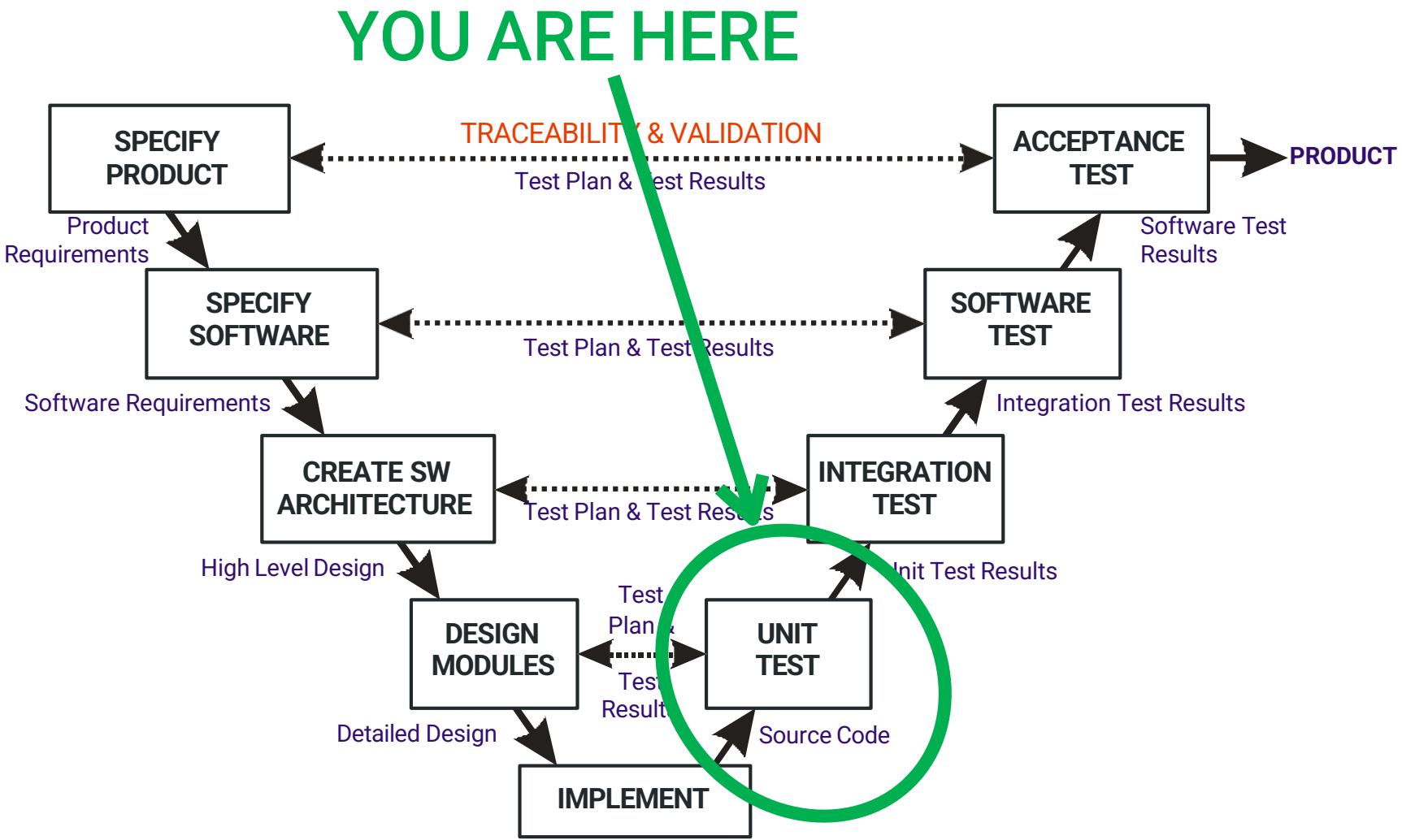
This lecture contains materials from:

- Philip Koopman - CMU

Unit Testing

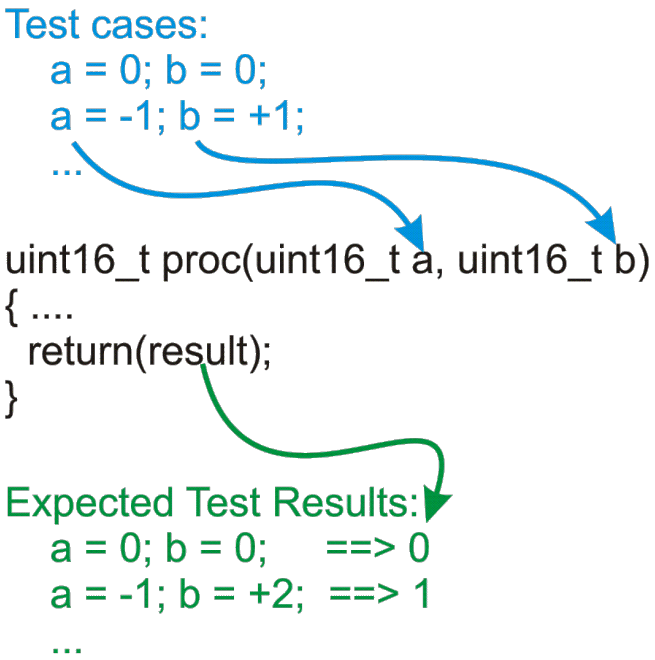
"Quality is free, but only to those who are willing to pay heavily for it."

— DeMarco & Lister


YOU ARE HERE
SPECIFY PRODUCT
TRACEABILITY & VALIDATION
Test Plan & Test Results
ACCEPTANCE TEST
PRODUCT
Product Requirements
Software Test Results
SPECIFY SOFTWARE
Test Plan & Test Results
SOFTWARE TEST
Software Requirements
Integration Test Results
CREATE SW ARCHITECTURE
Test Plan & Test Results
INTEGRATION TEST
High Level Design
Unit Test Results
Test Plan
DESIGN MODULES
UNIT TEST
Test Results
Detailed Design
Source Code
IMPLEMENT

Unit Testing

- Anti-Patterns:
  - Only system testing
  - Testing only “happy paths”
  - Forgetting to test “missing” code



- Unit testing
  - Test a single subroutine/procedure/method
    - Use low level interface (“unit” = “code module”)
  - Test both based on structure and on functionality
    - White box structural testing + Black box functional testing
  - This is the best way to catch boundary-based bugs
    - Much easier to find them here than in system testing

Black Box Testing

- Tests designed based on behavior
  - But without knowledge of implementation
  - “Functional” or behavioral testing

- Test the what, but not the how
  - Example: cruise control black box test
    - Test operation at various speeds
    - BUT, no way to tell if special cases in code have been tested
  - Advantage: can be written only based on requirements or design
  - Disadvantage: difficult to exercise all code paths

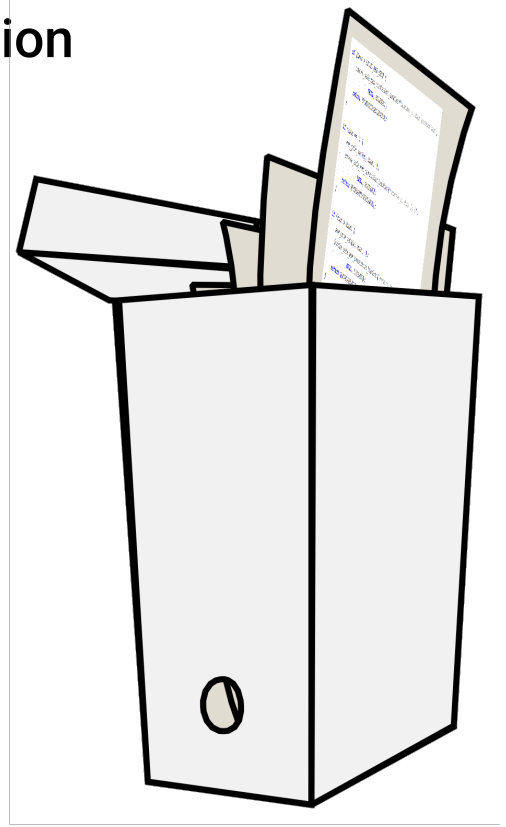
- Black box Unit Testing
  - Tests based on detailed design (statechart, flowchart)

White Box Testing

- Tests designed with knowledge of software implementation
  - Often called “structural” testing
  - Sometimes: “glass box” or “clear box”

- Idea is to exercise software <u>knowing how it is written</u>
  - Example: cruise control white box test
    - Exercise every line of code
      - » Tests that exercise both paths of every conditional branch statement
    - Test operation at every point in control loop lookup table

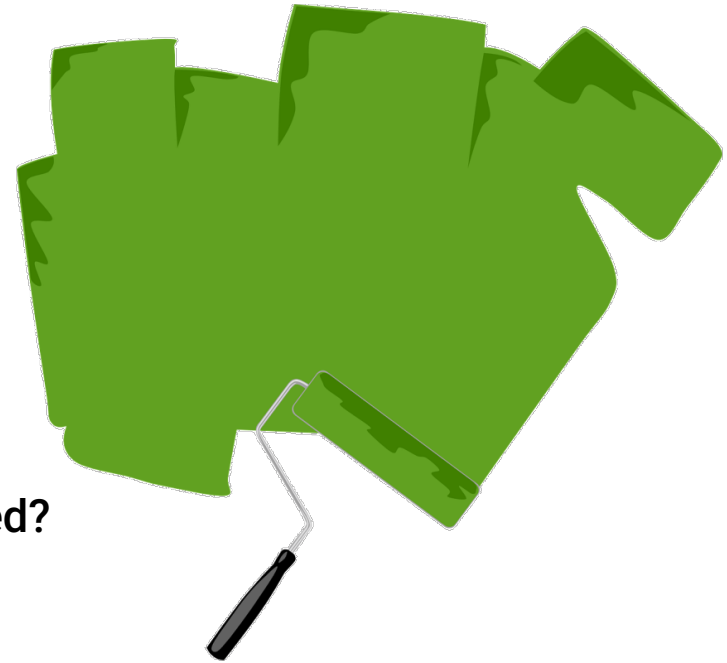
  - Advantage: helps getting high structural code coverage
  - Disadvantage: doesn’t prompt coverage of “missing” code
    - E.g., missing special case, missing exception handler



Unit Testing Coverage

Coverage is a metric for how thorough testing is

- Function coverage
  - What fraction of functions have been tested?
- Statement coverage
  - What fraction of code statements have been tested?
    - (Have you executed each line of code at least once?)
- Branch coverage (also Path Coverage)
  - Have both true and false branch paths been exercised?
  - Includes, e.g., testing the false path for if (x) { … }
- MCDC coverage (next slide)
- Getting to 100% coverage can be tricky
  - Error handlers for errors that aren't supposed to happen
  - Dead (unused) code that should be removed from source



MCDC Coverage

- Modified Condition/Decision Coverage (MC/DC)
  - Used by DO-178 for critical aviation software testing
  - Exercise all ways to reach all the code
    - Each entry and exit point is invoked
    - Each decision tries every possible outcome
    - Each condition in a decision generates all outcomes
    - Each condition in a decision is shown to independently affect the outcome of the decision

https://www.youtube.com/watch?v=DivaWCNohdw

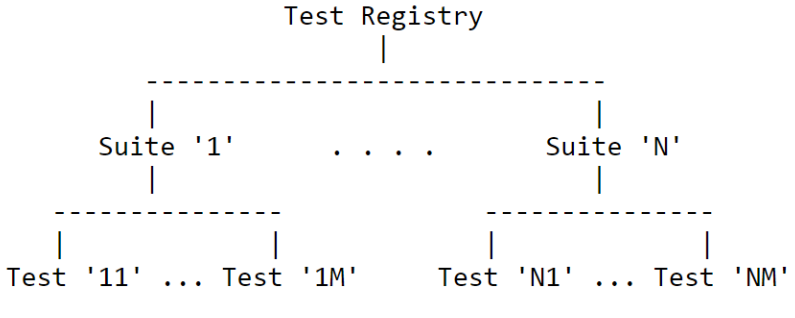
  - For example: "if (A == 3 || B == 4)" ➔ you need to test at least
    - A == 3 ; B != 4 (A causes branch, not masked by B)
    - A !=3 ; B == 4 (B causes branch, not masked by A)
    - A !=3 ; B != 4 (Fall-through case)
    - A == 3 ; B == 4 is NOT tested because it's redundant (no new information gained)
  - Might need trial & error test creation to generate 100% MCDC coverage

Unit Testing Coverage Strategies

- Boundary tests:
  - At borders of behavioral changes
  - At borders of min & max values, counter rollover
  - Time crossings: hours, days, years, …
- Exceptional values:
  - NULL, NaN, Inf, null string, …
  - Undefined inputs, invalid inputs
  - Unusual events: leap year, DST change, …
- Justify your level of coverage
  - Trace to unit design
  - Get high code coverage
  - Define strategy for boundary & exception coverage

Unit Testing Frameworks

- **Cunit as an example framework**
  - Test Suite: set of related test cases
  - Test Case: A procedure that runs one or more executions of a module for purpose of testing
  - Assertion: A statement that determines if a test has passed or failed



http://cunit.sourceforge.net/doc/introduction.html

- **Test case example:** (http://cunit.sourceforge.net/doc/writing_tests.html#tests)

```
int    maxi( int    i1,    int    i2)
{ return (i1 > i2) ? i1 : i2; }
…
void    test_maxi    (void)
{ CU_ASSERT(maxi(0,2) == 2);    // this is both a test case + assertion
    CU_ASSERT(maxi(0,  - 2) == 0);
    CU_ASSERT(maxi(2,2) == 2); }
```

Best Practices For Unit Testing

- Unit Test every module
  - Use high coverage combination of white box & black box
  - Use a unit testing framework
    - Multiple simple tests better than one huge, complex test
  - Get good coverage of data values
    - Especially, validate all lookup table entries
- Unit Testing Pitfalls
  - Creating test cases is a development effort
    - Code quality for test cases matters; test cases can have bugs!
  - Difficult to test code can lead to dysfunctional “unit test” strategies
    - Breakpoint debugging is not an effective unit test strategy
    - Using Cunit to test 100K lines of code is not really unit testing
  - Pure white box testing is “doomed to succeed” (neglects “missing” code)
  - Don’t substitute unit tests for peer reviews and static analysis

Your application is a special snowflake

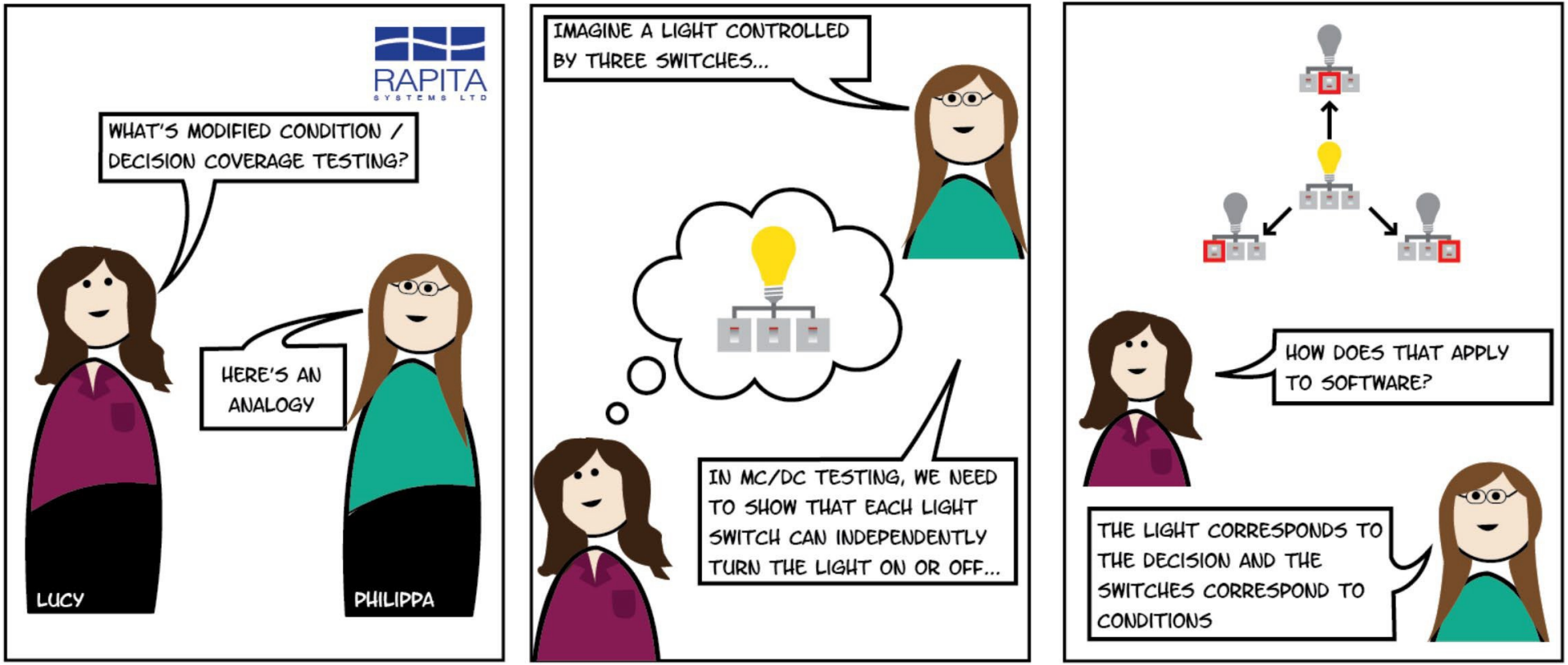
Expert

Excuses for Not Writing Unit Tests

O RLY?

@ThePracticalDev

https://goo.gl/pvDMHX CC BY-NC 2.0



www.rapitasystems.com

Disclaimer

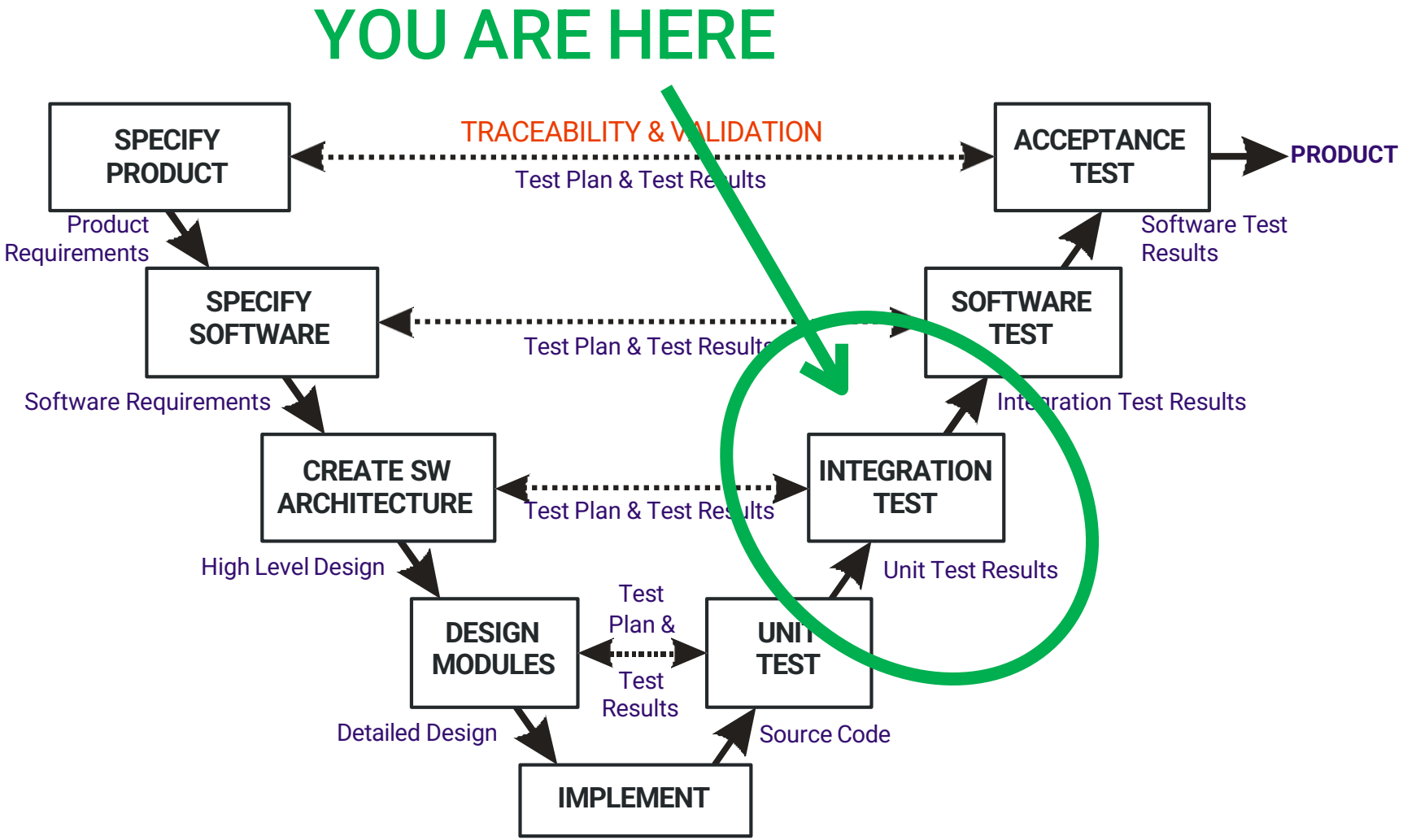
This lecture contains materials from:

- Philip Koopman - CMU

Integration Testing

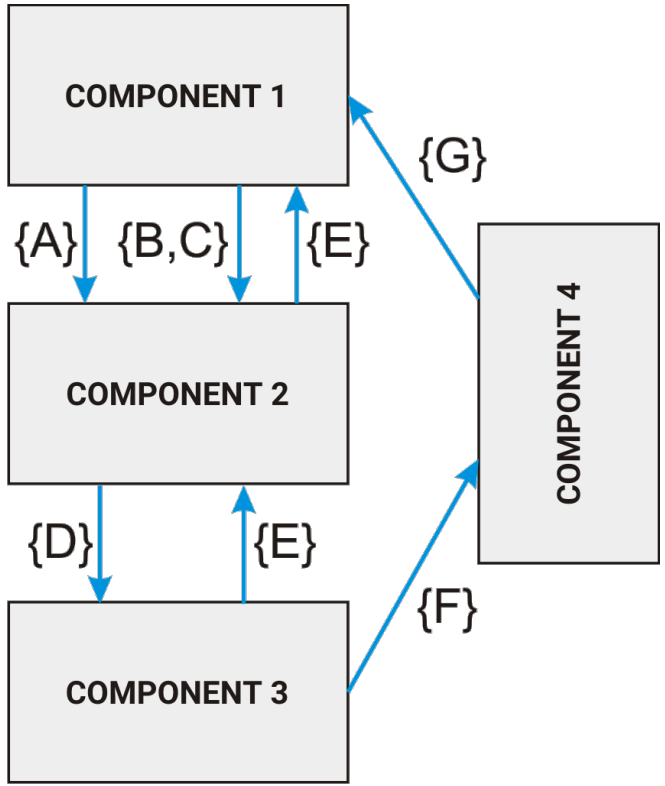
“It's hard enough to find an error in your code when you're looking for it; it's even harder when you've assumed your code is error-free.”

– Steve McConnell


YOU ARE HERE
SPECIFY PRODUCT
TRACEABILITY & VALIDATION
Test Plan & Test Results
ACCEPTANCE TEST
PRODUCT
Product Requirements
Software Test Results
SPECIFY SOFTWARE
Test Plan & Test Results
SOFTWARE TEST
Software Requirements
Integration Test Results
CREATE SW ARCHITECTURE
Test Plan & Test Results
INTEGRATION TEST
High Level Design
Unit Test Results
Test Plan &
DESIGN MODULES
Test Results
UNIT TEST
Detailed Design
Source Code
IMPLEMENT

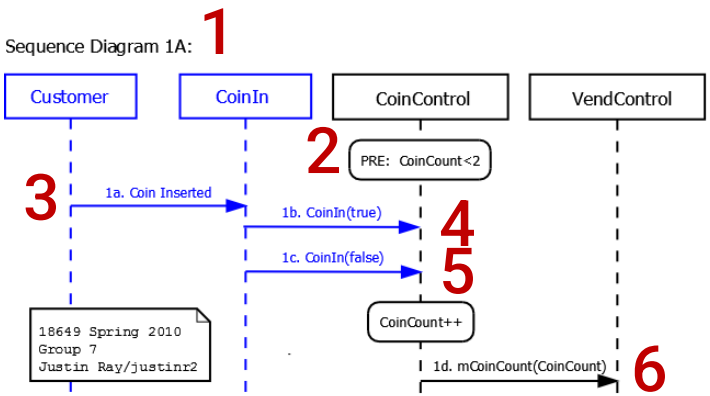
Integration Testing

- Anti-Patterns:
  - Skipping straight to system test
  - No traceability from integration test to High Level Design
  - Integration test “pass” criterion based on system function, not interfaces
- Testing component integration:
  - Exercise all component interfaces
    - Correct responses to input sequences?
    - Handle all types of data on interfaces?
  - Ensure modules match HLD, including SDs
    - Assume unit test has vetted each component
    - Concentrate on component interactions



Integration Test Approach To SDs

- **Exercise all interfaces**
  - All inputs result in correct outputs
  - Every component interface exercised
    - With all relevant values
    - With all relevant timing & sequencing
  - Use SDs and HLD info drive testing
    - Pass/fail: does it match SD?
- **Integration test coverage:**
  - All arcs on all SDs exercised?
  - Off-nominal behaviors tested?
    - Invalid sequencing and extraneous inputs?
    - Extraneous outputs?

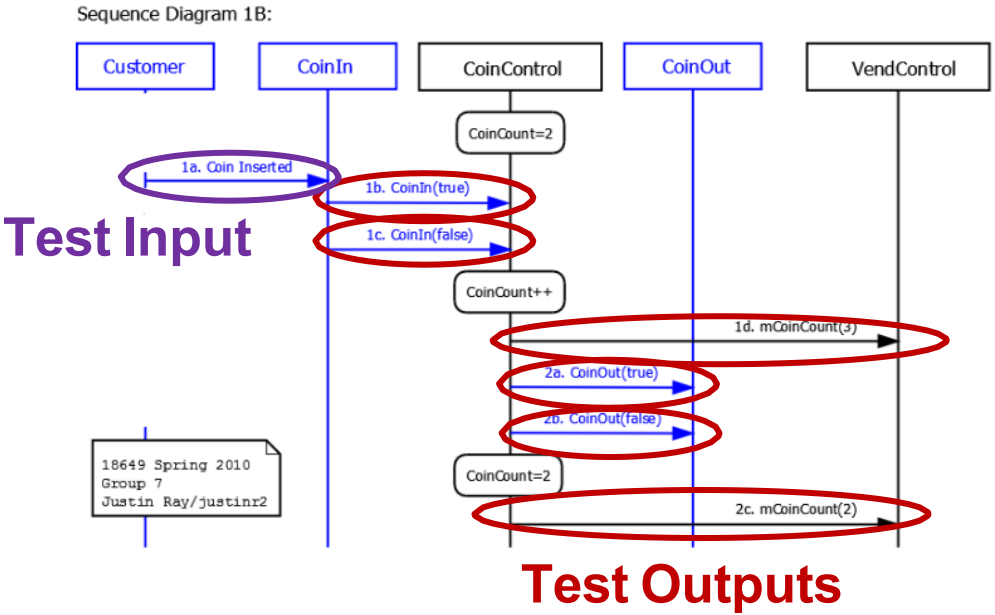


Integration Test IT-1a:

1. Initialize modules
2. Test setup: CoinCount to zero
3. Insert coin (1a)
4. Observe CoinIn(true) (1b)
5. Observe CoinIn(false (1c)
6. Observe mCoinCount == 1 (1d)

Tracing Integration Tests to SDs

- Observe module interactions
  - Set up test
    - Meet SD preconditions
  - Feed input arc(s) to modules
  - Observe intermediate arcs
  - Observe output arcs
  - Find a way to observe documented side effects (e.g., final CoinCount)



- Integration test "pass" is <u>not</u> just based on final output
  - Do all the arcs appear in expected sequence?
  - Is timing appropriate?

Integration Tests and Messaging

- Interfaces often look like “messages”
  - Categorical values (enums)
  - Data structures
  - Network packets
- Integration testing should exercise “message” structure
  - All types of messages
  - Valid and invalid field values
  - Timing, exception handling
    - e.g., bad checksum, bad sequence number
- HLD will have the message dictionary
  - Defines message types, formats, etc.
  - Accompanied by a validation test suite

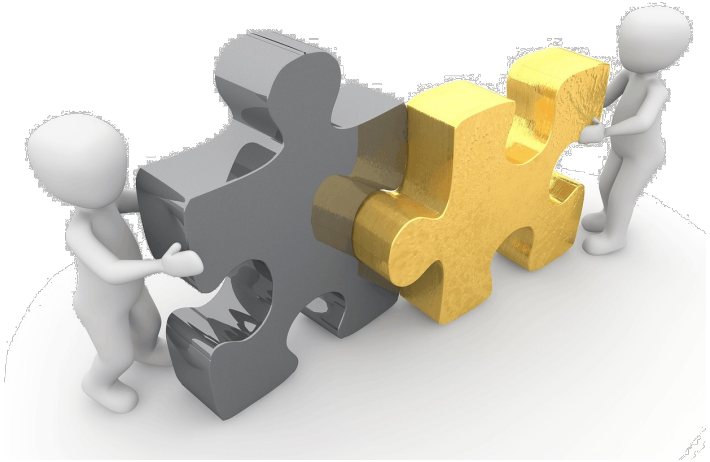
Mode 01 [edit]

PID (hex)	PID (Dec)	Data bytes returned	Description	Min value	Max value	Units	Formula[a]
00	0	4	PIDs supported [01 - 20]				Bit encoded [A7..D0] == [PID $01..PID $20] See below
01	1	4	Monitor status since DTCs cleared. (Includes malfunction indicator lamp (MIL) status and number of DTCs.)				Bit encoded. See below
02	2	2	Freeze DTC				
03	3	2	Fuel system status				Bit encoded. See below
04	4	1	Calculated engine load	0	100	%	$\frac{100}{255}A$ (or $\frac{A}{2.55}$)
05	5	1	Engine coolant temperature	-40	215	°C	$A - 40$

OBDii Parameter ID message dictionary (CAN Network Messages)
[https://en.wikipedia.org/wiki/OBD-II_PIDs]

Integration Test Best Practices

- Trace Integration tests to HLD
  - Exercise all arcs on every SD
  - Cover all modules; all interfaces
  - Cover all message types and fields



- Integration test pitfalls
  - System testing alone misses system integration edge cases
    - Sometimes a misbehaving system appears to work at system test
    - Can be difficult to exercise off-nominal SDs at system level
  - If you skip HLD, you can’t trace Integration Tests back to design

Solutions that might fix the problem without breaking anything

Essential

Hoping This Works

https://goo.gl/pvDMHX CC BY-NC 2.0

O RLY?

@ThePracticalDev

Disclaimer

This lecture contains materials from:

- Philip Koopman - CMU

Software Maintenance

“There is no code so big, twisted, or complex that maintenance can't make it worse.”

- Gerald M. Weinberg

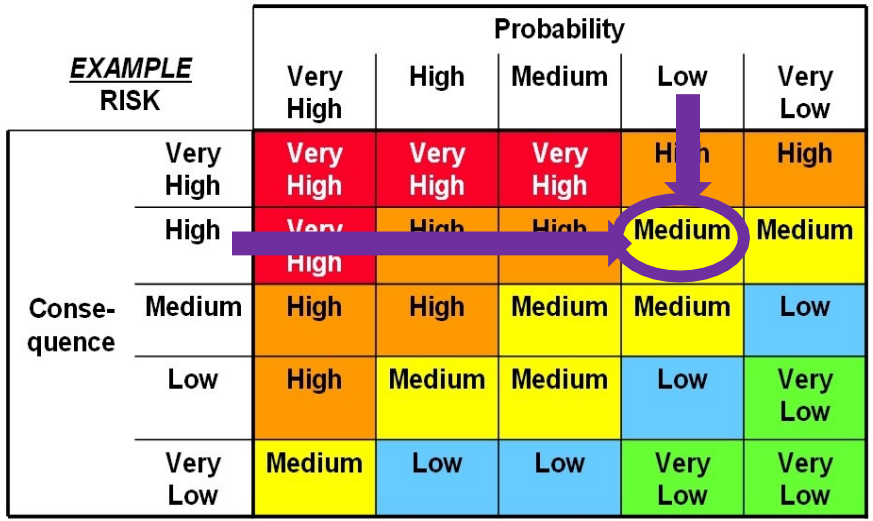
Software Maintenance

- Anti-Patterns:
  - Informal bug tracking
  - Not allocating post-release staffing
    - Bad prior release distracts team
  - Not paying off technical debt

- Code maintenance during and after development
  - You need a process to identify bugs and track to resolution
  - Most software is an update, not a clean-slate project
  - Ongoing effort is required to repay “technical debt”

Managing Bugs

- Map reported issue to an actual bug
  - L1/L2/L3 support to capture bug report
  - Sorting out duplicate reports takes effort
- Prioritize the bug fix (e.g., risk table)
  - Combination of frequency, business cost
- Find someone with right skills to fix it
  - Does this derail new development tasks?
  - Quick and dirty? Or a solid re-engineer fix?
- Validate the fix
  - Did you inject a new fault with the fix?
- Package the fix and deploy it
  - Hot patch? Defer to future schedule release?



EXAMPLE RISK		Probability: Very High	High	Medium	Low	Very Low
Consequence	Very High	Very High	Very High	Very High	Hi[illegible]	High
	High	Very High	High	High	Medium	Medium
	Medium	High	High	Medium	Medium	Low
	Low	High	Medium	Medium	Low	Very Low
	Very Low	Medium	Low	Low	Very Low	Very Low

- Risk table example:
  - High consequence defect
  - With low probability of occurrence

  ➔ Medium risk / medium priority bug

Maintenance Matters Most

- Most SW work is on existing code, not a clean slate
  - "Clean slate" often works with COTS components
- 60/60 rule [Glass, *IEEE Software* May 2001]
  - Maintenance can average 60% of lifecycle cost
  - About 60% of maintenance is adding new features
- Maintenance is harder than development
  - Need to understand existing system
    - Motivation for keeping entire V document chain up to date
    - Optimized code is more painful to maintain
  - Need to modify system without breaking things
    - Complete rewrite usually impractical – and might be worse

Managing Technical Debt

- Technical debt: messy code/design/architecture that hasn't been cleaned up
  - Some signs of debt:
    - Degraded code quality (spaghetti code, globals, warnings, …)
    - Skipped process steps (missing peer reviews, unit tests, …)
    - High fault reinjection ratio (new bugs when fixing old bugs)
  - You incur debt by taking a shortcut
    - Short-term debt can be useful (e.g., meet a deadline)
  - Repay debt by refactoring the system
- Technical debt incurs interest
  - Shortcuts often lead to bugs, fragility
  - Accumulated debt becomes unsustainable
- Use the right amount of debt
  - It's like using a credit card responsibly
  - Devote part of each development cycle to repaying technical debt

Best Practices for Maintenance

- Most development is maintenance
  - Plan for and staff maintenance
    - Most development is on the next revision
    - Plan for high priority emergency fixes
  - Keep up with technical debt payments

- Maintenance pitfalls
  - Not allocating time for bugs, maintenance & technical debt
    - For example, need perhaps 10% budget for technical debt repayment
    - Leave slack in deadlines for fixing urgent previous-version bugs
  - Evaluating programmers only for clean-sheet development skills

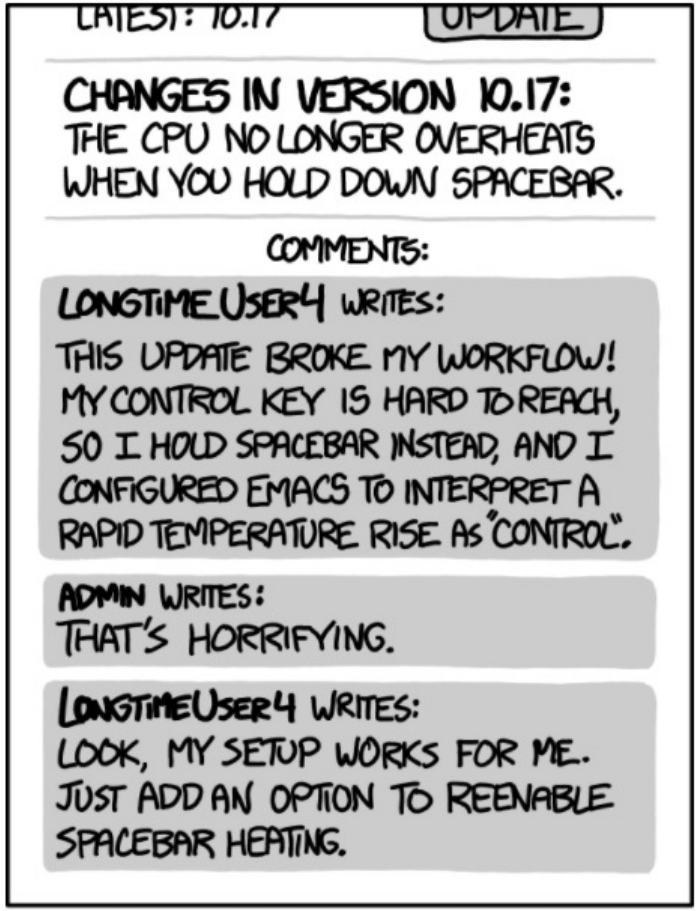
Just put the technical debt on my credit card

Moving Fast and Breaking Things

Fragile Development Guide

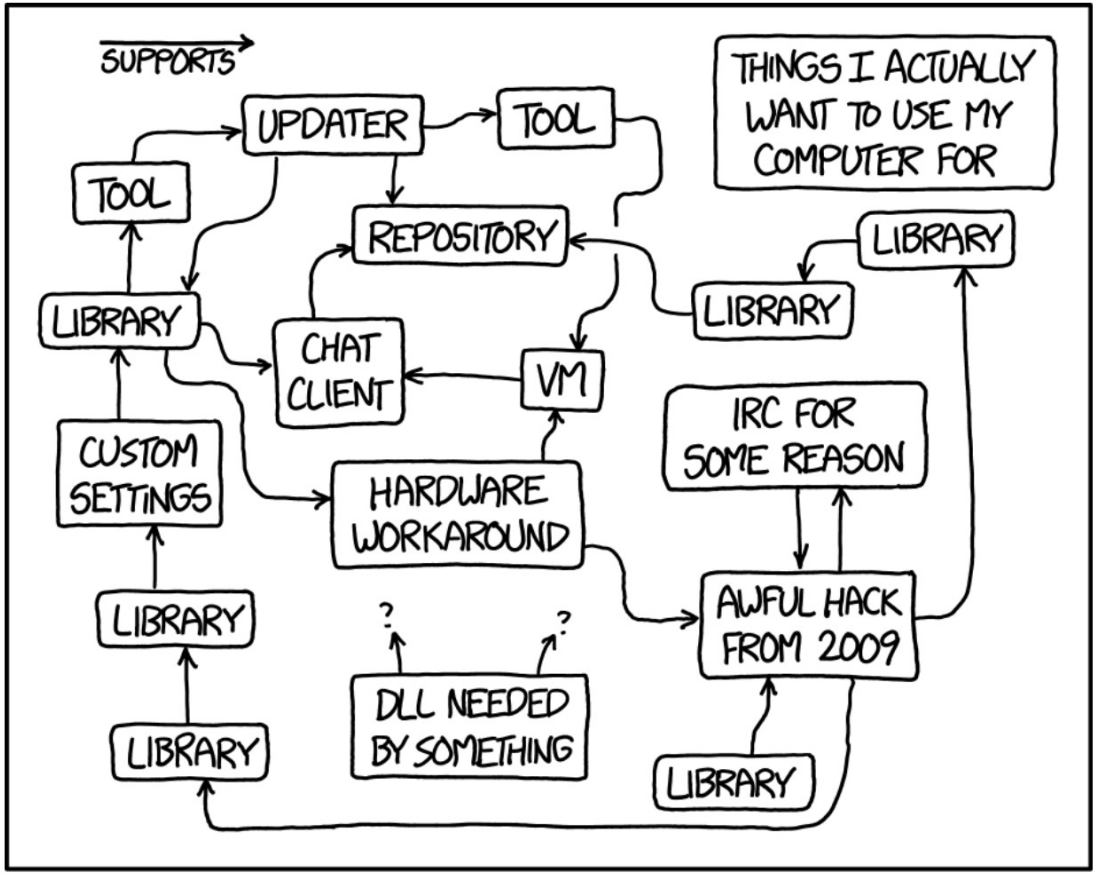
O RLY?

@ThePracticalDev



EVERY CHANGE BREAKS SOMEONE'S WORKFLOW.

https://xkcd.com/1172/



EVERY NOW AND THEN I REALIZE I'M MAINTAINING A HUGE CHAIN OF TECHNOLOGY SOLELY TO SUPPORT ITSELF.

https://xkcd.com/1579/